Podręcznik użytkownika TestGEN

Spis treści

- Podręcznik użytkownika TestGEN — spis treści
  - Plik 1 — Wprowadzenie *(Część I konspektu)*
  - Plik 2 — Praca z systemem *(Część II konspektu)*
  - Plik 3 — Zaawansowane funkcje *(Część III konspektu)*
  - Plik 4 — Administracja i rozwiązywanie problemów *(Części IV–V konspektu)*
  - Poradniki modułowe (uzupełnienie podręcznika)
- Podręcznik użytkownika — Część 1: Wprowadzenie
  - Spis treści części
  - Rozdział 1. O systemie TestGEN
  - Rozdział 2. Szybki start
- Podręcznik użytkownika — Część 2: Praca z systemem
  - Spis treści części
  - Rozdział 3. Zarządzanie projektami
  - Rozdział 4. Analiza kodu źródłowego
  - Rozdział 5. Analiza dokumentacji
  - Rozdział 6. Analiza UML i GUI
  - Rozdział 7. Analiza bezpieczeństwa
  - Rozdział 8. Generowanie testów
  - Rozdział 9. Raporty i eksport
- Podręcznik użytkownika — Część 3: Zaawansowane funkcje
  - Spis treści części
  - Rozdział 10. Integracje
  - Rozdział 11. Webhooks, API i CLI
  - Rozdział 12. Dostępność i personalizacja
- Podręcznik użytkownika — Część 4: Administracja i rozwiązywanie problemów
  - Spis treści części
- Część IV — Administracja i zarządzanie
  - Rozdział 13. Zarządzanie kontem i subskrypcją
  - Rozdział 14. Bezpieczeństwo i prywatność
- Część V — Rozwiązywanie problemów
  - Rozdział 15. FAQ
  - Rozdział 16. Troubleshooting
  - Rozdział 17. Wsparcie techniczne
- Przewodnik — nawigacja i ekrany główne
  - 1. Logowanie i start
  - 2. Górny pasek (navbar)

3. Ekrany i ich adresy
4. Błędy i stany brakowe
5. Aplikacja mobilna/desktopowa

Przewodnik — projekty i upload plików

1. Tworzenie projektu
2. Upload plików (drag & drop)
3. Dokumenty i źródła projektu
4. Uruchomienie analizy
5. Porządkowanie

Przewodnik — wyniki analiz i test case'y

1. Przeglądarka wyników
2. Zarządzanie test case'ami
3. Statusy i priorytety
4. Limity

Przewodnik — raporty i eksport

1. Raporty dwupoziomowe
2. Formaty eksportu
3. Eksport test case'ów do TMS
4. Powiązania

Przewodnik — dostępność w praktyce

1. Co gwarantuje platforma
2. Wysoki kontrast
3. Redukcja animacji
4. Nawigacja klawiaturą — skróty i wzorce
5. Czytniki ekranu
6. Zgłaszanie problemów z dostępnością

Przewodnik — plany i zakup subskrypcji

1. Porównanie planów
2. Zakup planu (Owner)
3. Moduły zablokowane planem
4. Zmiana planu

Przewodnik — zarządzanie subskrypcją

1. Szczegóły subskrypcji
2. Zmiana planu (FR55)
3. Anulowanie subskrypcji
4. Reaktywacja
5. Nieudane odnowienie (dunning)

Przewodnik — metody płatności

1. Lista metod
2. Dodanie karty (bez obciążenia)
3. Usuwanie metody
4. Która metoda płaci przy odnowieniu
5. Bank nie obsługuje Płatności Powtarzalnych

Przewodnik — faktury i dane fakturowe

1. Faktury
2. Dane fakturowe (Owner)

3. Kiedy faktura może nie przyjść mailem?
4. History transakcji

Przewodnik — zużycie zasobów i powiadomienia

1. Dashboard zużycia
2. Prognoza wyczerpania limitu
3. Powiadomienia o progach
4. Co się dzieje po wyczerpaniu limitu
5. Widoczność

FAQ — moduł billingowy

Płatności
Subskrypcja
Limity i faktury

Przewodnik użytkownika — uruchamianie skanu

1. Typy skanów
2. Uruchomienie skanu (API)
3. Śledzenie statusu
4. Skan DAST — wymagania
5. SBOM (tylko DEPENDENCY)
6. Dostęp z UI

Przewodnik użytkownika — interpretacja wyników

1. Pola znaleziska (SecurityFinding)
2. Severity vs CVSS
3. Confidence — jak czytać
4. Raport zagregowany
5. Dashboard
6. Filtrowanie znalezisk

Przewodnik użytkownika — priorytetyzacja napraw

1. Jak liczony jest priorytet
2. Szacowanie wysiłku (effort)
3. Eskalacja severity
4. Sekwencja napraw (dashboard)
5. Praca z patchami zbiorczymi
6. Triage statusu

Przewodnik użytkownika — rekomendacje, patche i PoC

1. Rekomendacje naprawcze
2. Patche (git apply)
3. Patche zbiorcze i konflikty
4. Proof of Concept (PoC)
5. Bezpieczeństwo PoC (poc_safety)

Przewodnik użytkownika — FAQ bezpieczeństwa

Pytania ogólne
Jakość wyników
Naprawy
Troubleshooting

Uruchamianie analizy luk

Wymagania wstępne

- Krok 1: Otwórz Project Detail
- Krok 2: Uruchom analizę luk
- Krok 3: Monitoruj postęp
- Krok 4: Interpretuj status
- Krok 5: Automatyzacja (opcjonalne)
- Rozwiązywanie problemów
- Następne kroki

Interpretacja wyników analizy luk
- Widok tabeli findings
- Filtry
- Klasyfikacje luk — kiedy się pojawiają
- Sortowanie i priorytetyzacja
- Eksport
- Panel podsumowania
- Następne kroki

Compliance i priorytetyzacja
- Widok Compliance
- Eksport CSV
- Snapshoty i porównanie wersji
- Priorytetyzacja napraw
- Następne kroki

Rekomendacje i skeleton generator
- Widok Rekomendacji
- Skeleton generator
- Tryby pracy z rekomendacjami
- Konfidencja rekomendacji
- Następne kroki

Integracje z trackerami (Jira, Azure DevOps)
- Dostęp
- Konfiguracja Jira
- Konfiguracja Azure DevOps
- Sync znalezisk
- Mapowanie pól
- Status sync
- Deduplikacja
- Bezpieczeństwo
- Troubleshooting
- Następne kroki

FAQ — moduł luk funkcjonalnych
- Uruchamianie
- Wyniki
- Compliance
- Skeleton
- Integracje
- Wydajność
- Bezpieczeństwo

Następne kroki
Indeks terminów podręcznika
Glosariusz TestGEN
A — analiza, integracje
B — billing
D—F
G—I
J—M
N—P
R—S
T—Ż

Podręcznik użytkownika TestGEN — spis treści

> Podręcznik składa się z czterech plików o ciągłej numeracji rozdziałów (1–17), obejmujących pięć części konspektu: Wprowadzenie, Praca z systemem, Zaawansowane funkcje, Administracja i zarządzanie oraz Rozwiązywanie problemów. Czytaj po kolei od Części 1 albo skocz do rozdziału z listy. Definicje pojęć: Glosariusz; szybka nawigacja: Indeks terminów.

Plik 1 — Wprowadzenie *(Część I konspektu)*

- **Rozdział 1. O systemie TestGEN** — 1.1. Czym jest TestGEN · 1.2. Kluczowe funkcjonalności · 1.3. Korzyści dla użytkowników · 1.4. Jak działa analiza · 1.5. Plany · 1.6. Wymagania systemowe
- **Rozdział 2. Szybki start** — 2.1. Rejestracja i pierwsze logowanie · 2.2. Pierwszy pulpit · 2.3. Konfiguracja pierwszego projektu · 2.4. Pierwsza analiza kodu · 2.5. Pierwsze testy · 2.6. Pierwszy raport · 2.7. Interfejs w pigułce

Plik 2 — Praca z systemem *(Część II konspektu)*

- **Rozdział 3. Zarządzanie projektami** — 3.1. Tworzenie i konfiguracja · 3.2. Lista, filtry i sortowanie · 3.3. Karta projektu · 3.4. Członkowie zespołu i role · 3.5. Edycja, usuwanie, archiwizacja · 3.6. Dobre praktyki · 3.7. Ograniczenia
- **Rozdział 4. Analiza kodu źródłowego** — 4.1. Wgrywanie kodu (pliki, ZIP, repozytorium) · 4.2. Inne źródła materiałów · 4.3. Konfiguracja i uruchomienie · 4.4. Postęp · 4.5. Preprocessing · 4.6. Interpretacja wyników · 4.7. Testy regresji kodu · 4.8. Przykłady dla różnych języków · 4.9. Dobre praktyki · 4.10. Ograniczenia
- **Rozdział 5. Analiza dokumentacji** — 5.1. Wspierane formaty · 5.2. Wgrywanie · 5.3. Wymagania · 5.4. Niespójności · 5.5. Glosariusz · 5.6. Porównywanie dokumentacji z kodem (luki, compliance) · 5.7. Generowanie testów z dokumentacji · 5.8. Dobre praktyki · 5.9. Ograniczenia
- **Rozdział 6. Analiza UML i GUI** — 6.1. Wgrywanie diagramów UML · 6.2. Widoki diagramów · 6.3. Analiza interfejsów użytkownika · 6.4. Generowanie testów UI (Gherkin/BDD) · 6.5. Testy regresji GUI · 6.6. Macierz śledzenia · 6.7. Dobre praktyki · 6.8. Ograniczenia
- **Rozdział 7. Analiza bezpieczeństwa** — 7.1. Typy skanów · 7.2. Uruchamianie · 7.3. Interpretacja wyników · 7.4. Zależności i SBOM · 7.5. Rekomendacje naprawcze · 7.6. Priorytetyzacja · 7.7. Dobre praktyki · 7.8. Ograniczenia
- **Rozdział 8. Generowanie testów** — 8.1. Wybór frameworka · 8.2. Przeglądanie i edycja przypadków · 8.3. Statusy i przegląd · 8.4. Operacje masowe · 8.5. Generowanie kodu testów · 8.6. Eksport testów · 8.7. Dobre praktyki · 8.8. Ograniczenia

- **Rozdział 9. Raporty i eksport** — 9.1. Poziomy raportów · 9.2. Generowanie i podgląd · 9.3. Formaty plików · 9.4. Integracja z systemami zewnętrznymi · 9.5. Dobre praktyki · 9.6. Ograniczenia

Plik 3 — Zaawansowane funkcje *(Część III konspektu)*

- **Rozdział 10. Integracje** — 10.1. Co z czym łączy TestGEN · 10.2. Jira · 10.3. Azure DevOps · 10.4. Synchronizacja wyników · 10.5. GitHub i GitLab · 10.6. Status checki w pull requestach · 10.7. Konfiguracja CI/CD · 10.8. TestRail, Xray i Zephyr · 10.9. Synchronizacja dwukierunkowa z TMS · 10.10. Import wymagań z Jiry · 10.11. Ograniczenia
- **Rozdział 11. Webhooks, API i CLI** — 11.1. Zasady dostępu · 11.2. Uwierzytelnianie · 11.3. Limity żądań · 11.4. Format błędów · 11.5. Najczęściej używane endpointy · 11.6. Przykłady (curl i Python) · 11.7. Praca z CLI · 11.8. Webhooki wychodzące · 11.9. Ograniczenia
- **Rozdział 12. Dostępność i personalizacja** — 12.1. Dostępność (WCAG 2.1 AA) · 12.2. Tryb wysokiego kontrastu i animacje · 12.3. Skróty klawiszowe · 12.4. Wygląd na różnych ekranach · 12.5. Personalizacja interfejsu · 12.6. Ograniczenia

Plik 4 — Administracja i rozwiązywanie problemów *(Części IV–V konspektu)*

Część IV — Administracja i zarządzanie:

- **Rozdział 13. Zarządzanie kontem i subskrypcją** — 13.1. Profil · 13.2. Użytkownicy firmy · 13.3. Plany i zmiana subskrypcji · 13.4. Płatności (tpay) · 13.5. Faktury · 13.6. Zużycie limitów i powiadomienia · 13.7. Usuwanie konta · 13.8. Ograniczenia
- **Rozdział 14. Bezpieczeństwo i prywatność** — 14.1. Hasło · 14.2. Weryfikacja dwuetapowa (MFA) · 14.3. Izolacja danych firmy · 14.4. RODO i prywatność danych · 14.5. Dostęp API i klucze · 14.6. Ograniczenia

Część V — Rozwiązywanie problemów:

- **Rozdział 15. FAQ** — 15.1. Konto i dostęp · 15.2. Projekty · 15.3. Analiza kodu · 15.4. Dokumentacja, UML i GUI · 15.5. Generowanie testów · 15.6. Bezpieczeństwo · 15.7. Eksport i raporty · 15.8. Integracje · 15.9. Płatności i plany · 15.10. Błędy i problemy
- **Rozdział 16. Troubleshooting** — 16.1. Problemy z analizą · 16.2. Problemy z generowaniem testów · 16.3. Problemy z integracjami · 16.4. Problemy z wydajnością · 16.5. Problemy z dostępem i kontem
- **Rozdział 17. Wsparcie techniczne** — 17.1. Jak uzyskać pomoc · 17.2. Kanały wsparcia · 17.3. Zgłaszanie błędów · 17.4. Propozycje nowych funkcji

Poradniki modułowe (uzupełnienie podręcznika)

- Interfejs webowy — nawigacja · projekty i upload · wyniki i test case'y · raporty i eksport · dostępność
- Rozliczenia — plany i zakup · subskrypcja · metody płatności · faktury · zużycie i powiadomienia · FAQ
- Bezpieczeństwo — skan · wyniki · priorytety · rekomendacje i patche · FAQ
- Luki funkcjonalne — analiza · wyniki · compliance · rekomendacje i skeleton · integracje · FAQ

Podręcznik użytkownika — Część 1: Wprowadzenie

> **Wersja:** 2.0 (2026-09-20) · **Odbiorcy:** użytkownicy platformy TestGEN (analitycy QA, deweloperzy, kierownicy projektów) · **Część 1 z 4** (całość: 02 — Praca z systemem, 03 — Zaawansowane funkcje, 04 — Administracja i rozwiązywanie problemów, Spis treści całego podręcznika)
>
> **Konwencje:** forma bezpośrednia („Ty"), nazwy przycisków i ekranów w cudzysłowie dokładnie tak, jak widzisz je w aplikacji, czas wykonania podany przy każdym kroku. Ramki: 💡 **Wskazówka** (oszczędza czas) i ⚠️ **Uwaga** (częsta pułapka). Definicje pojęć: Glosariusz, nawigacja po pojęciach: Indeks terminów.

Spis treści części

- **Rozdział 1. O systemie TestGEN** — 1.1. Czym jest TestGEN · 1.2. Kluczowe funkcjonalności · 1.3. Korzyści dla użytkowników · 1.4. Jak działa analiza · 1.5. Plany · 1.6. Wymagania systemowe
- **Rozdział 2. Szybki start** — 2.1. Rejestracja i pierwsze logowanie · 2.2. Pierwszy pulpit · 2.3. Konfiguracja pierwszego projektu · 2.4. Pierwsza analiza kodu · 2.5. Pierwsze testy · 2.6. Pierwszy raport · 2.7. Interfejs w pigułce

---

Rozdział 1. O systemie TestGEN

1.1. Czym jest TestGEN

TestGEN to platforma, która **automatycznie tworzy przypadki testowe i kod testów** na podstawie tego, co już masz w projekcie: kodu źródłowego, dokumentacji, diagramów UML i zrzutów ekranu interfejsu.

Jaki problem rozwiązuje? Pisanie przypadków testowych ręcznie jest wolne, powtarzalne i łatwo w nim o przeoczenia. Analityk QA spędza dni na przekładaniu wymagań na tabele kroków; deweloperzy odkładają testy „na później", bo żal im czasu na żmudne wzorce; kierownik projektu nie ma pod koniec sprintu twardej odpowiedzi na pytanie „co jest przetestowane?". TestGEN analizuje Twoje materiały i proponuje gotowe przypadki testowe w kilka minut — Ty decydujesz, co zaakceptować.

Jak wygląda praca z platformą w praktyce?

1. Zakładasz konto i firmę (bez karty płatniczej — startujesz na planie Freemium).

2. Tworzysz projekt i wgrywasz materiały: pliki kodu, archiwum ZIP, dokumentację, diagramy, zrzuty ekranu — albo podłączasz repozytorium Git.
3. Uruchamiasz analizę jednym przyciskiem; działa w tle, a Ty w tym czasie pracujesz dalej.
4. Przeglądasz wyniki: struktury kodu, wymagania, luki, podatności — i zestaw proponowanych przypadków testowych.
5. Zatwierdzasz przypadki, generujesz kod testów w swoim frameworku i eksportujesz raporty albo paczkę testów do narzędzia TMS.

Dla kogo jest TestGEN?

Rola	Korzyść
Analityk QA	przypadki testowe z wymagań i kodu w minutę, nie w dni; mniej przeklepywania, więcej testowania
Deweloper	widzi, czego brakuje w kodzie względem dokumentacji; gotowe szkielety testów bez żmudnego powtarzania wzorców
Kierownik projektu	raporty pokrycia i zgodności dla całego zespołu; twarde liczby na spotkaniach
Audytor / bezpiecznik	skany podatności z CVSS, bilans komponentów (SBOM), ścieżka dowodowa

1.2. Kluczowe funkcjonalności

- **Analiza kodu źródłowego** — rozpoznaje język i framework, mapuje funkcje, klasy i zależności (11 języków rozpoznawanych, 8 z pełną analizą struktury).
- **Analiza dokumentacji** — czyta dokumenty techniczne (SRS, podręczniki, opisy API), wyciąga wymagania funkcjonalne i niefunkcjonalne, wykrywa niespójności w tekście i buduje glosariusz pojęć.
- **Analiza UML i GUI** — rozumie diagramy klas, sekwencji, aktywności, przypadków użycia i stanów (źródła PlantUML/XMI oraz rysunki z OCR), a także zrzuty ekranu interfejsu (komponenty, układ, tekst).
- **Generowanie przypadków testowych** — kompletne kroki, oczekiwane rezultaty, priorytety; scenariusze pozytywne, negatywne i brzegowe z kodu oraz z wymagań.
- **Generowanie kodu testów** — szablony testów automatycznych dla dziesięciu frameworków (pytest, JUnit, NUnit, Mocha/Jest, Cypress, Selenium, Playwright, Appium, SikuliX, PHPUnit/Pest) oraz scenariusze Gherkin/BDD.
- **Skanowanie bezpieczeństwa** — statyczna analiza (SAST), śledzenie przepływu danych (taint), podatności zależności (CVE) z bilansem komponentów (SBOM), analiza konfiguracji i testy dynamiczne (DAST); rekomendacje napraw z gotowym kodem i plikami `.patch`.
- **Analiza luk funkcjonalnych** — porównanie dokumentacji z kodem: czego brakuje, co się rozjechało, co jest nadmiarowe; wskaźnik zgodności (compliance) z trendem w czasie.
- **Raporty i eksport** — dwupoziomowe raporty (dla zarządu i dla zespołu), eksport do PDF, HTML, CSV, Excel, JSON, YAML oraz paczki przypadków do narzędzi TMS (TestRail, Jira Xray).

💡 **Wskazówka:** nie musisz używać wszystkiego naraz. Większość zespołów zaczyna od analizy kodu i generowania przypadków testowych, a pozostałe moduły włącza według potrzeb — każdy kolejny moduł zasilany jest tymi samymi materiałami, które już wgrałeś.

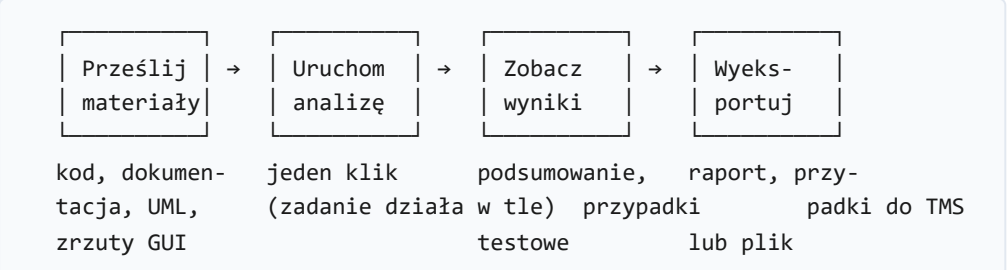
1.3. Korzyści dla użytkowników

Dlaczego zespoły sięgają po TestGEN i co z tego realnie mają:

Obszar	Bez TestGEN	Z TestGEN
Przypadek testowy z wymagania	20–40 minut przekładania	minuta przeglądu propozycji
Pokrycie wymagań testami	ręczna tabela w arkuszu	macierz śledzenia liczona automatycznie
Regresja po zmianach	„uruchamiamy wszystko, na wszelki wypadek"	zestaw regresyjny z porównania wersji
Zgodność kodu z dokumentacją	dyskusje przy kawie	wskaźnik compliance 0–100 z listą luk
Podatności zależności	nikt nie śledzi wersji	skan CVE z wersją naprawczą
Onboarding nowego QA	tygodnie czytania kodu	podręcznik + glosariusz projektu generowane z materiałów

Korzyść jest największa, gdy materiały już istnieją — TestGEN nie wymaga zmiany sposobu pracy zespołu, tylko czyta to, co i tak piszecie (kod, dokumentację, diagramy).

1.4. Jak działa analiza — z perspektywy użytkownika



Trzy zasady, które warto zapamiętać na start:

- **Analiza działa asynchronicznie** — uruchamiasz ją jednym kliknięciem i możesz w tym czasie pracować dalej; o zakończeniu informuje komunikat na ekranie.
- **Silniki analizy są deterministyczne** — te same dane wejściowe dają ten sam wynik, co miesiąc temu i dziś. Wyniki można więc porównywać między wersjami i śledzić trend.
- **Wszystko zaczyna się od materiałów** — jakość wyników zależy od jakości kodu, dokumentacji i diagramów, które wgrasz. Na śmieciach nie da się zbudować dobrego testu — garbage in, garbage out.

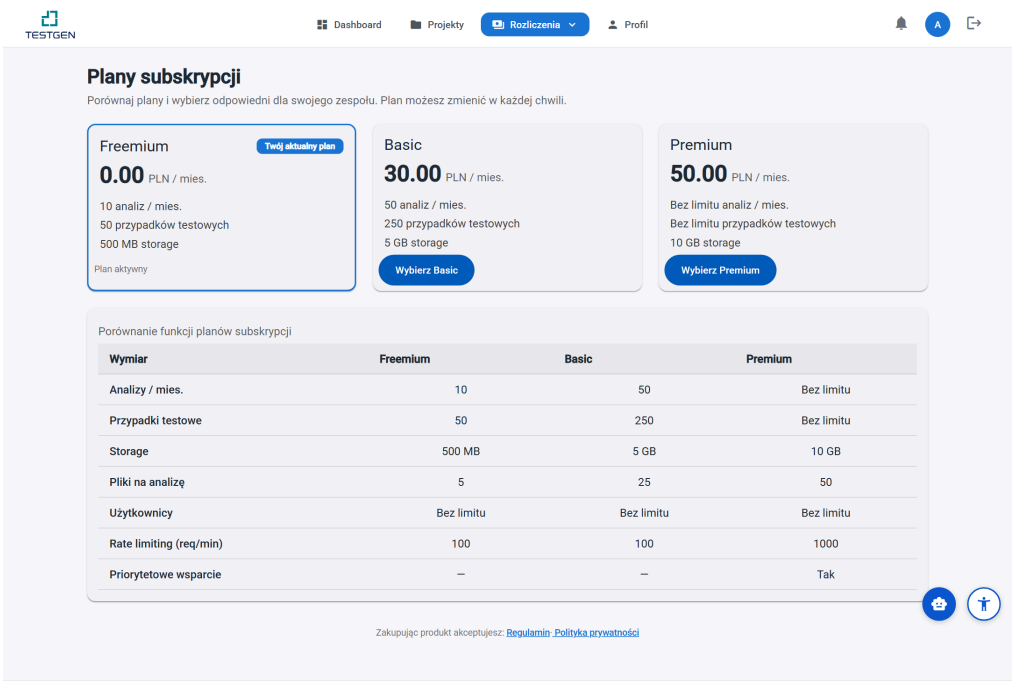
1.5. Plany

TestGEN działa w modelu miesięcznym. Nowa firma startuje na planie **Freemium** — bez płacenia, od razu.

	Freemium	Basic	Premium
Cena (miesiąc)	0 zł	30 zł	50 zł
Analizy / miesiąc	10	50	bez limitu
Przypadki testowe / miesiąc	50	250	bez limitu
Miejsce na pliki	0,5 GB	5 GB	10 GB
Plików na analizę	5	25	50
Analiza kodu + generowanie testów	✓	✓	✓
Dokumentacja, UML, GUI, CI/CD	—	✓	✓
Bezpieczeństwo + luki funkcjonalne	—	—	✓
Priorytetowe wsparcie	—	—	✓

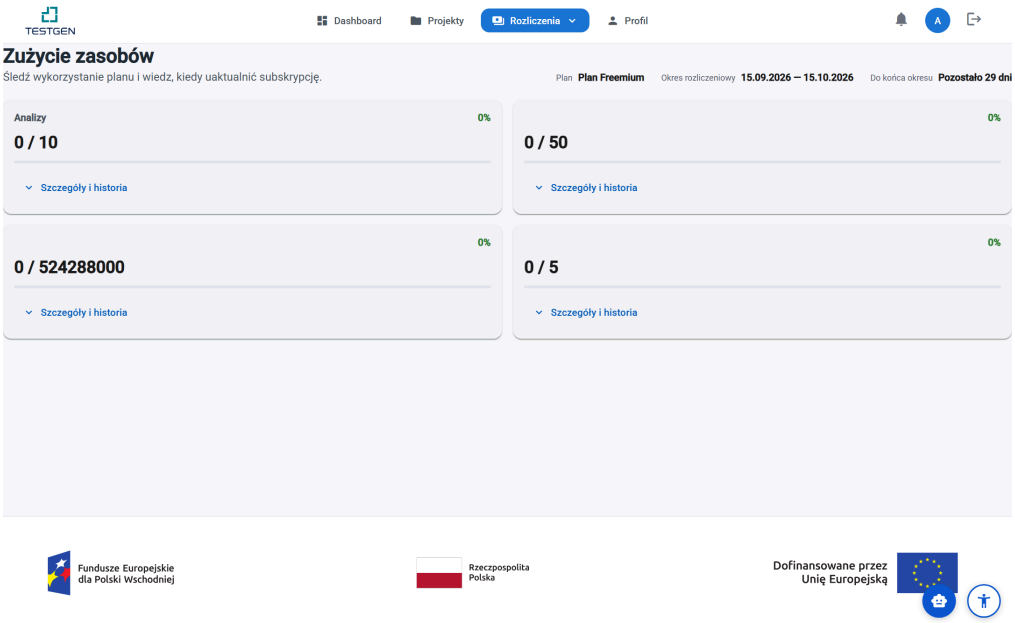
Moduły spoza Twojego planu są w aplikacji **widoczne, ale zablokowane** (kłódka) — zawsze wiesz, co zyskujesz wyższym planem. Zmiana planu w każdej chwili; dopłata przy przejściu na wyższy plan liczona jest proporcjonalnie.

Szczegóły rozliczeń: Rozliczenia — przewodnik.



Porównanie planów: menu „Rozliczenia" → zakładka z planami.

Bieżące zużycie limitów swojej firmy sprawdzisz w menu **„Rozliczenia" → „Zużycie zasobów"**:



Zużycie zasobów: liczniki z paskami postępu i prognozą wyczerpania limitu.

1.6. Wymagania systemowe

- **Przeglądarka:** aktualne wersje Chrome, Edge, Firefox lub Safari. Aplikacja działa też na ekranach dotykowych.
- **Telefon i tablet:** pełna wersja serwisu w przeglądarce mobilnej — układ składa się do jednej kolumny, menu chowa się pod przyciskiem (szczegóły i zrzuty w rozdziale 12, poświęconym dostępności i personalizacji).
- **Dostępność:** interfejs spełnia standard WCAG 2.1 AA — obsługę klawiaturą i czytnikami ekranu opisuje rozdział 12.1.
- **Konto:** zakładasz adresem e-mail — nie potrzebujesz kont Google ani Apple; weryfikacja dwuetapowa (TOTP) jest opcjonalna i zalecana dla Właścicieli firm.
- **Materiały:** pliki na dysku lub repozytorium Git (github.com, gitlab.com, bitbucket.org) — listy obsługiwanych formatów szukaj w rozdziale 4.

Rozdział 2. Szybki start

Ten rozdział przeprowadzi Cię od zera do **pierwszego wygenerowanego raportu w około 20 minut**. Czas podany przy każdym kroku to realny, zmierzony czas dla małego projektu (kilka plików). Na końcu rozdziału znajdziesz też mapę interfejsu — wszystko, czego potrzebujesz, żeby swobodnie poruszać się po aplikacji.

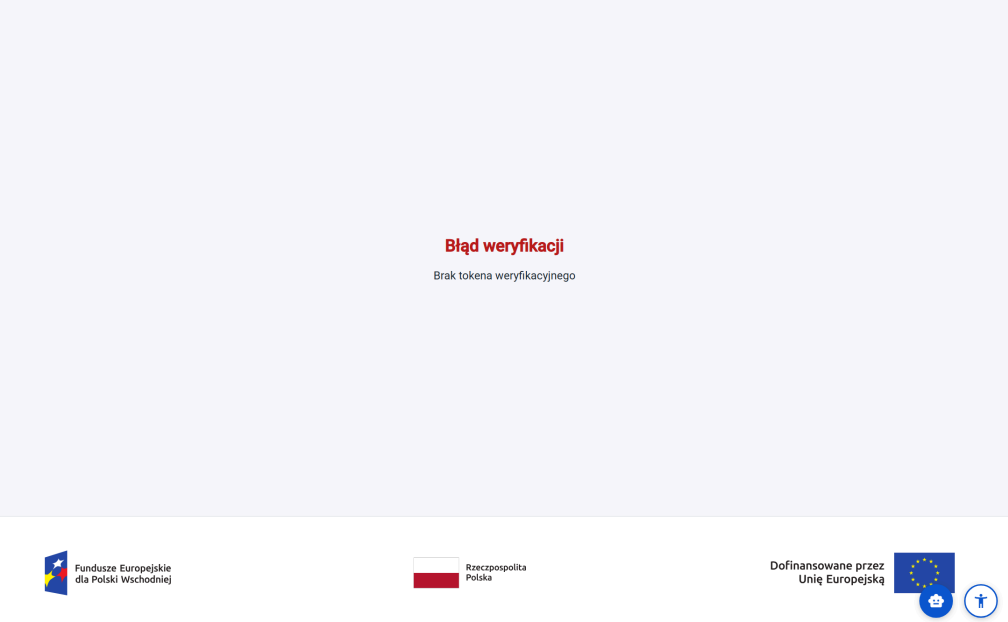
Mapa podróży:

Krok	Co robisz	Czas	Rozdział ze szczegółami
1	Zakładasz konto	3 min	2.1
2	Tworzysz pierwszy projekt	1 min	2.3

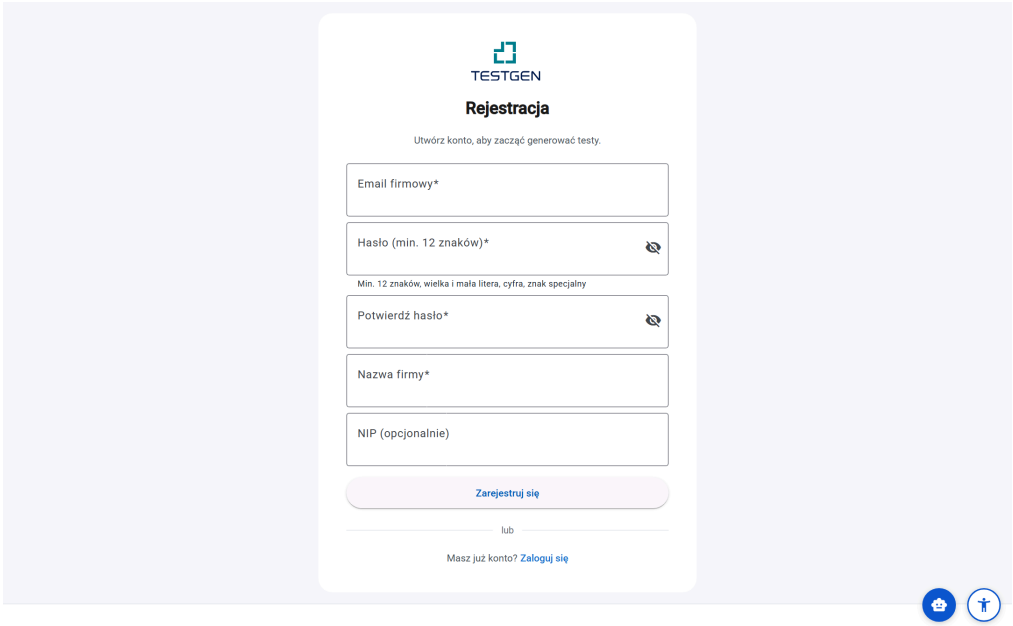
Krok	Co robisz	Czas	Rozdział ze szczegółami
3	Przesyłasz kod źródłowy	2 min	2.3
4	Uruchamiasz analizę	1 min (+ ok. 2 min w tle)	2.4
5	Przeglądasz wyniki i przypadki testowe	7 min	2.4 i 2.5
6	Generujesz i pobierasz raport	3 min	2.6

2.1. Rejestracja i pierwsze logowanie

1. Otwórz w przeglądarce adres platformy i kliknij **„Zarejestruj się"**.
2. Wypełnij formularz: imię i nazwisko, firmowy adres e-mail, hasło (minimum 12 znaków), nazwę firmy i NIP (jeśli chcesz faktury).
3. Kliknij **„Zarejestruj"** — na Twój adres przyjdzie wiadomość z linkiem aktywacyjnym.
4. Otwórz wiadomość i kliknij link. Konto jest aktywne.

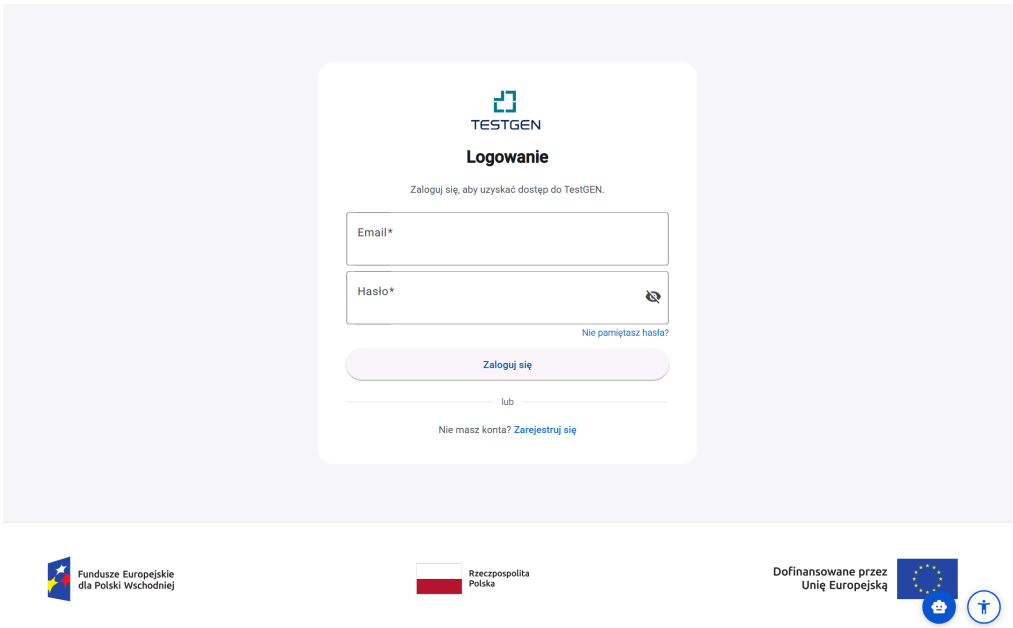


Potwierdzenie adresu e-mail kończy rejestrację.



Rejestracja: dane firmy użyjemy później na fakturach — możesz je zmienić w ustawieniach.

⚠️ **Uwaga:** link w wiadomości może wylądować w folderze oferty/reklamy. Jeśli wiadomości nie ma po 5 minutach, sprawdź te foldery zanim zaczniesz rejestrację od nowa.



Ekran logowania — z opcją zapamiętania sesji.

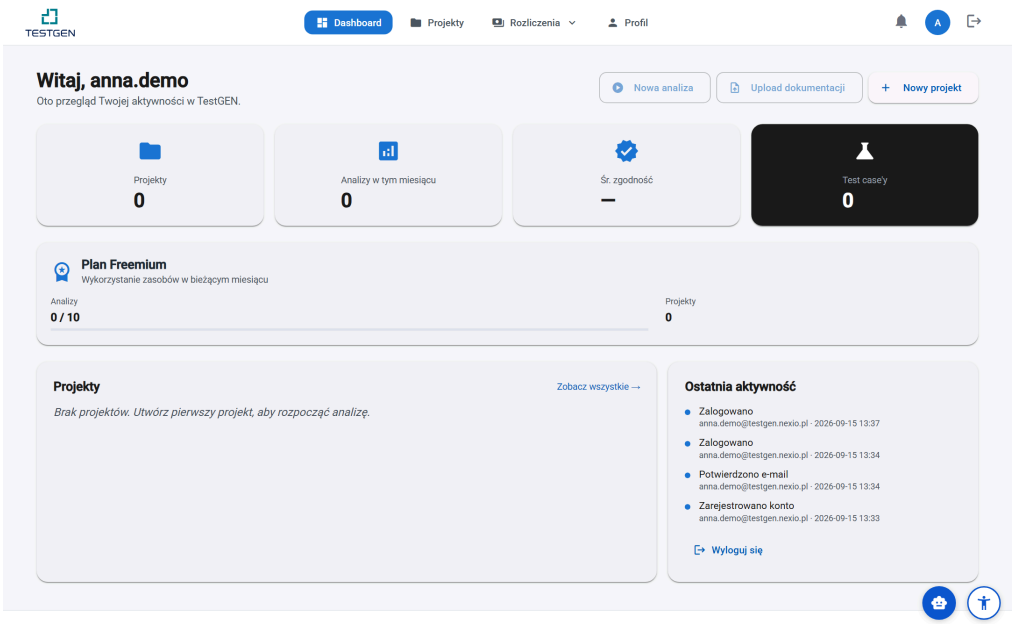
Rejestrujący zakłada nową **firmę** (organizację) i zostaje jej **Właścicielem**; później zaprosisz współpracowników w **„Profil" → „Użytkownicy"** (rozdział 13). Jeśli należysz do kilku firm, przy logowaniu wybierzesz organizację, na której chcesz pracować.

💡 **Wskazówka:** włącz **weryfikację dwuetapową (MFA)** zaraz po rejestracji — Profil → „Uwierzytelnianie MFA". Zajmuje dwie minuty, a chroni całą firmę, bo konto Właściciela zarządza subskrypcją i danymi fakturowymi. Instrukcja: rozdział

14.

2.2. Pierwszy pulpit

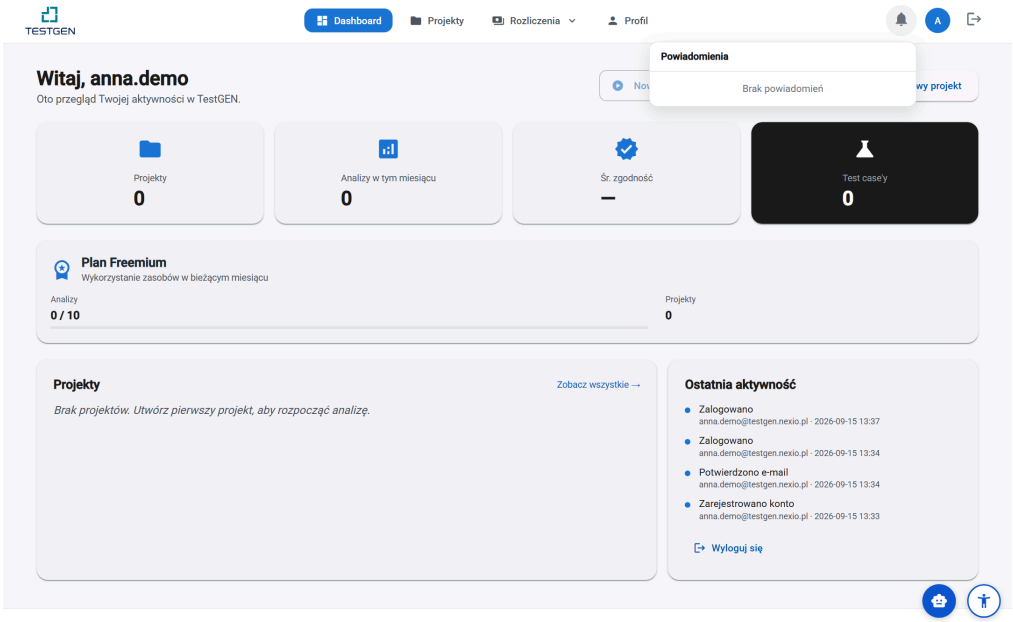
Po pierwszym logowaniu zobaczysz pulpit startowy:



Pulpit: statystyki na górze, karty szybkich akcji poniżej.

- **Karty statystyk** — liczba projektów, analiz, przypadków testowych i bieżące zużycie limitów; kliknij kartę, żeby przejść do odpowiedniego ekranu.
- **Ostatnia aktywność** — co działo się ostatnio w Twoich projektach (analizy, wygenerowane przypadki).
- **Szybkie akcje** — karty „Nowy projekt", „Prześlij pliki", „Uruchom analizę", „Zobacz raporty" — skróty do najczęstszych czynności z tego rozdziału.

Dzwonek w menu pokazuje powiadomienia — np. o przekroczeniu 80% limitu:

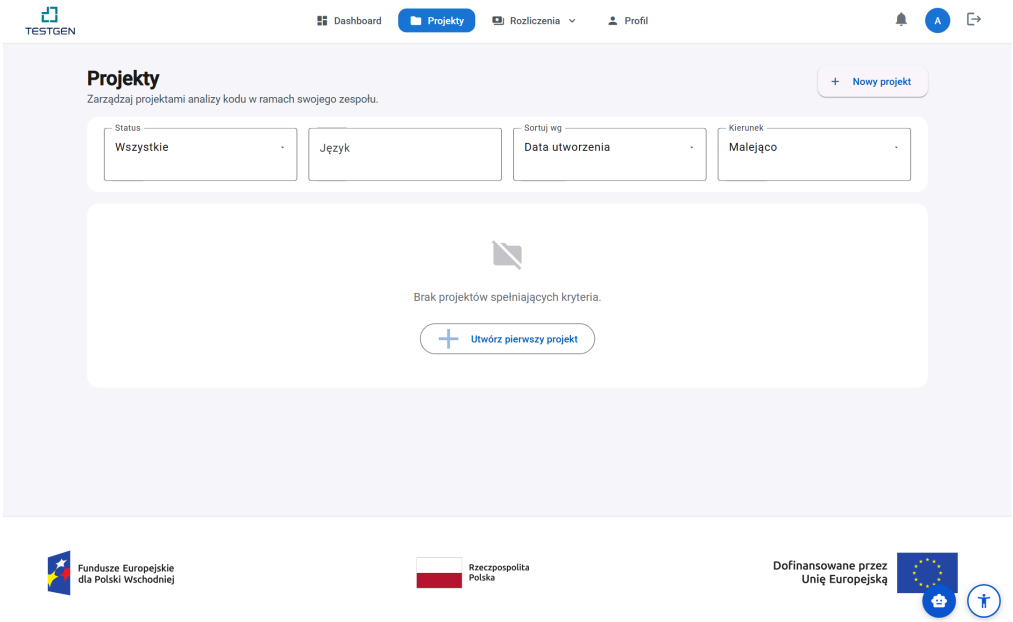


Powiadomienia o progach zużycia — te same treści mogą przychodzić e-mailem.

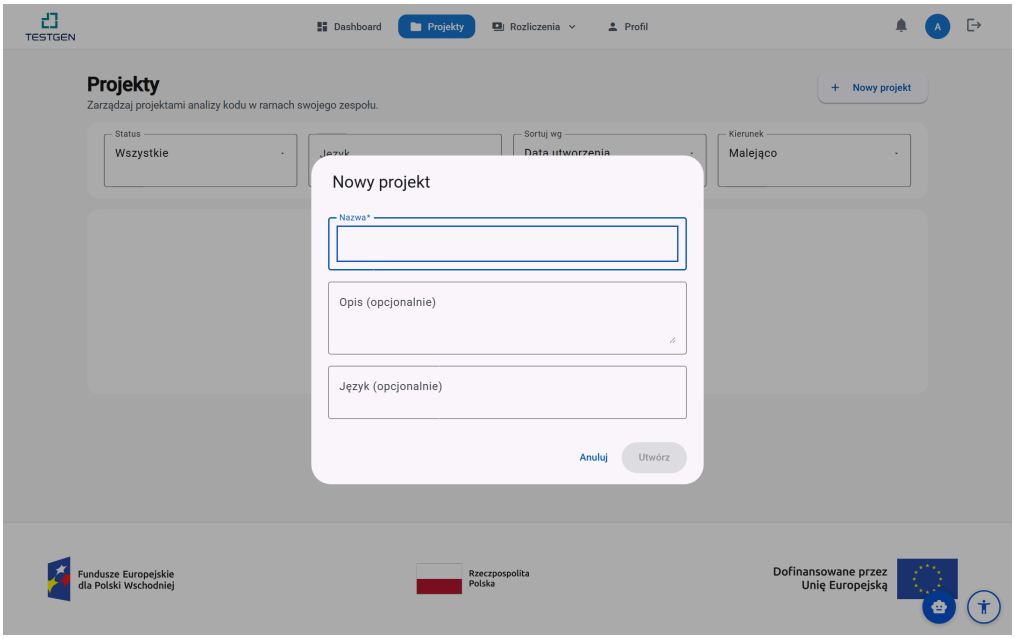
2.3. Konfiguracja pierwszego projektu

Projekt to folder roboczy dla jednej aplikacji: jej pliki, analizy, przypadki testowe i raporty żyją razem. Jeśli testujesz kilka aplikacji albo jedną w kilku wersjach — załóż osobne projekty.

1. Z menu bocznego wybierz **„Projekty"** — na start lista jest pusta.
2. Kliknij **„Nowy projekt"**.
3. Wpisz nazwę (np. „Sklep internetowy"), opcjonalnie opis i podpowiedź języka (np. „python"). Kliknij **„Utwórz"**.



Każdy nowy użytkownik zaczyna od czystej listy.

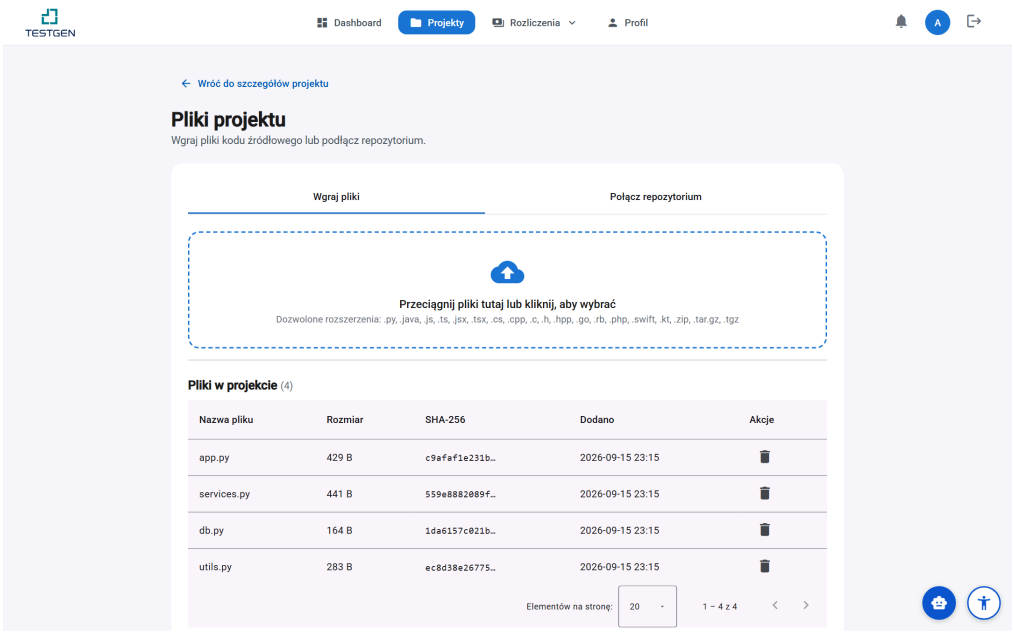


Formularz: nazwa jest wymagana, opis i język — opcjonalne.

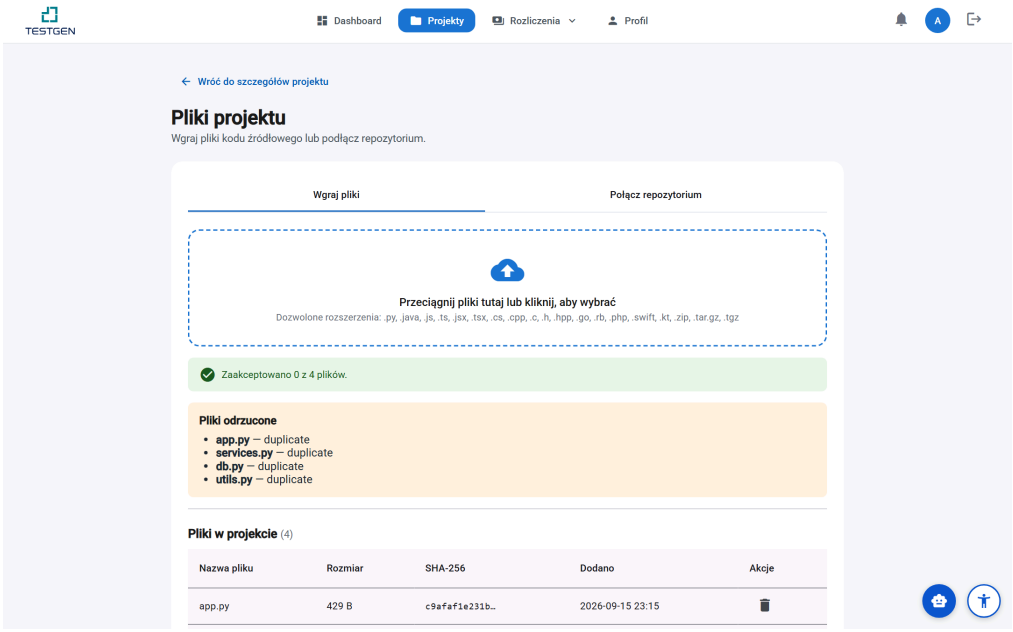
💡 **Wskazówka:** podpowiedź języka przyspiesza analizę — platforma sama rozpoznaje język z plików, ale podpowiedź pomaga przy mieszanych projektach.

Teraz prześlij kod. W projekcie otwórz zakładkę **„Pliki"**:

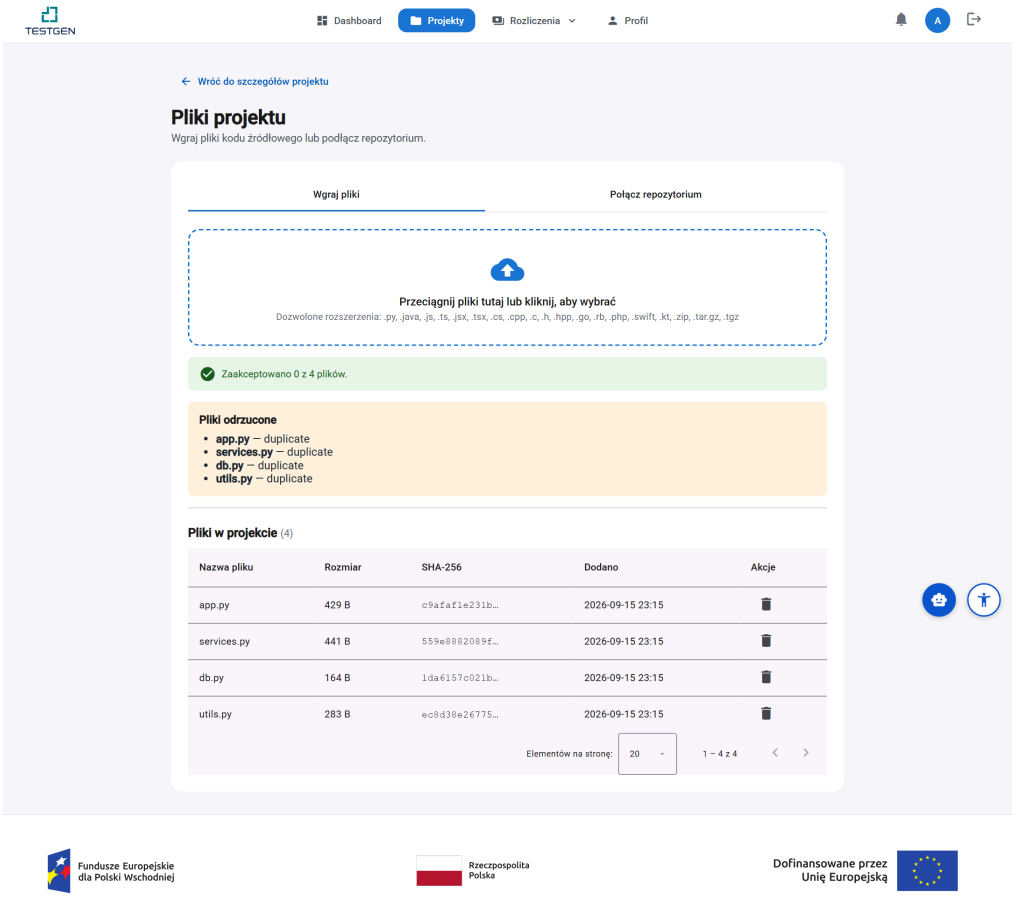
1. Przeciągnij pliki na obszar przesyłania (albo kliknij i wybierz je z dysku). Możesz zaznaczyć wiele plików naraz albo spakowane archiwum ZIP.
2. Poczekaj, aż wszystkie pliki dostaną status **„Gotowy"**.



Przeciągnij pliki tutaj albo kliknij, aby wybrać.



Każdy plik ma własny pasek postępu; przerwane można przesłać ponownie.



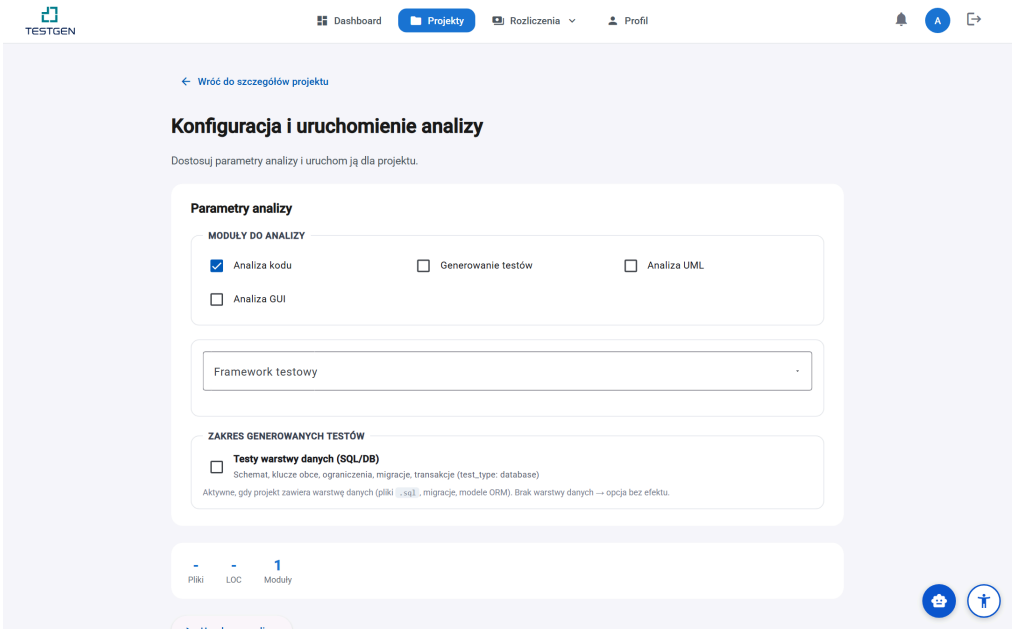
Fundusze Europejskie dla Polski Wschodniej
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Status „Gotowy" przy każdym pliku oznacza: można analizować.

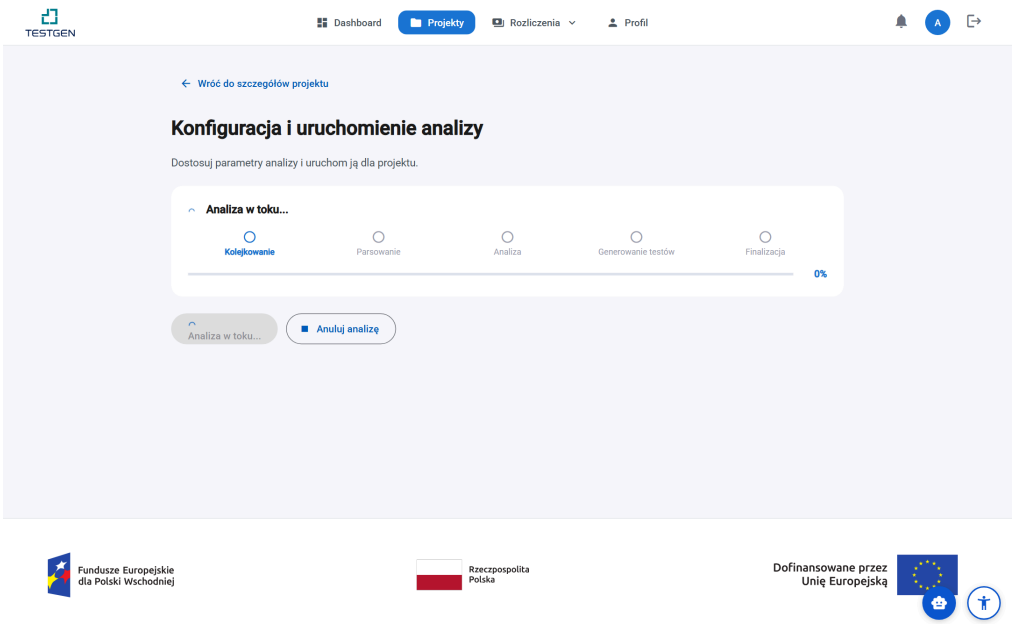
⚠️ **Uwaga:** plan Freemium pozwala na 5 plików jednorazowo i 0,5 GB miejsca. Na potrzeby pierwszej analizy w zupełności wystarczy kilka plików źródłowych. Pełne limity i listę formatów znajdziesz w rozdziale 4.

2.4. Pierwsza analiza kodu

1. Otwórz zakładkę **„Analiza"**.
2. Sprawdź parametry — na start zostaw domyślne (analiza kodu i generowanie testów, framework „— Automatyczny —").
3. Kliknij **„Uruchom analizę"**. Analiza działa w tle — możesz przełączyć się na inny ekran; o zakończeniu powiadomi komunikat.



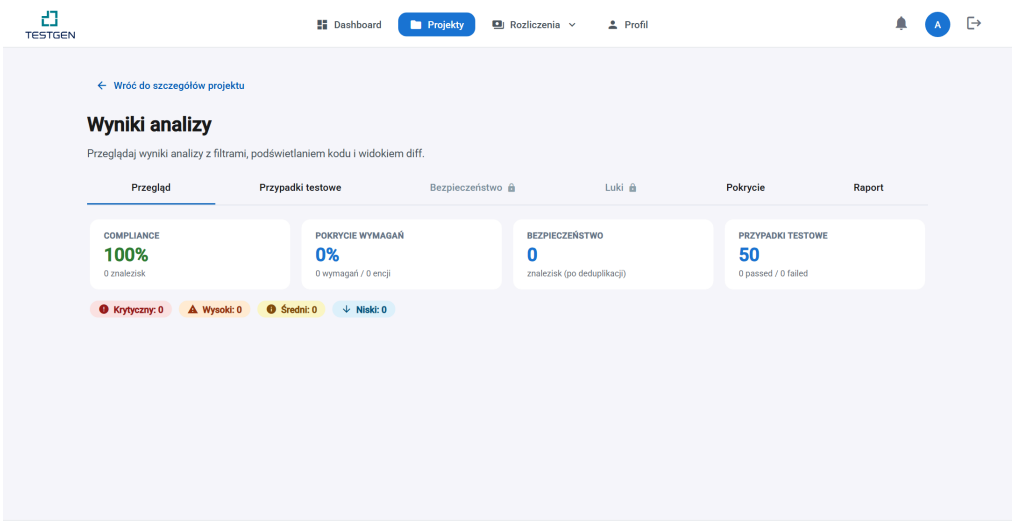
Analiza kodu: wystarczy jeden przycisk — typ i język rozpoznaje platforma.



Postęp widoczny na ekranie; zakończenie ogłasza komunikat.

💡 **Wskazówka:** analizy zużywają limit planu (Freemium: 10 miesięcznie). Limit odnawia się na początku każdego okresu rozliczeniowego — bieżące zużycie sprawdza się w menu **„Rozliczenia"** → **„Zużycie zasobów"**.

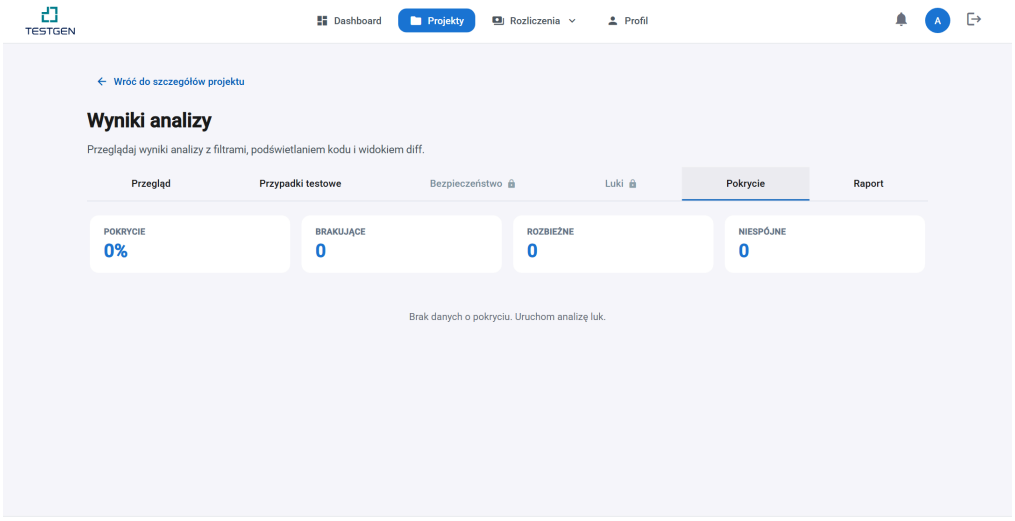
Po zakończeniu zobaczysz, że platforma rozpoznała projekt: języki z liczbą plików, frameworki, łączna liczba linii kodu — a w zakładce **„Wyniki"** czeka podsumowanie analizy:



Fundusze Europejskie dla Polski Wschodniej
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Podsumowanie: liczba rozpoznanych elementów kodu i wykrytych znalezisk.

- **Przegląd** — podsumowanie: ile funkcji i klas rozpoznano, ile znalezisk zgłoszono.
- **Pokrycie** — które elementy kodu mają pokrycie w wymaganiach.
- **Przypadki testowe** — najciekawsze: gotowe przypadki wygenerowane z Twojego kodu.



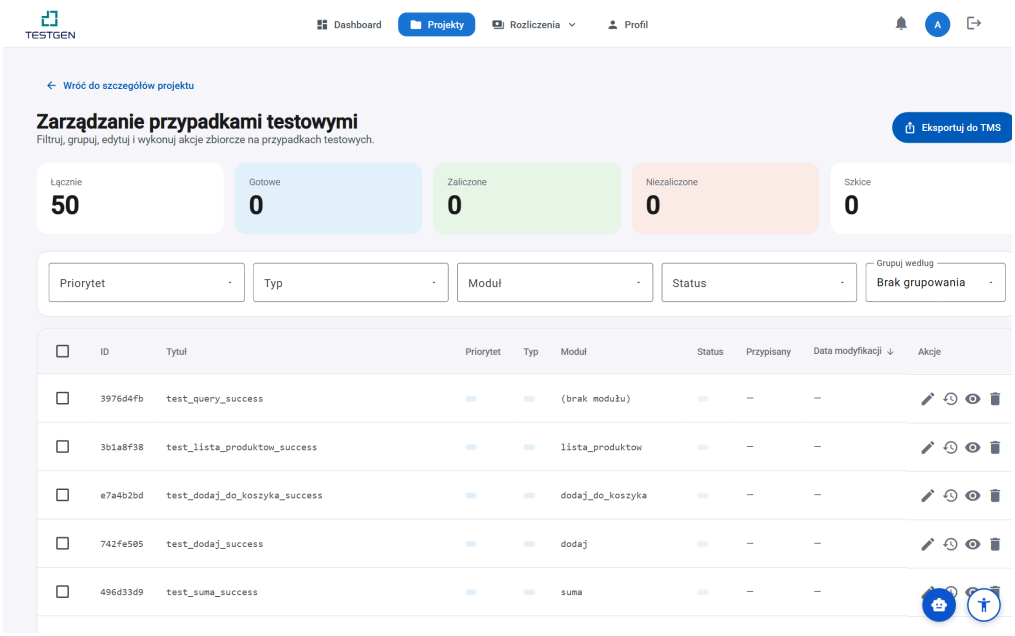
Fundusze Europejskie dla Polski Wschodniej
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Pokrycie kodu wymaganiami — na wyższych planach porównasz też dokumentację.

Co dokładnie oznaczają wyniki i jak je interpretować — opisuje właściwy rozdział analityczny.

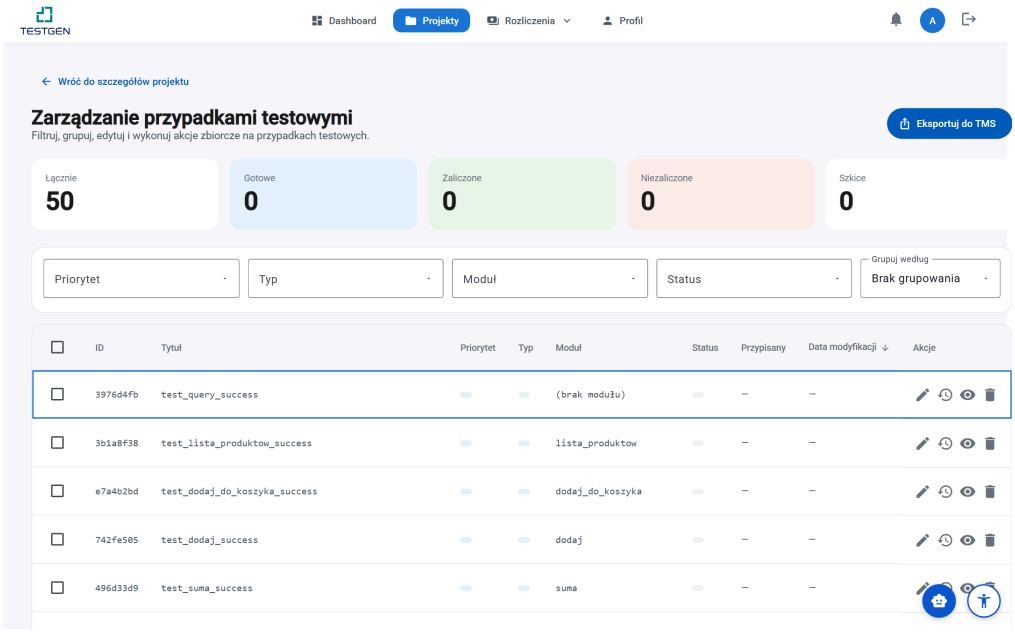
2.5. Pierwsze testy

Zakładka **„Przypadki testowe"** to centrum pracy analityka:



Każdy przypadek ma tytuł, priorytet, typ i status (nowy przypadek to „Szkica").

Kliknij wiersz, aby zobaczyć szczegóły — kroki, oczekiwane rezultaty i kod źródłowy, z którego przypadek powstał:



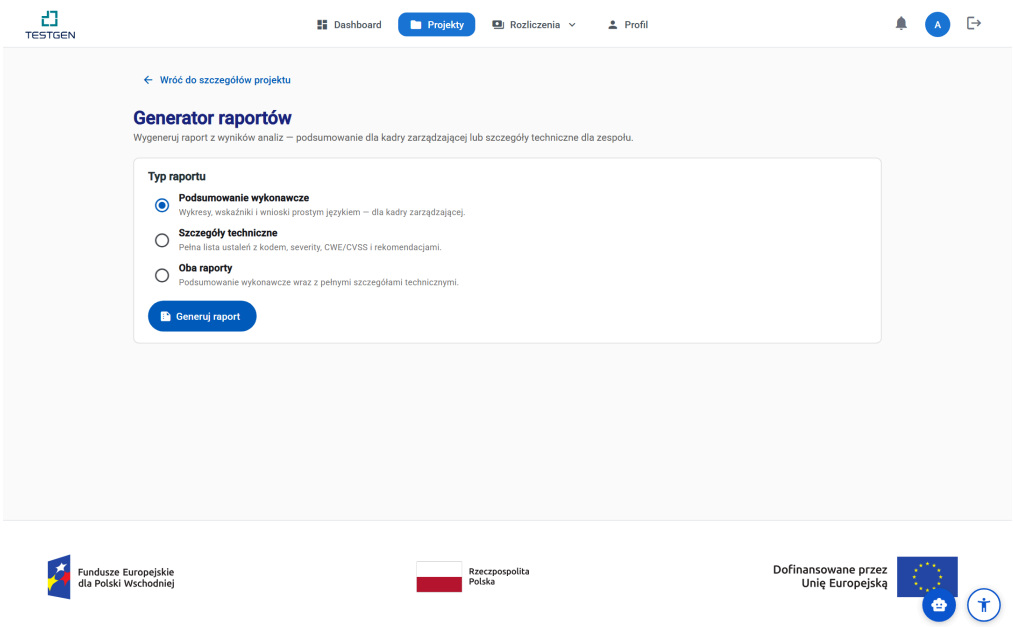
Szczegóły przypadku: kroki i powiązanie z kodem.

⚠️ **Uwaga:** nowe przypadki mają status **„Szkic"** — przejrzyj je przed użyciem. Zatwierdź sensowne przyciskiem **„Zatwierdź"** (status „Gotowy"); zbędne usuń. Dopiero zaakceptowane przypadki warto eksportować do zespołu.

Pełny cykl życia przypadków (edycja, operacje masowe, uruchamianie, generowanie kodu testów) opisuje rozdział 8.

2.6. Pierwszy raport

1. Otwórz zakładkę **„Raporty"**.
2. Wybierz typ raportu — **„Podsumowanie wykonawcze"** (dla zarządu) albo **„Szczegóły techniczne"** (dla zespołu).
3. Kliknij **„Generuj raport"** — podgląd pokaże dokładnie to, co trafi do pliku.
4. Pobierz w wybranym formacie: PDF, HTML, CSV, Excel, JSON lub YAML.



Raporty: wybierz typ, wygeneruj, pobierz — dwa poziomy szczegółowości.

💡 **Wskazówka:** na spotkanie z zarządem wystarczy „Podsumowanie wykonawcze"; zespół testowy doceni „Szczegóły techniczne" z pełną listą znalezisk.

Gratulacje — przeszedłeś/łaś cały obieg! Dalsze tematy — analiza dokumentacji, UML i GUI, bezpieczeństwo, eksport do narzędzi — czekają w Części 2.

2.7. Interfejs w pigułce

Żebyś nie zgubił/zgubiła się poza ścieżką szybkiego startu, oto mapa wszystkiego, czego używasz na co dzień:

- **Menu boczne** (po lewej): Pulpit („Dashboard"), Projekty, Rozliczenia, Profil. Zwija się do paska ikon na wąskich ekranach.
- **Okruszki** nad treścią: np. *Projekty → Sklep internetowy → Wyniki* — każde ogniwo klikalne; strzałka „Wróć..." prowadzi o poziom wyżej.
- **Karta projektu** to centrum dowodzenia — kafelki wszystkich funkcji; moduły spoza planu mają kłódkę.
- **Dzwonek** — powiadomienia (progi limitów, zdarzenia rozliczeń).
- **Przycisk „Przejdź do treści głównej"** — pierwszy element strony, widoczny po naciśnięciu Tab; pomija menu i przenosi od razu do treści (przydatny przy nawigacji klawiaturą).

- **Ekrany zachowują spokój:** ładowanie ma czytelne komunikaty, błędy podpowiadają następny krok, a puste listy wyjaśniają, od czego zacząć.

Dostępność (tryb wysokiego kontrastu, wyłączanie animacji), skróty klawiszowe i wygląd na różnych ekranach opisuje rozdział 12.

Utrzymanie dokumentu: zmiany UI = aktualizacja zrzutów i opisów; zrzuty pochodzą z rzeczywistej aplikacji (stan na 2026-09-20). Spójność terminów: Glosariusz. Kontynuacja: Część 2 — Praca z systemem.

Podręcznik użytkownika — Część 2: Praca z systemem

> **Wersja:** 2.0 (2026-09-20) · **Odbiorcy:** użytkownicy platformy TestGEN (analitycy QA, deweloperzy, kierownicy projektów) · **Część 2 z 4** (całość: 01 — Wprowadzenie, 03 — Zaawansowane funkcje, 04 — Administracja i rozwiązywanie problemów, Spis treści)
>
> **Konwencje:** forma bezpośrednia („Ty"), nazwy przycisków i ekranów w cudzysłowie dokładnie tak, jak widzisz je w aplikacji. Ramki: 💡 **Wskazówka** (oszczędza czas) i ⚠️ **Uwaga** (częsta pułapka). Definicje pojęć: Glosariusz.

Spis treści części

- **Rozdział 3. Zarządzanie projektami** — 3.1. Tworzenie i konfiguracja · 3.2. Lista, filtry i sortowanie · 3.3. Karta projektu · 3.4. Członkowie zespołu i role · 3.5. Edycja, usuwanie, archiwizacja · 3.6. Dobre praktyki · 3.7. Ograniczenia
- **Rozdział 4. Analiza kodu źródłowego** — 4.1. Wgrywanie kodu (pliki, ZIP, repozytorium) · 4.2. Inne źródła materiałów · 4.3. Konfiguracja i uruchomienie analizy · 4.4. Postęp · 4.5. Preprocessing · 4.6. Interpretacja wyników · 4.7. Testy regresji kodu · 4.8. Przykłady dla różnych języków · 4.9. Dobre praktyki · 4.10. Ograniczenia
- **Rozdział 5. Analiza dokumentacji** — 5.1. Wspierane formaty · 5.2. Wgrywanie dokumentacji · 5.3. Wymagania · 5.4. Niespójności · 5.5. Glosariusz · 5.6. Porównywanie dokumentacji z kodem (luki) · 5.7. Generowanie testów z dokumentacji · 5.8. Dobre praktyki · 5.9. Ograniczenia
- **Rozdział 6. Analiza UML i GUI** — 6.1. Wgrywanie diagramów UML · 6.2. Widoki diagramów · 6.3. Analiza interfejsów użytkownika · 6.4. Generowanie testów UI (Gherkin/BDD) · 6.5. Testy regresji GUI · 6.6. Macierz śledzenia · 6.7. Dobre praktyki · 6.8. Ograniczenia
- **Rozdział 7. Analiza bezpieczeństwa** — 7.1. Typy skanów · 7.2. Uruchamianie · 7.3. Interpretacja wyników · 7.4. Zależności i SBOM · 7.5. Rekomendacje naprawcze · 7.6. Priorytetyzacja · 7.7. Dobre praktyki · 7.8. Ograniczenia
- **Rozdział 8. Generowanie testów** — 8.1. Wybór frameworka testowego · 8.2. Przeglądanie i edycja przypadków · 8.3. Statusy i przegląd · 8.4. Operacje masowe · 8.5. Generowanie kodu testów · 8.6. Eksport testów · 8.7. Dobre praktyki · 8.8. Ograniczenia
- **Rozdział 9. Raporty i eksport** — 9.1. Poziomy raportów · 9.2. Generowanie i podgląd · 9.3. Formaty plików · 9.4. Integracja z systemami zewnętrznymi · 9.5. Dobre praktyki · 9.6. Ograniczenia

---

Rozdział 3. Zarządzanie projektami

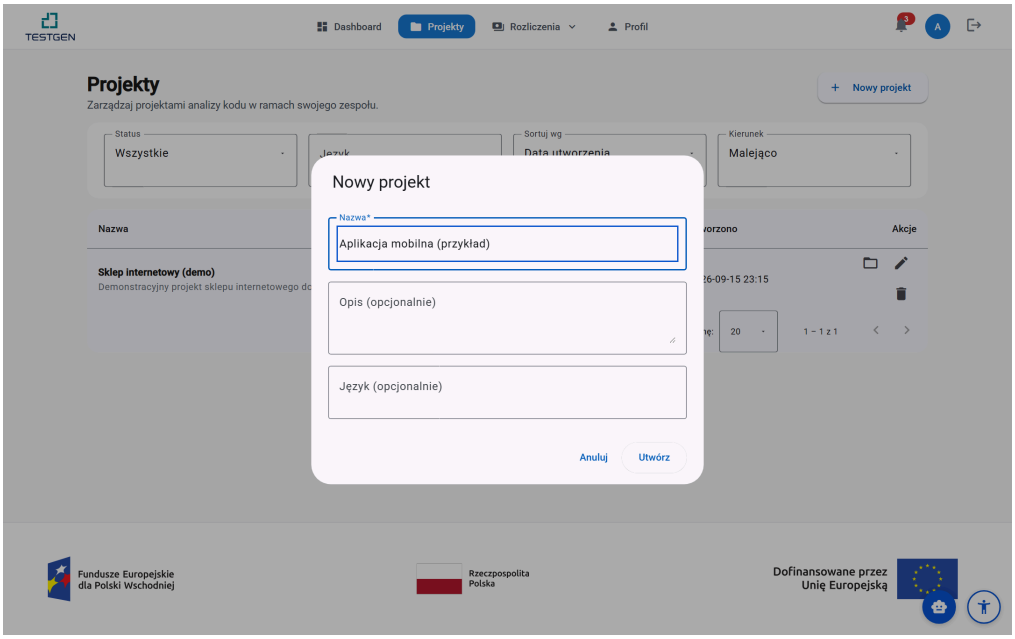
Projekt to folder roboczy dla jednej aplikacji: jej pliki, analizy, przypadki testowe i raporty żyją razem. Jeśli testujesz kilka aplikacji albo jedną aplikację w kilku wersjach — załóż osobne projekty.

3.1. Tworzenie i konfiguracja

1. Z menu bocznego wybierz **„Projekty"**, a następnie kliknij **„Nowy projekt"**.
2. Wypełnij formularz:

Pole	Wymagane	Limit	Opis
Nazwa	tak	255 znaków	unikalna w ramach firmy, np. „Sklep internetowy"
Opis (opcjonalnie)	nie	2000 znaków	cel projektu, zakres, zespół — pomoże po pół roku
Język (opcjonalnie)	nie	50 znaków	podpowiedź, np. „python, java, typescript"

3. Kliknij **„Utwórz"**. Nowy projekt dostaje status **„Utworzony"** — przechodzi w **„Aktywny"** po pierwszym przesłaniu plików lub podłączeniu repozytorium; analiza w trakcie daje **„W analizie"**, a jej zakończenie **„Zakończony"**.



Formularz ma trzy pola — nazwa jest wymagana, opis i język to podpowiedzi.

💡 **Wskazówka:** podpowiedź języka przyspiesza analizę — platforma sama rozpoznaje języki z plików, ale podpowiedź pomaga przy projektach mieszanych (kilka języków naraz).

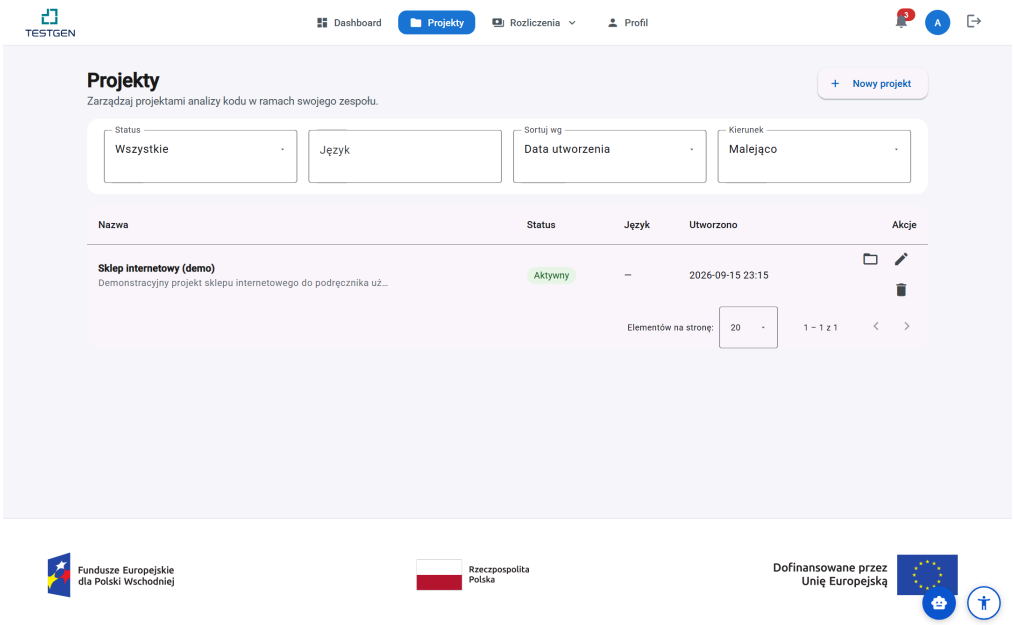
⚠️ **Uwaga:** nazwa musi być unikalna w firmie. Przy powtórce zobaczysz „Projekt o tej nazwie już istnieje." — zmień nazwę albo otwórz istniejący projekt z listy.

3.2. Lista, filtry i sortowanie

Lista projektów to tabela: **Nazwa** (link do karty, pod spodem opis), **Status**, **Język** (podpowiedź lub „—"), **Utworzono** oraz **Akcje**. Nad tabelą — karta filtrów:

- **„Status"** — Wszystkie / Utworzony / Aktywny / W analizie / Zakończony / Zarchiwizowany,
- **„Język"** — wpisujesz frazę, np. „python",
- **„Sortuj wg"** — Data utworzenia / Data modyfikacji / Nazwa / Status,
- **„Kierunek"** — Malejąco / Rosnąco.

Filtry reagują bez przeładowania (półsekundowe opóźnienie po wpisaniu); tabela stronicuje po 10/20/50/100 wierszy (domyślnie 20).

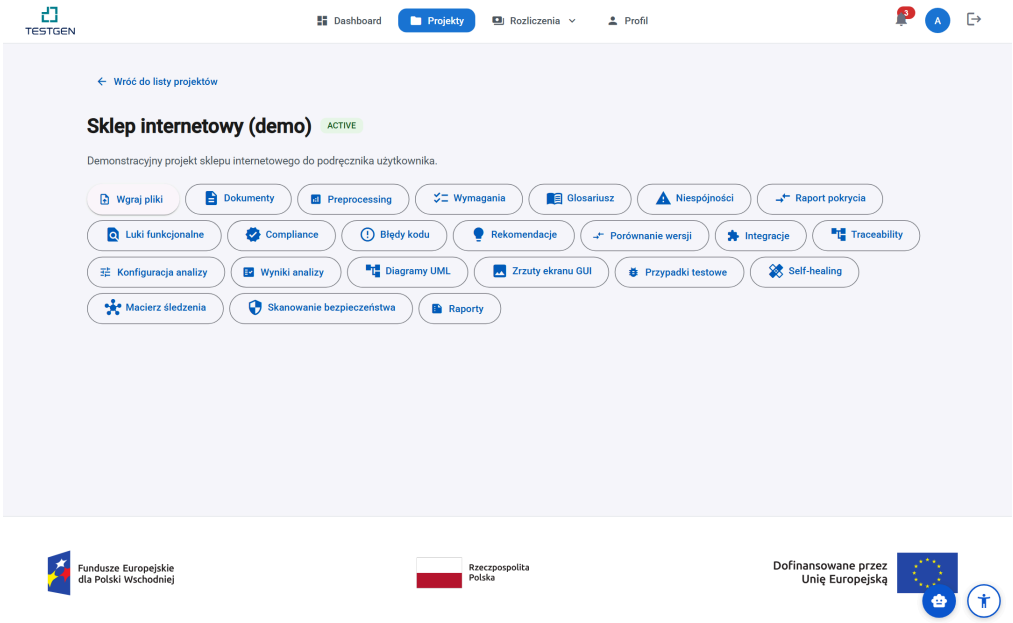


Fundusze Europejskie dla Polski Wschodniej
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Filtry Status i Język zawężają listę; sortowanie po dacie domyślne.

3.3. Karta projektu

Karta projektu to centrum dowodzenia: nazwa, status i opis na górze, poniżej kafelki wszystkich funkcji platformy:



Każdy kafelek to wejście do jednej funkcji; moduły spoza planu mają kłódkę.

Najważniejsze kafelki: **„Wgraj pliki"** (rozdz. 4), **„Konfiguracja analizy"** i **„Wyniki analizy"** (rozdz. 4), **„Dokumenty"** i pochodne (rozdz. 5), **„Diagramy UML"**, **„Zrzuty ekranu GUI"**, **„Macierz śledzenia"** (rozdz. 6), **„Skanowanie bezpieczeństwa"** (rozdz. 7), **„Przypadki testowe"** (rozdz. 8), **„Raporty"** (rozdz. 9) oraz **„Integracje"** (Część 3, rozdz. 10).

Kafelki modułów spoza Twojego planu mają kłódkę i podpis „Funkcja dostępna w planie {plan}" — kliknięcie otwiera stronę planów; po zmianie planu moduł odblokuje się bez ponownego konfigurowania projektu.

Powrót o poziom wyżej to link **„Wróć do listy projektów"**; okruszki nad treścią (*Projekty → {nazwa} → …*) są klikalne na każdym poziomie.

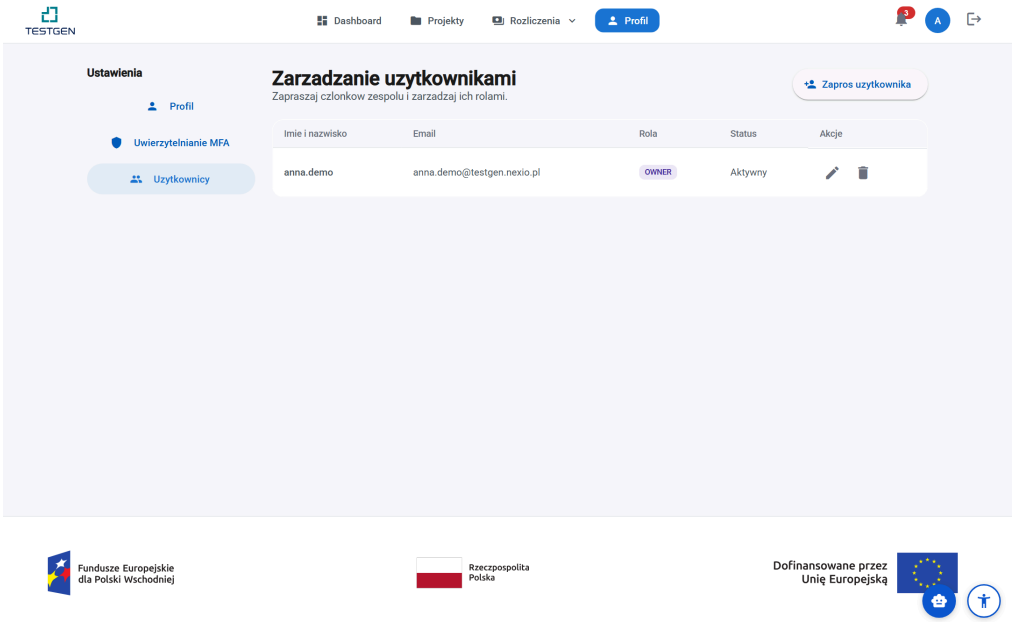
3.4. Członkowie zespołu i role

Uprawnienia działają **na poziomie firmy**, nie pojedynczego projektu: wszyscy członkowie firmy widzą wszystkie jej projekty.

Rola	Kto	Uprawnienia
Użytkownik	analitycy, deweloperzy	projekty, pliki, analizy, wyniki, przypadki testowe
Właściciel	zarządza firmą	to co Użytkownik + edycja i usuwanie projektów i plików, zapraszanie ludzi, subskrypcja i dane fakturowe
Administrator platformy	obsługa TestGEN	pełny dostęp techniczny (m.in. przywracanie usuniętych projektów)

Zapraszanie współpracowników: **„Profil"** → **„Użytkownicy"** (widoczne tylko dla Właściciela) → **„Zaproś użytkownika"** — wpisujesz adres e-mail i rolę; zaproszeni ustawiają hasło z linku w wiadomości i od razu widzą projekty firmy.

Szczegóły ekranu: rozdział 13.

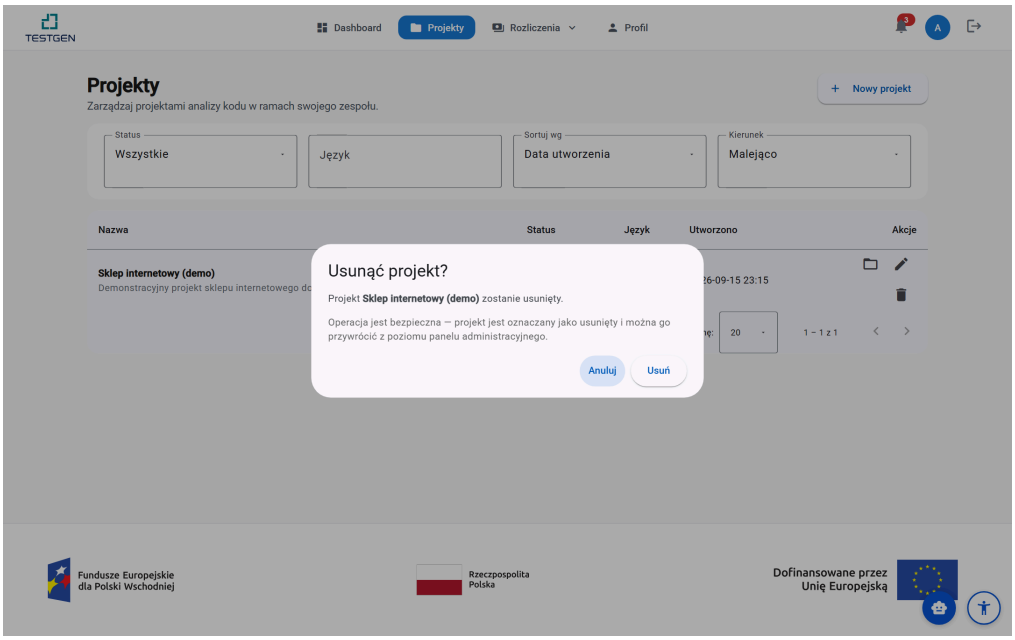


Właściciel zaprasza członków firmy i zmienia ich role w jednym miejscu.

3.5. Edycja, usuwanie, archiwizacja

Edycja. Na liście projektów akcja **„Edytuj projekt"** (ikona ołówka; tylko Właściciel) otwiera dialog z tymi samymi polami co tworzenie — zapis przyciskiem **„Zapisz"**.

Usuwanie. Akcja **„Usuń projekt"** (ikona kosza) otwiera dialog potwierdzenia:



Usuwanie jest bezpieczne: projekt można przywrócić z panelu administratora platformy.

Usunięcie nie kasuje danych natychmiast — projekt znika z Twojej listy, ale administrator platformy może go przywrócić. Skasowanie obejmuje powiązane dane (pliki, analizy, przypadki), więc wcześniej wyeksportuj raporty (rozdz. 9).

Archiwizacja. Status „Zarchiwizowany" widnieje w filtrach, ale samodzielne zamknięcie projektu na stanie (dane zostają, projekt znika z aktywnego obiegu) wykonuje administrator platformy na Twoje życzenie.

3.6. Dobre praktyki

- **Jeden projekt = jedna aplikacja.** Nie wrzucaj dwóch systemów do jednego projektu — analiza porówna je ze sobą, a wyniki będą szumem.
- **Opis mówiący, po co projekt.** Po pół roku „API płatności v2 — migracja z REST na gRPC" mówi więcej niż sama nazwa.
- **Wersje w osobnych projektach.** Porównywanie wersji działa wewnątrz projektu (4.7); większe przebudowy prowadź w projekcie „v2", a stan obu wersji zestawiaj raportami.
- **Sprzątaj przed analizą.** Usuń zewnętrzne biblioteki i artefakty budowania (`node_modules`, katalog `build`) — zaniżają metryki i zużywają miejsce.
- **Konwencja nazewnicza zamiast filtra właściciela.** Lista projektów jest wspólna dla firmy; prefiks zespołu w nazwie porządkuje listę taniej niż jakiekolwiek ustawienia.

3.7. Ograniczenia

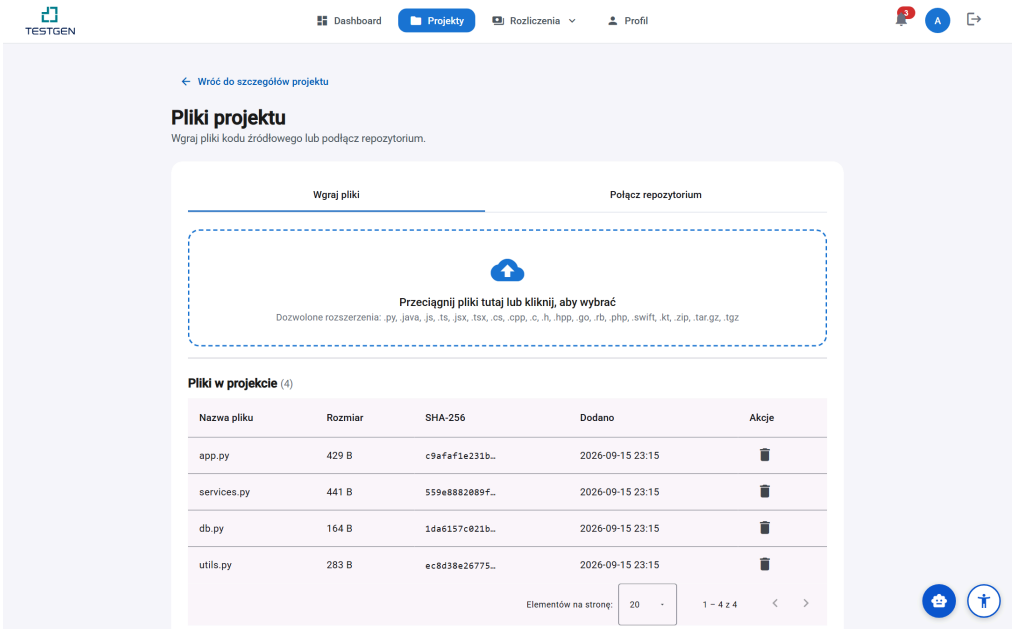
- **Liczba projektów bez limitu** w każdym planie — limitowane jest miejsce na pliki (4.1).
- **Brak archiwizacji z poziomu aplikacji** — tylko usunięcie (odwracalne przez administratora) lub zamknięcie przez obsługę.
- **Brak ról wewnątrz projektu** — dostęp zawsze obejmuje całą firmę.

Rozdział 4. Analiza kodu źródłowego

Analiza kodu to serce platformy: rozkłada przesłany kod na funkcje, klasy i zależności, wykrywa logikę biznesową, mierzy metryki i na tej podstawie proponuje przypadki testowe. Silniki analizy są **deterministyczne** — te same dane wejściowe dają ten sam wynik, dziś i za miesiąc. Tryb pracy silnika należy do konfiguracji platformy (ustala go administrator), nie do ustawień projektu — na niektórych ekranach widzisz go w nieaktywnym polu „Silnik analizy", wyłącznie informacyjnie.

4.1. Wgrywanie kodu (pliki, ZIP, repozytorium)

W karcie projektu kliknij **„Wgraj pliki"** — ekran „Pliki projektu" ma dwie zakładki: **„Wgraj pliki"** i **„Połącz repozytorium"**.

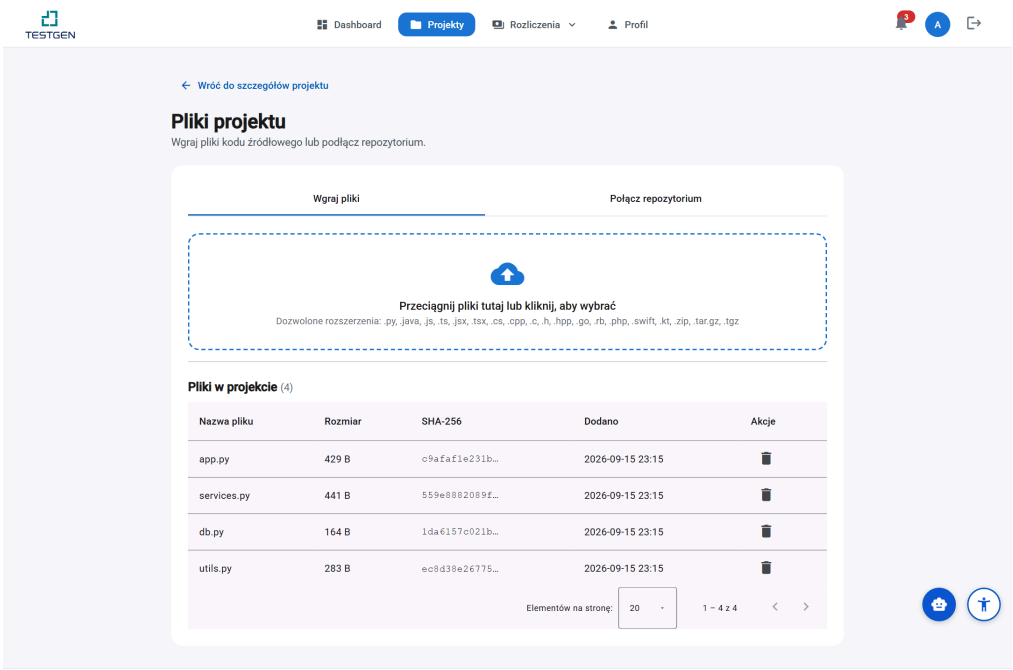


Przeciągnij pliki tutaj albo kliknij, aby wybrać; lista dozwolonych rozszerzeń podpowiedziana pod spodem.

Pojedyncze pliki i wielokrotny wybór. Przeciągnij pliki na obszar przesyłania albo kliknij i wybierz je z dysku. Pliki są sprawdzane od razu — rozszerzenia i rozmiar — a błędne odrzucane **przed** startem przesyłania, więc nic nie „wisi" na serwerze.

Archiwa ZIP i TAR.GZ. Spakowane archiwum (`.zip`, `.tar.gz`, `.tgz`) rozpakuje się na serwerze na osobne pliki. Bezpiecznik: maksymalnie 600 MB skompresowane, 1024 MB po rozpakowaniu i 10 000 plików — nadmiar zostanie pominięty.

Duplikaty. Każdy plik dostaje skrót SHA-256; plik o identycznej zawartości w ramach projektu jest odrzucany jako duplikat. Kolor tej samej wersji pliku między analizami łatwo porównasz po skrócie.

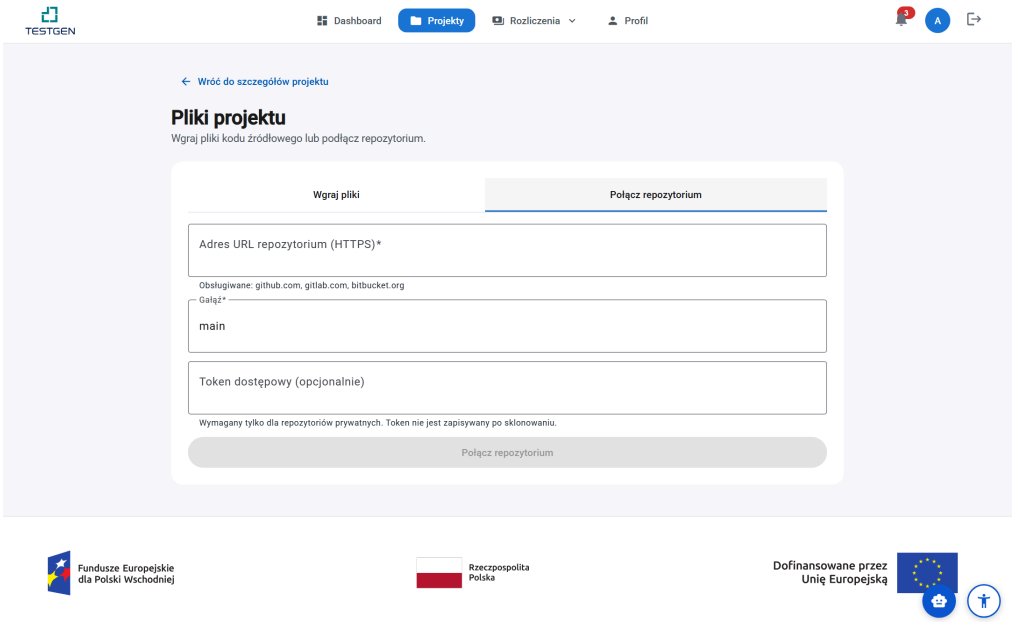


Fundusze Europejskie dla Polski Wschodniej
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Lista plików: nazwa, rozmiar, skrót SHA-256 i data — usuwanie plików (Właściciel) odzyskuje limit miejsca.

Repozytorium Git. Zamiast klikać pliki, podłącz repozytorium: zakładka **„Połącz repozytorium"**:

1. Wpisz **adres HTTPS** — obsługiwane `github.com`, `gitlab.com`, `bitbucket.org`.
2. Podaj **gałąź** (domyślnie „main").
3. Dla repozytorium prywatnego wklej **token dostępowy** — użyty raz do sklonowania, nie jest zapisywany.
4. Kliknij **„Połącz repozytorium"** — komunikat „Repozytorium podłączone. Zindeksowano {n} plików." podsumowuje płytkie klonowanie wybranej gałęzi.



Klon pobiera wybraną gałąź; token jest wymagany tylko dla repozytoriów prywatnych.

⚠️ **Uwaga:** połączenie jest **jednorazowe** — po zmianach połącz repozytorium ponownie; nowe pliki zostaną dodane, a niezmienione pominięte (rozpoznane po skrócie). Automatycznej synchronizacji nie ma; pojedynczy plik z repozytorium do 5 MB.

Limity wgrywania:

	Freemium	Basic	Premium
Plików na analizę	5	25	50
Miejsce na pliki (łącznie)	0,5 GB	5 GB	10 GB
Maksymalny rozmiar pliku	100 MB	500 MB	500 MB

Miejsce liczy się narastająco (stare pliki możesz usuwać, by je odzyskać); liczniki analiz resetują się co okres rozliczeniowy. Bieżące zużycie: **„Rozliczenia"** → **„Zużycie zasobów"** (poradnik billingowy).

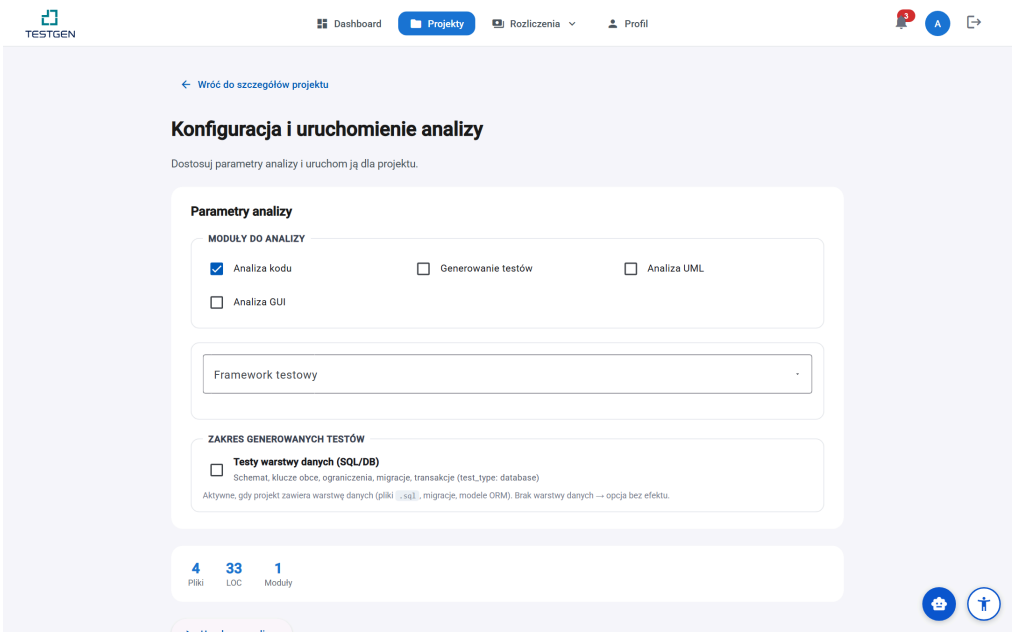
4.2. Inne źródła materiałów

Kod to jedno z czterech źródeł analizy. Pozostałe wgrywasz z osobnych ekranów (opisane we właściwych rozdziałach):

Źródło	Ekran	Rozdział
Dokumentacja (PDF, DOCX, MD, HTML, TXT)	„Dokumenty"	5
Diagramy UML (PlantUML, XMI, rysunki)	„Diagramy UML"	6
Zrzuty ekranu GUI	„Zrzuty ekranu GUI"	6.3

4.3. Konfiguracja i uruchomienie analizy

Kafelek **„Konfiguracja analizy"** otwiera ekran z podsumowaniem projektu (karty „Pliki", „LOC", „Moduły") i formularzem **„Parametry analizy"**:



Moduły, framework testowy i zakres generowanych testów — resztę platforma wykrywa sama.

1. **„Moduły do analizy"** — zaznacz co najmniej jeden:
   - **„Analiza kodu"** — struktura, logika biznesowa, metryki,
   - **„Generowanie testów"** — przypadki testowe i kod testów,
   - **„Analiza UML"** — parsowanie diagramów (rozdz. 6),
   - **„Analiza GUI"** — rozpoznawanie zrzutów (rozdz. 6.3).
2. **„Framework testowy"** — domyślnie „— Automatyczny —"; możesz wymusić Pytest, JUnit, NUnit, Cypress lub Mocha/Jest.
3. **„Zakres generowanych testów"** — opcjonalnie **„Testy warstwy danych (SQL/DB)"**: schemat, klucze obce, ograniczenia, migracje, transakcje; działa, gdy projekt zawiera warstwę danych (`.sql`, migracje, ORM).

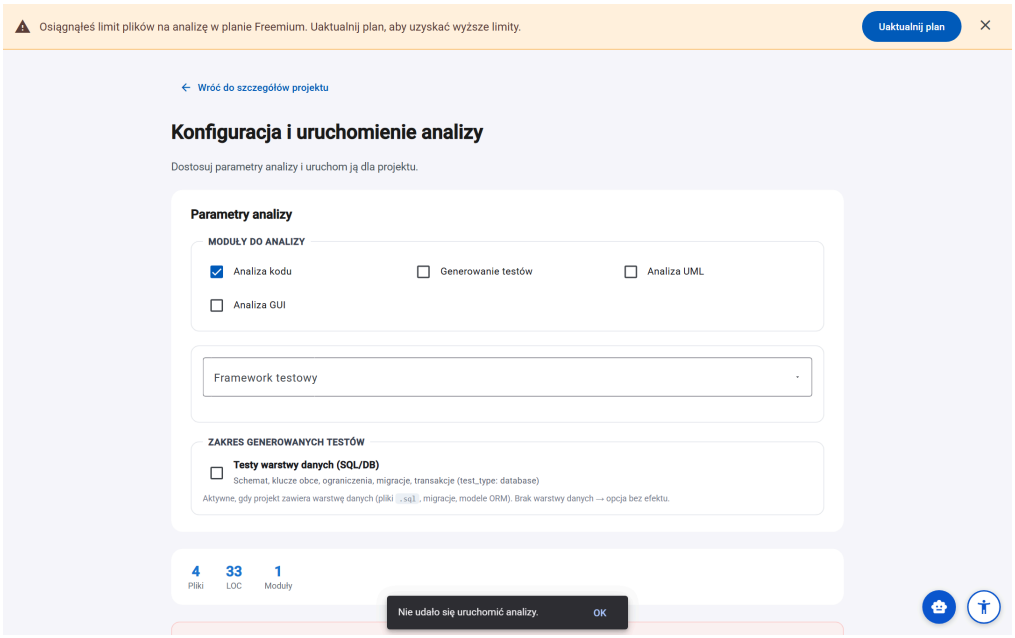
Parametry zapamiętywane są osobno dla każdego projektu. Uruchomienie: **„Uruchom analizę"** (w trakcie „Analiza w toku...", po zakończeniu „Uruchom ponownie", po błędzie „Spróbuj ponownie").

⚠️ **Uwaga:** każde uruchomienie zużywa miesięczny limit analiz (Freemium: 10); liczba plików w analizie limitowana planem (5/25/50).

4.4. Postęp

Analiza działa **w tle** — możesz przełączyć się na inny ekran, a o zakończeniu powiadomi komunikat. Postęp widać na pięciu etapach:

Kolejkowanie → Parsowanie → Analiza → Generowanie testów → Finalizacja



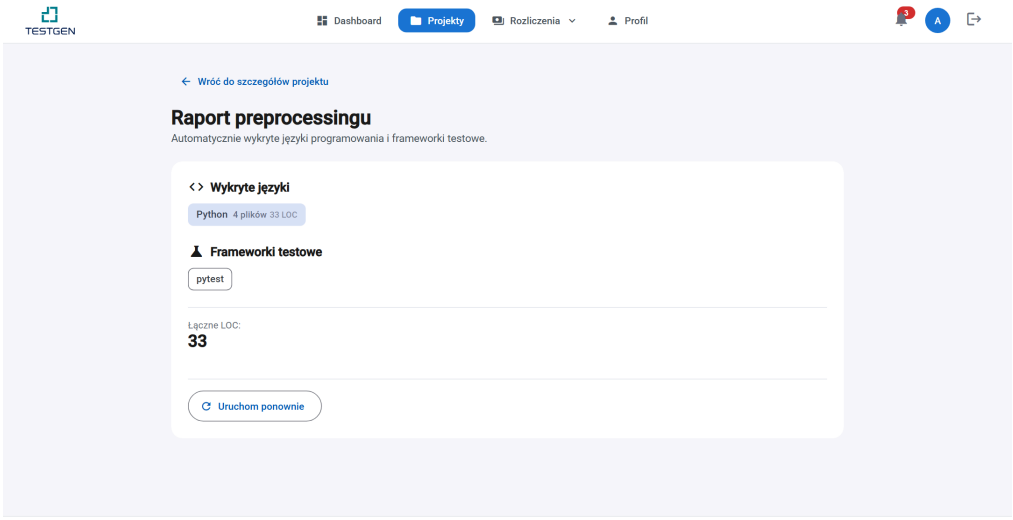
Zakończona analiza: każdy z pięciu etapów ma „ptaszka", pasek doszedł do 100%, a pod spodem widać wykryte technologie.

Trwającą analizę przerwiesz przyciskiem **„Anuluj analizę"**. Po zakończeniu ekran pokazuje „Analiza zakończona. Wszystkie kroki zostały wykonane pomyślnie." oraz **„Wykryte technologie"** — chipy języków (z liczbą plików) i frameworki; szybkie sprawdzenie, czy platforma dobrze rozpoznała projekt. Przerwana analiza kończy się statusem anulowana, a nieudana — komunikatem „Analiza nie powiodła się" z przyczyną.

4.5. Preprocessing

Zanim kod trafi do analizy, przechodzi preprocessing — wyniki w **„Raporcie preprocessingu"** (kafelek „Preprocessing"):

- **„Wykryte języki"** — chipy „język (liczba plików)" z liczbą linii,
- **„Frameworki testowe"** — rozpoznane z plików konfiguracyjnych,
- **„Łączne LOC"** — rozmiar projektu w liniach kodu,
- **„Pliki wymagające klasyfikacji"** — pliki bez rozszerzenia; każdemu ręcznie przypisz język z listy „Wybierz język".



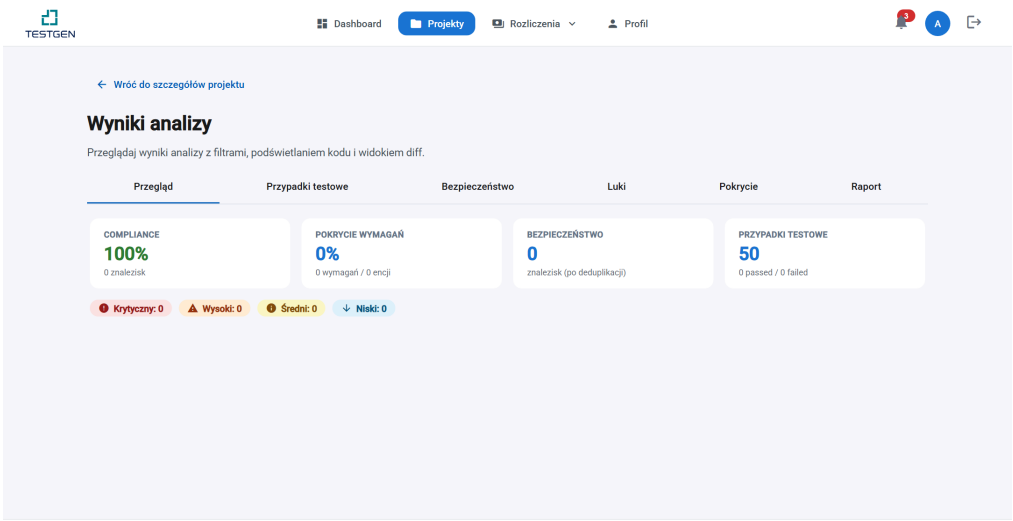
Fundusze Europejskie dla Polski Wschodniej
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Preprocessing potrafi rozpoznać język także z zawartości pliku — nagłówki, słowa kluczowe.

💡 **Wskazówka:** złe wykrycie języka popraw tutaj — dopiero wtedy analiza policzy struktury poprawnie. „Uruchom ponownie" odświeża raport po zmianach w plikach.

4.6. Interpretacja wyników

Kafelek **„Wyniki analizy"** otwiera przeglądarkę wyników z sześcioma zakładkami:



Fundusze Europejskie dla Polski Wschodniej
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Karty podsumowania: Compliance, pokrycie wymagań, bezpieczeństwo i przypadki testowe.

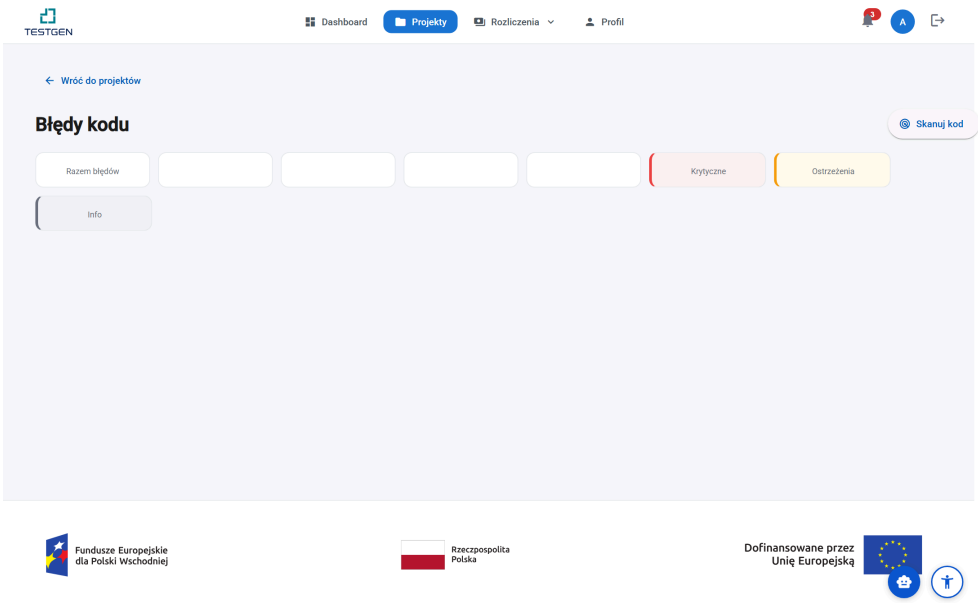
- **„Przegląd"** — karty **Compliance** (zgodność kodu z dokumentacją w procentach, z liczbą znalezisk), **„Pokrycie wymagań"**, **„Bezpieczeństwo"**,

„Przypadki testowe" (ile przeszło, ile nie) oraz zestawienie znalezisk według wagi: Krytyczny / Wysoki / Średni / Niski.

- **„Przypadki testowe"** — tabela wygenerowanych przypadków (szczegóły w rozdz. 8 i poradniku Wyniki i test case'y).
- **„Bezpieczeństwo"** 🔒 i **„Luki"** 🔒 — moduły planu Premium (rozdz. 7 i 5.6).
- **„Pokrycie"** — karty „Pokrycie", „Brakujące", „Rozbieżne", „Niespójne" oraz tabela dla poszczególnych plików.
- **„Raport"** — liczniki „Znaleziska łącznie", „Deduplikacja", „Korelacje".

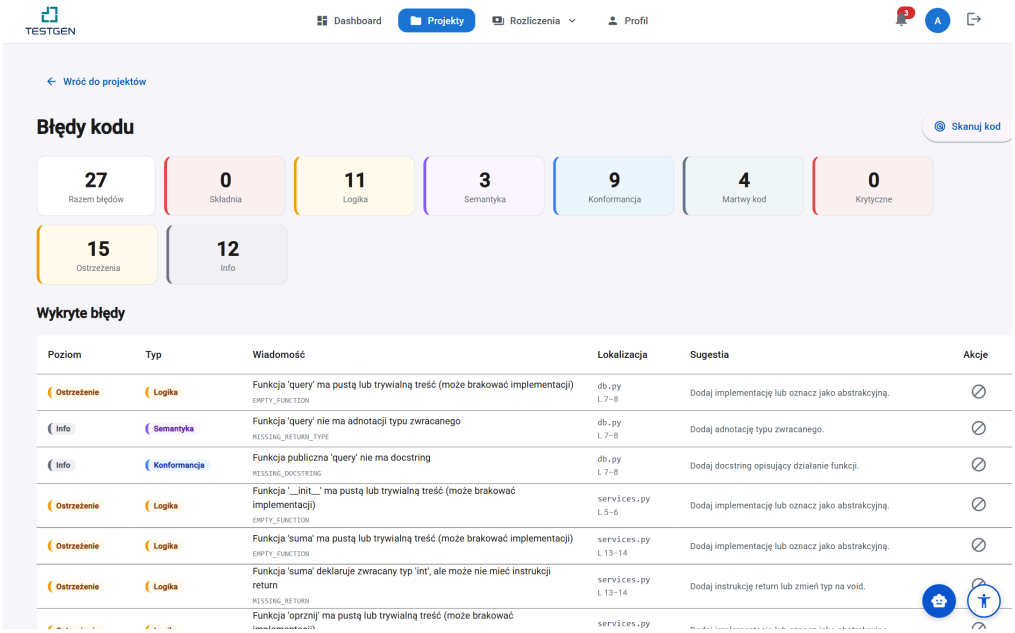
Panel filtrów zawęża wyniki: szukajka, przełączniki wagi i statusu, listy „Typ"/„Moduł", autouzupełnianie pliku, „Wyczyść". Kliknięcie wiersza otwiera szczegóły znaleziska: opis, **„Fragment kodu"** z podświetloną składnią i **„Sugerowaną poprawkę (diff)"**. Statusy znalezisk: „Nowy", „Zaakceptowany", „Odrzucony".

Błędy kodu. Sąsiedni ekran **„Błędy kodu"** listuje problemy samego kodu: Poziom (Krytyczny / Ostrzeżenie / Info), Typ, Wiadomość, Lokalizacja (plik:linia) i Sugestię poprawy.



Każdy błąd ma lokalizację i sugestię — gotową podpowiedź dla dewelopera.

Typ **„Martwy kod"** (poziom „Ostrzeżenie") wskazuje funkcje i metody **prywatne lub wewnętrzne, których nic nie wywołuje w projekcie** — każde znalezisko zawiera lokalizację (plik:linia) i liczbę „0 wywołań". Celowo NIE są zgłaszane elementy osiągalne pośrednio: endpointy API, `main`, funkcje testowe (`test_*`, `spec`), konstruktory, nazwy importowane gdziekolwiek w projekcie oraz handlery frameworków (dekoratory route/endpoint/command) — wywołania dynamiczne (getattr, wstrzykiwanie zależności, refleksja) czyniłyby z żywego kodu martwy. Funkcję utrzymywaną celowo oznacz dekoratorem `@internal` — detektor ją pominie; standardowa sugestia brzmi: „Sprawdź i usuń albo oznacz `@internal`". Fałszywy pozytyw odrzucisz przyciskiem **„Odrzuć"** — odrzucone znalezisko nie wróci przy ponownym skanie tego samego stanu kodu (edycja treści funkcji przywraca zgłoszenie). Detektor wyłącznie **raportuje** — niczego nie usuwa.



Kategoria „Martwy kod" w podsumowaniu i tabeli; w kolumnie Akcje przycisk „Odrzuć" (fałszywy pozytyw).

Co dokładnie „rozumie" analiza:

- **Struktury** — funkcje i metody (nazwa, plik, zakres linii, parametry z typami, typ zwracany), klasy (metody, atrybuty, dziedziczenie), importy i wywołania.
- **Punkty wejścia** — endpointy API (metoda, ścieżka, parametry) dla rozpoznanych frameworków webowych.
- **Zależności** — sprzężenia między modułami i **cykle** (zależności kołowe) — częsty sygnał architektonicznego długu.
- **Martwy kod** — definicje prywatne/wewnętrzne bez żadnego wywołania w projekcie; raportowane w kategorii „Błędy kodu" (patrz wyżej), z wykluczeniem punktów wejścia, testów i kodu oznaczonego `@internal`.
- **Reguły logiki biznesowej** — walidacje, ograniczenia, obliczenia, autoryzacje, reguły warunkowe i niezmienniki; operacje CRUD, polecenia i zapytania; zdarzenia i pojęcia domenowe (encje, obiekty wartości).
- **Metryki** — złożoność cyklomatyczna z progami: niska (≤ 5), średnia (5–10), wysoka (> 10); licznik linii kodu dla plików i całości.

To te struktury zasilają generowanie przypadków testowych (rozdz. 8).

4.7. Testy regresji kodu

Regresja w TestGEN to **zestaw przypadków, nie odrębny typ testu**. Mechanika opiera się na porównywaniu wersji:

1. Po istotnej zmianie prześlij pliki ponownie i uruchom analizę.
2. Platforma porównuje struktury między uruchomieniami: **added** (nowa funkcja — brak pokrycia testami), **removed** (usunięta — testy do weryfikacji/usunięcia), **changed** (zmieniona sygnatura/ciało — testy do ponownego uruchomienia), **unchanged** (pomijalne — szybsze wykonanie).

3. Przypadki powiązane ze zmienionymi strukturami tworzą **zestaw regresyjny** — niezmienione obszary odpadają, co skraca czas przy dużych bazach.

Pełny obraz czasu dają **zrzuty stanu zgodności** (compliance): karta **„Porównanie wersji"** pozwala utworzyć zrzut bieżącego wyniku, porównać dwa zrzuty (plakietki „Naprawione" / „Nowe" / „Bez zmian" / „Pogorszone", tabela „Szczegóły luk", wykres „Trend compliance") — szczegóły w rozdziale 5.6, gdzie leży sedno zgodności.

💡 **Wskazówka:** rób zrzut stanu po każdym sprincie. Trend mówi więcej niż pojedynczy wynik — „compliance 72%" nic nie znaczy, „72% i rośnie od trzech sprintów" znaczy wszystko.

4.8. Przykłady dla różnych języków

Platforma rozpoznaje **11 języków**; pełna analiza struktury (drzewo składniowe) obejmuje osiem z nich:

Język	Rozpoznawanie	Pełna analiza (funkcje, klasy)	Uwagi praktyczne
Python	✓	✓	najpełniejsze wsparcie: frameworki Flask/FastAPI/Django, pytest/unittest, testy bazodanowe
Java	✓	✓	Spring/JAX-RS, JUnit/TestNG; skeletony w javie
JavaScript	✓	✓	Express, Vitest/Jest/Mocha; `.js` / `.jsx`
TypeScript	✓	✓	NestJS; kod Cypress (`.cy.ts`) i Playwright (`.spec.ts`)
PHP	✓	✓	Laravel/Symfony, PHPUnit/Pest — dedykowany generator
C	✓	✓	—
C++	✓	✓	Google Test/Catch2 wykrywane
C#	✓	✓	ASP.NET, xUnit/NUnit/MSTest wykrywane
Go	✓	—	język i frameworki tak; bez rozkładu na funkcje/klasy
Ruby	✓	—	j.w.
Swift / Kotlin	✓	—	j.w.

Co to znaczy w praktyce: dla języków z pełną analizą dostaniesz struktury, logikę biznesową, metryki **i wygenerowane przypadki testowe**; dla Go, Ruby, Swift i Kotlin — wykrycie języka, frameworków i metryki plików, ale bez przypadków z kodu (testy możesz nadal generować z dokumentacji i diagramów).

Przykład wykładnika różnicy — funkcja w Pythonie po analizie zasila generator pozytywnym, negatywnym (walidacja `tax_rate < 0`) i brzegowym (`items = []`) scenariuszem; plik `.go` z tą samą logiką przejdzie do metryk i wykrycia frameworka bez scenariuszy.

4.9. Dobre praktyki

- **Analizuj czysty kod** — bez `node_modules`, katalogów budowania i wygenerowanych paczek.
- **Zacznij od „— Automatyczny —"** — wykrywanie frameworka wystarcza w przeważającej liczbie projektów.
- **Analizuj punktualnie** — po istotnej zmianie, nie po każdej edycji; różnice zobaczysz w porównaniu wersji.
- **Reaguj na czerwoną wagę** — zanim zaczniesz przeglądać przypadki, obejrzyj znaleziska „Krytyczny"; często unieważnią część scenariuszy.
- **Utrzymuj jedną wersję pliku** — duplikaty odrzuci platforma, ale stare kopie (`service_old.py`) wprowadzają szum do analizy.

4.10. Ograniczenia

- **Pełna analiza struktury: 8 języków** (lista wyżej) — dla pozostałych bez przypadków z kodu.
- **Analiza działa na przesłanych plikach**, nie na żywym repozytorium.
- **Analiza statyczna nie uruchamia kodu** — błędy wykonawcze łapie DAST (rozdz. 7).
- **Jedna analiza = limit plików planu** (5/25/50).

Rozdział 5. Analiza dokumentacji

Analiza dokumentacji czyta Twoje dokumenty techniczne — specyfikacje, podręczniki, opisy API — i zamienia je w ustrukturyzowane wymagania, wykrywa niespójności i buduje glosariusz pojęć. To pomost między tym, co obiecuje dokumentacja, a tym, co robi kod. Moduł wymaga planu Basic lub wyższego.

5.1. Wspierane formaty

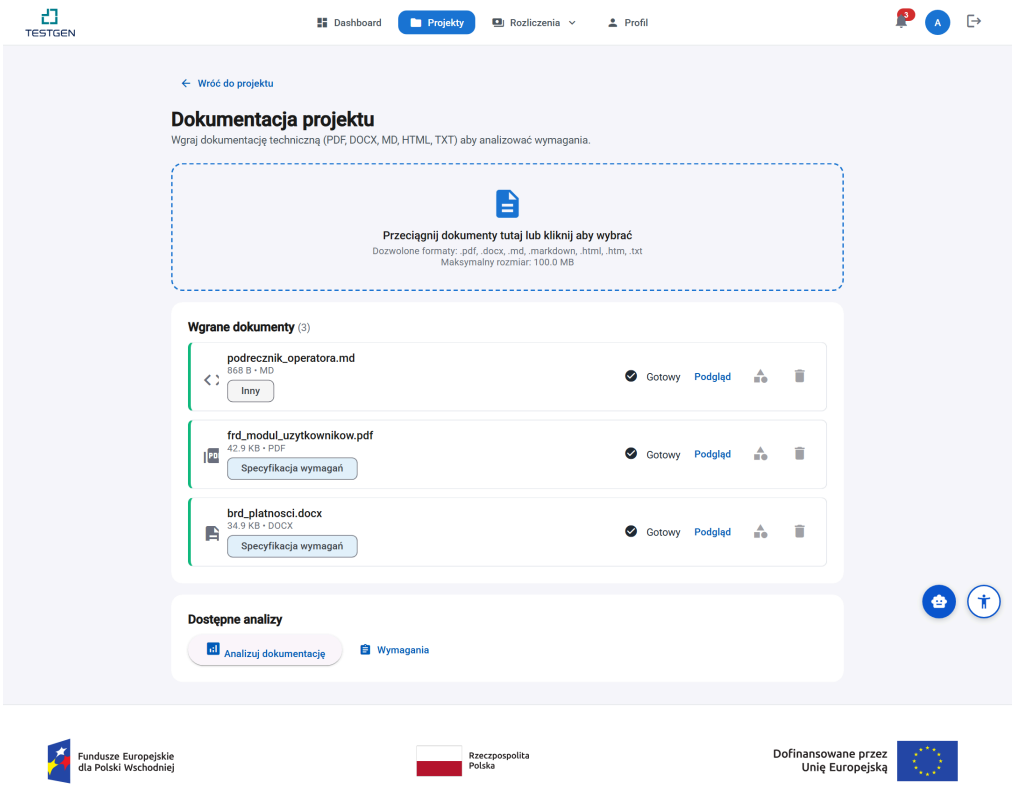
Format	Tekst	Tabele	Nagłówki	Uwagi
PDF	tak (warstwa tekstowa; skan → OCR)	nie	tak	do 500 stron; skany wolniejsze
DOCX	tak	tak	tak (style Nagłówek N)	tabele działają w pełni
Markdown	tak	tak (GFM)	tak (`#` – `######`)	—
HTML	tak	tak (`<table>`)	tak (`<h1>` – `<h6>`)	analizowany statycznie (bez JS)

Format	Tekst	Tabele	Nagłówki	Uwagi
TXT	tak	nie	nie	segmentacja po akapitach

Rozszerzenia: `.pdf`, `.docx`, `.md` / `.markdown`, `.html` / `.htm`, `.txt` (limit pliku 100 MB). Nieobsługiwane: `.doc`, `.rtf`.

5.2. Wgrywanie dokumentacji

Kafelek **„Dokumenty"** otwiera ekran przesyłania (drag & drop jak dla kodu). Każdy dokument przechodzi statusy **„Wysyłanie..."** → **„Przetwarzanie..."** → **„Gotowy"** (lub „Błąd" z przyczyną), a po przetworzeniu platforma **automatycznie klasyfikuje** jego typ: „Specyfikacja wymagań", „Podręcznik użytkownika", „Instrukcja techniczna", „Opis API", „Notatki projektowe" lub „Inny" — z procentem pewności (po najechaniu na etykietę). Pomyłkę poprawisz **„Klasyfikuj ponownie"**.

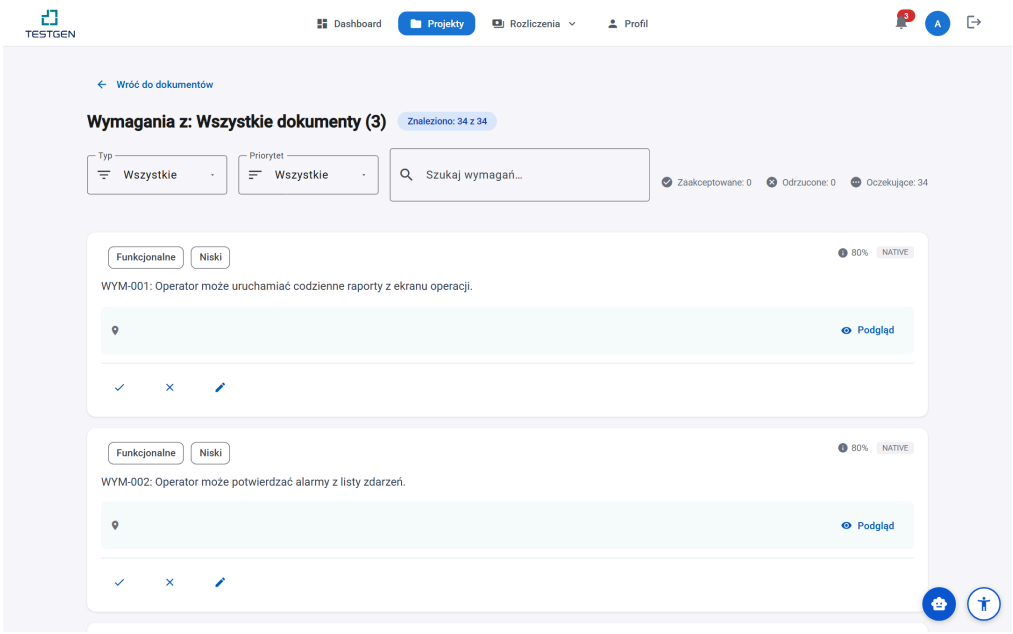


Każda karta pokazuje format, liczbę stron i rozpoznany typ dokumentu z pewnością klasyfikacji.

Skan zapisany jako PDF (bez warstwy tekstowej) jest rozpoznawany OCR-em — dłuższe dokumenty przetwarzają się wyraźnie dłużej niż pliki z tekstem. Analizę dokumentacji uruchamia przycisk **„Analizuj dokumentację"** (sekcja „Dostępne analizy"); działa w tle i zużywa miesięczny limit analiz. Powtórzenie analizy nie traci Twoich decyzji (zaakceptowane wymagania i zweryfikowane terminy zostają).

5.3. Wymagania

Ekran **„Wymagania z: {dokument}"** pokazuje listę wymagań z plakietką „Znaleziono: {n} z {m}" — widać, jaką część zdań platforma uznała za wymagania.



Każde wymaganie ma typ, priorytet, pewność i źródło w dokumencie.

Zdanie zostaje wymaganiem, gdy: ma identyfikator („FR-12", „WYM-3"), leży w sekcji wymagań albo brzmi jak wymaganie („system musi…"). Każde opisują: **typ** (Funkcjonalne/Niefunkcjonalne), **priorytet** (Krytyczny/Wysoki/Średni/Niski), **pewność** i **źródło** (fragment dokumentu z podglądem).

Wygenerowane wymagania to **propozycje** — dla każdego dostępne „Akceptuj wymaganie", „Odrzuć wymaganie", „Edytuj wymaganie" (treść i priorytet) oraz „Cofnij decyzję"; filtry Typ/Priorytet i szukajka zawężają listę, a statystyki („Zaakceptowane"/„Odrzucone"/„Oczekujące") pokazują postęp.

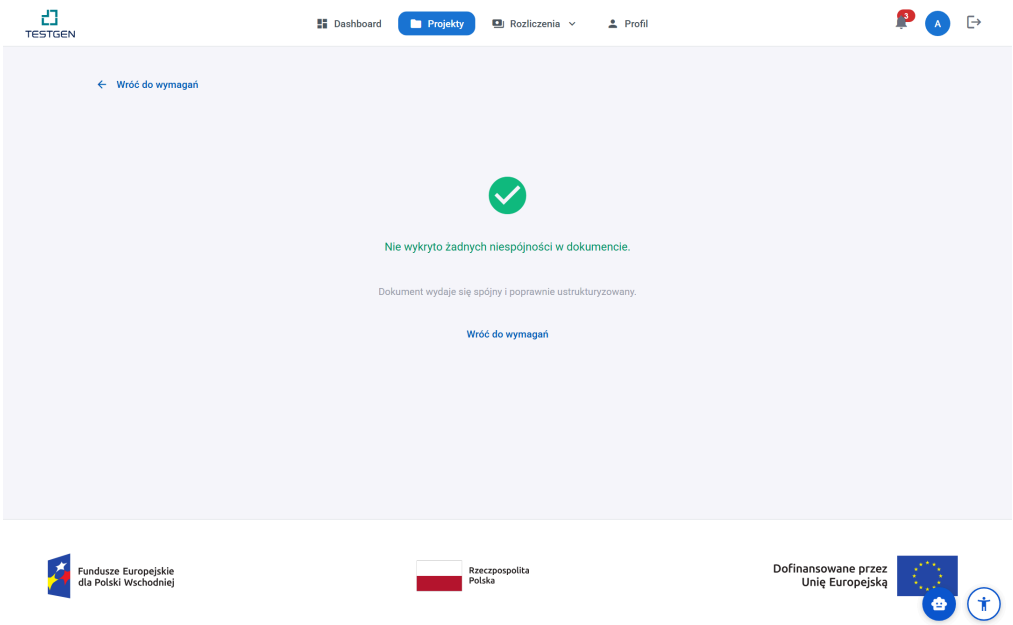
💡 **Wskazówka:** zaakceptowane wymagania mają znaczenie dalsze — to one porównywane są z kodem (pokrycie, compliance) i zasilają generowanie przypadków (5.7).

5.4. Niespójności

Ekran **„Niespójności: {dokument}"** zbiera problemy **wewnątrz dokumentacji** — zanim porównasz ją z kodem, warto tu posprzątać:

Typ	Co wykrywa	Typowa waga
Sprzeczności	wzajemnie wykluczające się zapisy („maksimum 5" vs „bez limitu")	krytyczna
Brakujące referencje	odwołanie do nieistniejącego identyfikatora	wysoka

Typ	Co wykrywa	Typowa waga
Duplikaty	ta sama treść w dwóch miejscach	średnia
Terminologia	jedna encja pod różnymi nazwami	średnia

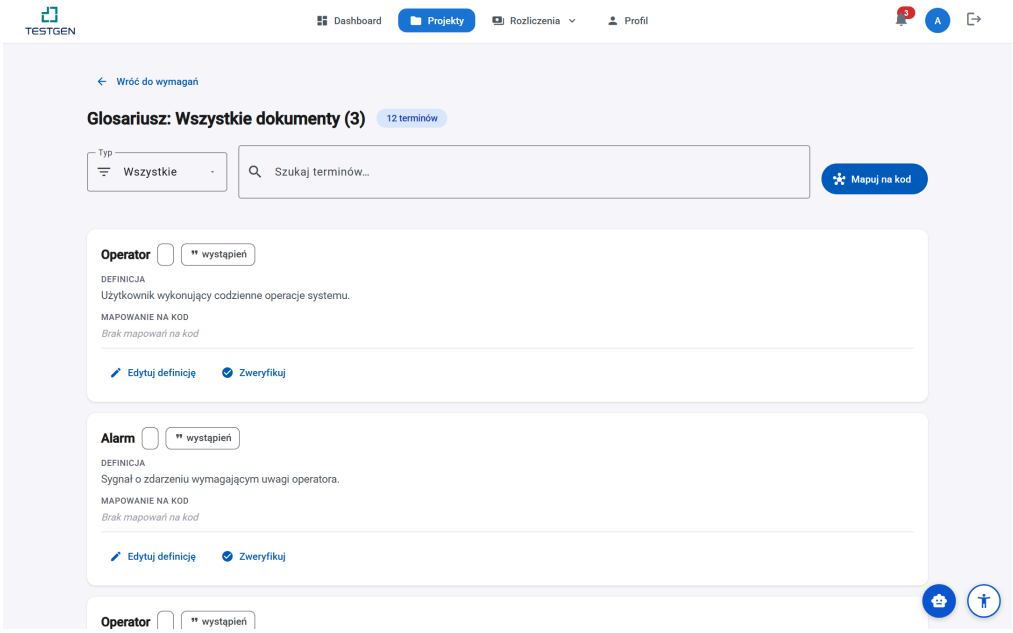


Każda niespójność ma opis, powiązane wymagania i sugestię poprawki.

Każda pozycja pokazuje opis, „Dotyczy wymagań" (linki) i „Sugestię". Statusy weryfikujesz przyciskami: „Otwarte" → „Zaakceptowane" (to realny problem), „False positive" (alarm fałszywy) lub „Naprawione".

5.5. Glosariusz

Ekran **„Glosariusz: {dokument}"** zbiera terminy domenowe: definicje z sekcji słownika, wzorce definicyjne oraz akronimy i terminy często powtarzane. Każdy termin ma typ („Dziedzina"/„Techniczny"/„Biznesowy"/ „Akronim"), liczbę wystąpień i edytowalną definicję; zweryfikowane dostają znacznik „Zweryfikowany".



Mapowanie terminu na encję kodu z procentem podobieństwa — potwierdzasz je jednym kliknięciem.

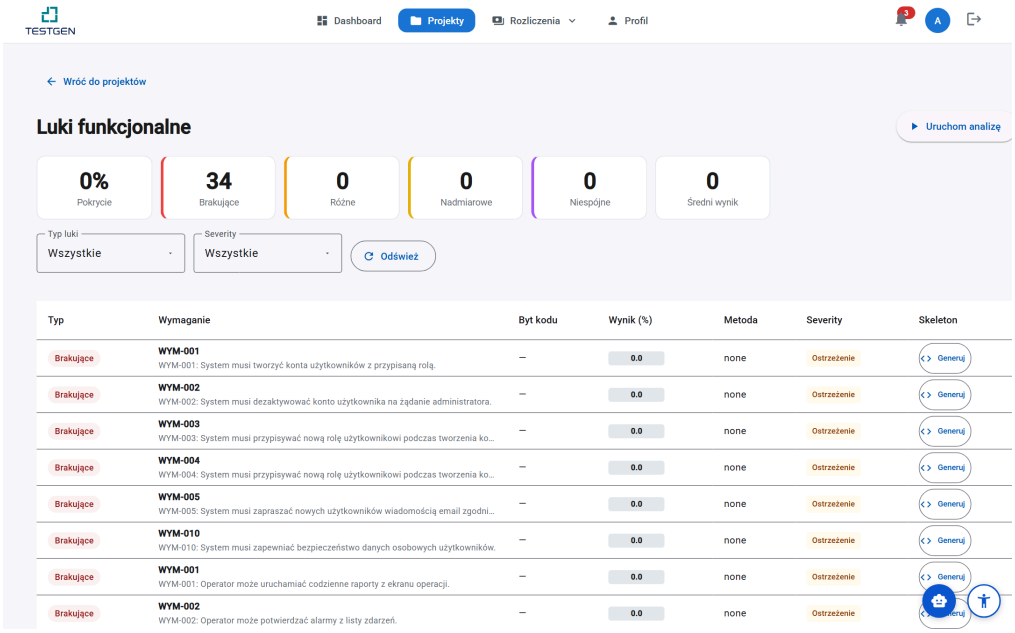
Przycisk **„Mapuj na kod"** dopasowuje terminy do encji wykrytych w kodzie projektu — do pięciu sugestii na termin z procentem podobieństwa, mapowanie potwierdzasz checkboxem. Widać, które pojęcia z dokumentacji mają odbicie w kodzie, a które są tylko na papierze.

5.6. Porównywanie dokumentacji z kodem (luki)

Najcenniejszy etap: **analiza luk funkcjonalnych** (plan Premium) odpowiada na pytanie „czy kod robi to, co obiecuje dokumentacja?".

Uruchomienie: kafelek **„Luki funkcjonalne"** → **„Uruchom analizę"** → w dialogu wybierz dokumenty z wymaganiami → analiza liczy się w tle (sekundy na małych projektach). Wymaga wcześniejszej analizy kodu i dokumentacji — bez nich dostaniesz „Brak wymagań" / „Brak bytów kodu".

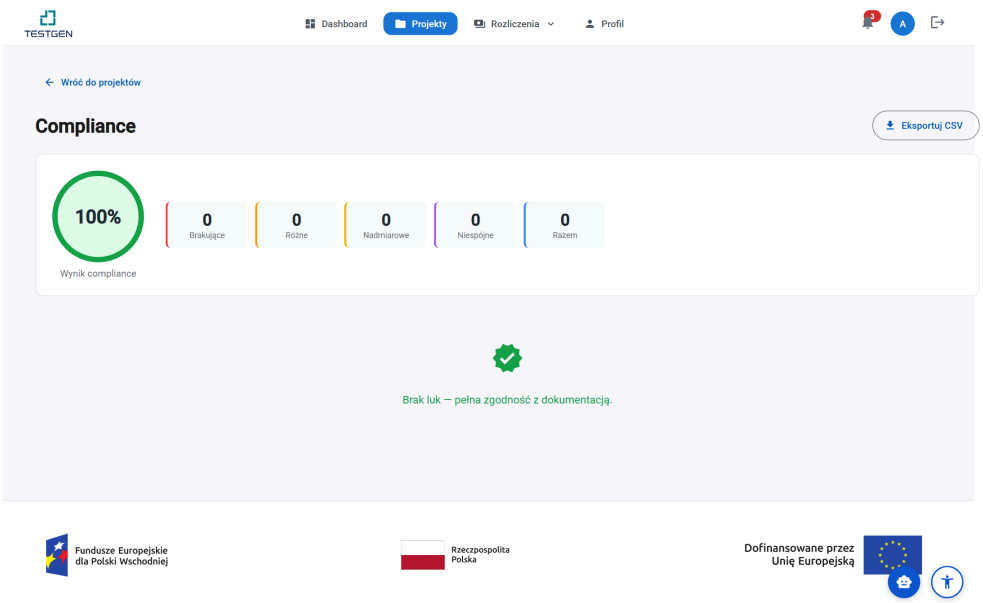
Tabela luk — Typ | Wymaganie | Byt kodu | Wynik (%) | Metoda | Severity | Skeleton — z filtrami i rozwinięciem (fragmenty wymaganienia i kodu, najlepszy kandydat, lokalizacja `plik:linia`):



Typ luki mówi, co zrobić: Brakujące — dopisać, Różne — poprawić, Nadmiarowe — zweryfikować.

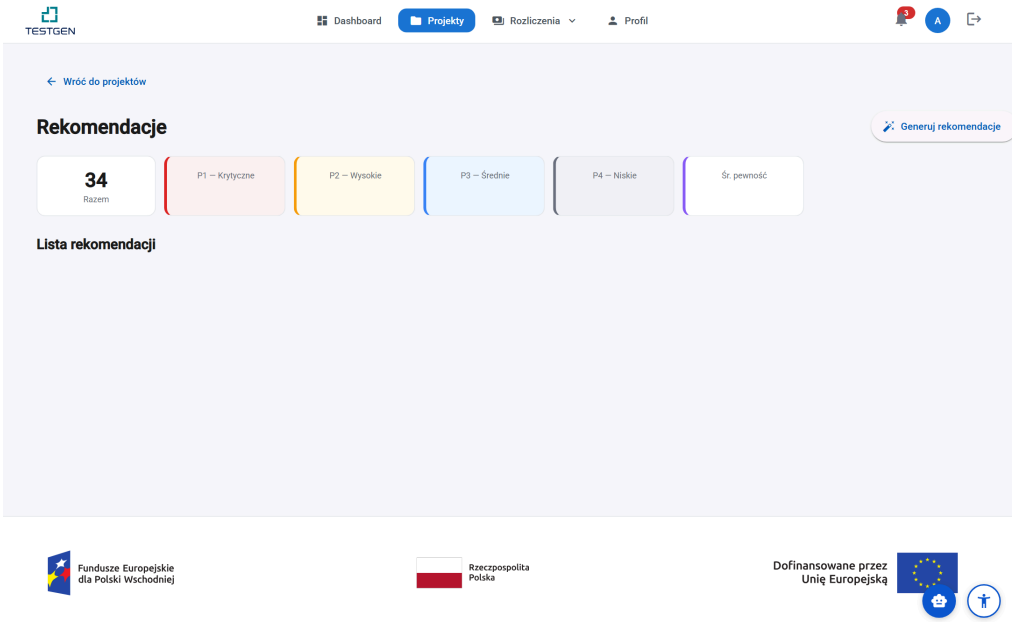
Typ luki	Co znaczy	Typowa akcja
Brakujące	wymaganie istnieje, kodu brak	zaplanuj implementację, wygeneruj szkielet
Różne	kod spełnia wymaganie częściowo (30–70%)	uzupełnij albo popraw nazwę
Nadmiarowe	kod bez wymagania	`@internal`, nowe wymaganie — albo usunięcie
Niespójne	jeden byt kodu mapuje się na sprzeczne wymagania	zawsze pilne — koordynuj z właścicielem produktu

Compliance. Wskaźnik zgodności 0–100 (100 = brak luk) z rozbiciem Brakujące/Różne/Nadmiarowe/Niespójne, tabelami dla modułów i funkcji oraz eksportem CSV — ekran „Compliance":



Wskaźnik 0–100 z rozbiciem luk na typy; wysoki wynik = kod pokrywa dokumentację.

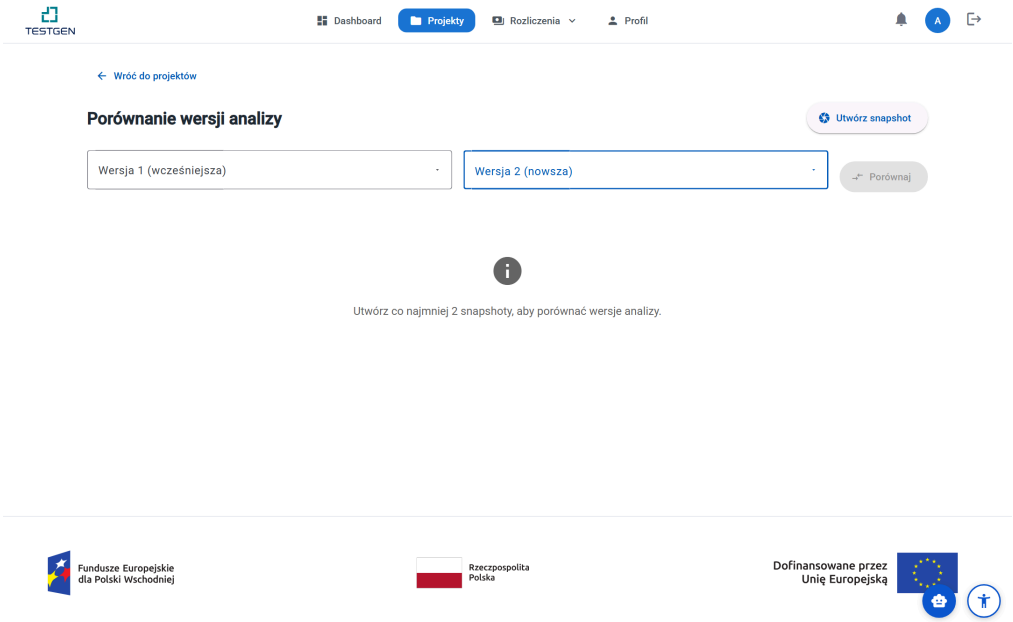
Rekomendacje i szkielety. Karta **„Rekomendacje"** („Generuj rekomendacje") zbiera sugestie z luk i błędów kodu w tabeli Priorytet (P1–P4) | Źródło | Tytuł | Pewność | Sugestia — karty podsumowania ostrzegają o niskiej pewności:



P1 to decyzje na dziś; niska pewność wymaga ludzkiego oka przed wdrożeniem.

Dla luk „Brakujące" kolumna **Skeleton** generuje **szkielet kodu** nowej funkcji/klasy (nazwa z wymagania, docstring z identyfikatorem, `NotImplementedError` do zastąpienia; języki: Python, TypeScript, Java, C#; składnia walidowana przed zwrotem). Rozwinięcie wiersza pokazuje kod z przyciskiem „Kopiuj".

Zrzuty stanu i trend. Karta **„Porównanie wersji"**: „Utwórz snapshot" (notowany wynik zgodności), wybór „Wersji 1 (wcześniejszej)" i „Wersji 2 (nowszej)", „Porównaj" → zmiana wyniku z deltą, plakietki „Naprawione"/ „Nowe"/„Bez zmian"/„Pogorszone", „Szczegóły luk" i wykres „Trend compliance":



Fundusze Europejskie dla Polski Wschodniej
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Dwa zrzuty stanu i wykres trendu — zgodność rośnie w dobrym kierunku?

💡 **Wskazówka:** najpierw posprzątaj wymagania (odrzuć śmieciowe) — luki liczone są z zaakceptowanych. „Niespójne" traktuj priorytetowo: sprzeczne wymagania mają dwie interpretacje i każda dalsza praca na nich mnoży koszt.

5.7. Generowanie testów z dokumentacji

Zaakceptowane wymagania zasilają generator przypadków testowych: dla wymagań funkcjonalnych powstają scenariusze pozytywny i negatywny, dla brzegowych warunków (jeśli wymaganie je opisuje) — brzegowe. Przypadki trafiają na wspólną listę projektu (rozdz. 8) ze statusem „Szkic", z priorytetem przejętym z wymagania — dzięki temu krytyczne wymaganie rodzi krytyczny test.

5.8. Dobre praktyki

- **Numeruj wymagania** (FR-1, WYM-2…) — identyfikator to najsilniejszy sygnał ekstrakcji i najpewniejsze ogniwo macierzy pokrycia.
- **Buduj hierarchię nagłówków** — zwykły tekst bez nagłówków dzieli się tylko na akapity.
- **Jedna nazwa dla jednego pojęcia** — synonimy rodzą „Niespójności: Terminologia" i rozmywają mapowanie na kod.
- **PDF z warstwą tekstową zamiast skanu**; monstrualne dokumenty dziel (limit 500 stron).
- **Zamknij pętlę:** rekomendacja → szkielet → implementacja → ponowna analiza → nowy zrzut stanu.

5.9. Ograniczenia

- **Tabele z PDF nieodczytywane** — dane w tabelach przenieś do DOCX/MD/HTML.

- **Skan PDF** zależy od jakości obrazu; diakrytyki bywają zniekształcone, układ wielokolumnowy zmienia kolejność zdań.
- **TXT bez nagłówków** — wymagania wykrywane wyłącznie po treści zdań.
- **HTML statyczny** — treść dogenerowywana JavaScriptem niewidoczna.
- **Makra dokumentów** (VBA) pomijane; dopasowanie wymaganie ↔ kod jest przybliżone (wynik % i metoda pokazują pewność).

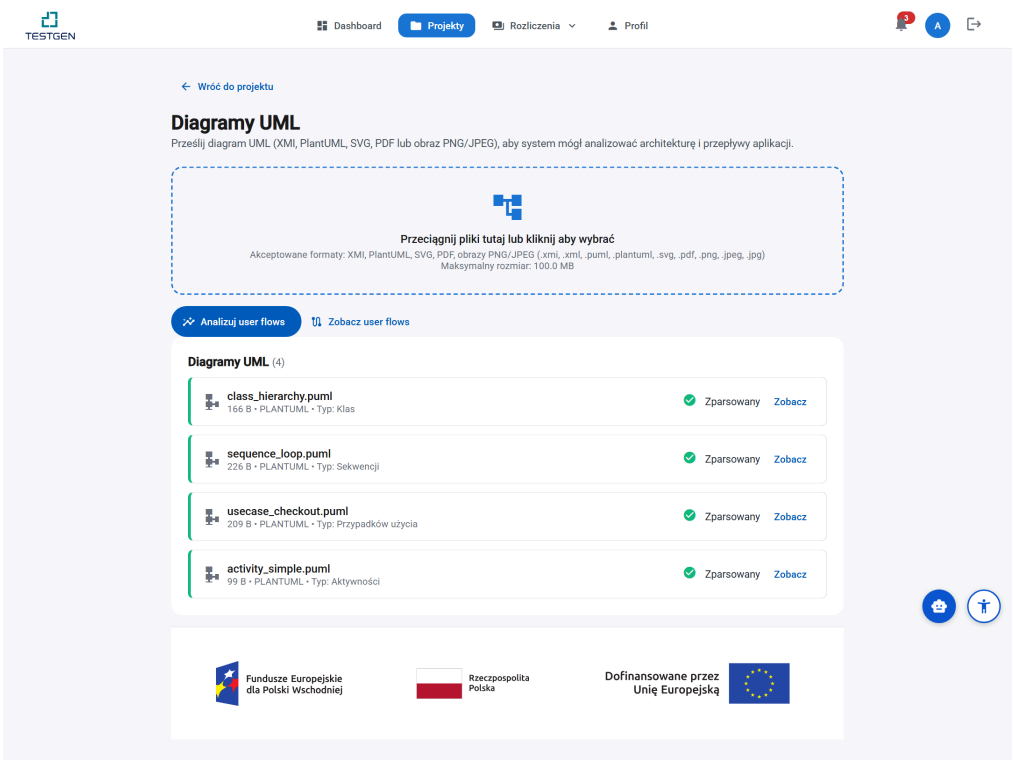
Rozdział 6. Analiza UML i GUI

Diagramy UML opisują architekturę i przepływy, zrzuty ekranu GUI — interfejs widziany przez użytkownika. Platforma sprowadza oba do wspólnego mianownika (ustrukturyzowanych elementów) i wiąże je z przepływami oraz scenariuszami testowymi. Moduły wymagają planu Basic lub wyższego.

6.1. Wgrywanie diagramów UML

Kafelek **„Diagramy UML"** przyjmuje dwa rodzaje plików:

Rodzaj	Formaty	Jak analizowany
Źródła diagramów	`.puml`, `.plantuml`, `.xmi`, `.xml`	pełna ekstrakcja struktury
Rysunki	`.svg`, `.pdf`, `.png`, `.jpeg`, `.jpg`	analiza wizualna: OCR tekstu + rozpoznawanie kształtów



Statusy: Oczekuje, Parsowanie, Zparsowany lub Błąd; „Zobacz" otwiera widok danego typu diagramu.

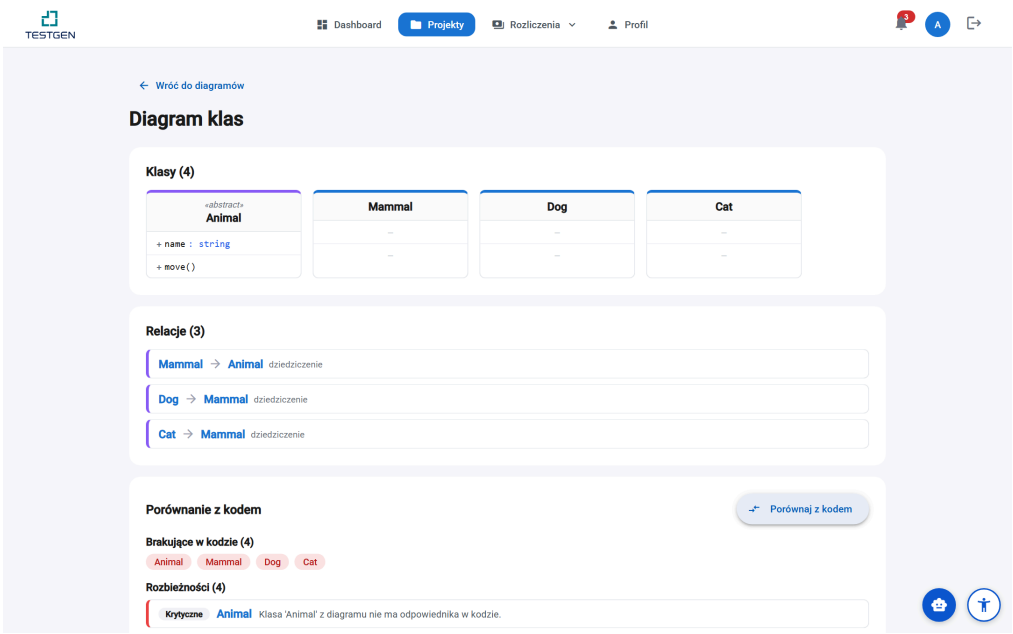
Obsługiwane typy: **sekwencji, przypadków użycia, klas, aktywności i stanów** — typ rozpoznawany automatycznie (źródła: zawsze poprawnie; rysunki: z pewnością rozpoznania). Po sparsowaniu przycisk **„Analizuj user flows"** łączy wiele diagramów w przepływy użytkownika.

6.2. Widoki diagramów

Każdy „Zparsowany" diagram otwiera widok dopasowany do typu:

Typ	Co zobaczysz
Sekwencji	Uczestnicy, Wiadomości, Fragmenty, Ścieżki testowe
Przypadków użycia	Aktorzy, Przypadki użycia, Granica systemu, Relacje, Scenariusze
Klas	Klasy (atrybuty i metody), Interfejsy, Enumeracje, Relacje, Pakiety
Aktywności	Węzły, Tory odpowiedzialności, Przepływy, Ścieżki testowe
Stanów	Stany, Stany złożone, Przejścia, Scenariusze (pokrycie przejść, negatywne)

Sercem widoku klas jest **„Porównanie z kodem"** — zestawienie klas z diagramu z klasami z kodu (rozdz. 4.6):

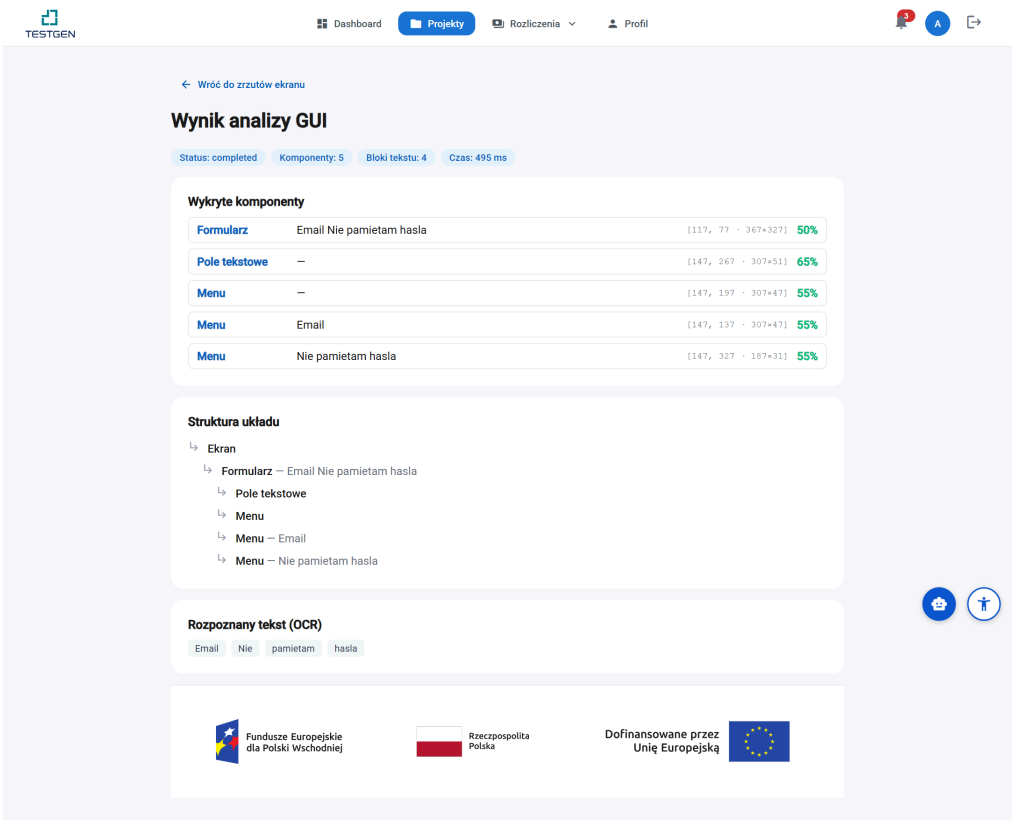


Dopasowane klasy z procentami zgodności atrybutów i metod; osobno brakujące po obu stronach.

„Dopasowane klasy" (zgodność atrybutów/metod w %), „Brakujące w kodzie", „Brakujące na diagramie", „Rozbieżności" — najszybszy audyt spójności „projekt vs implementacja".

6.3. Analiza interfejsów użytkownika

Kafelek **„Zrzuty ekranu GUI"** przyjmuje obrazy PNG, JPEG, BMP, TIFF, WebP (do 20 MB). Analiza kończy się ekranem **„Wynik analizy GUI"**:



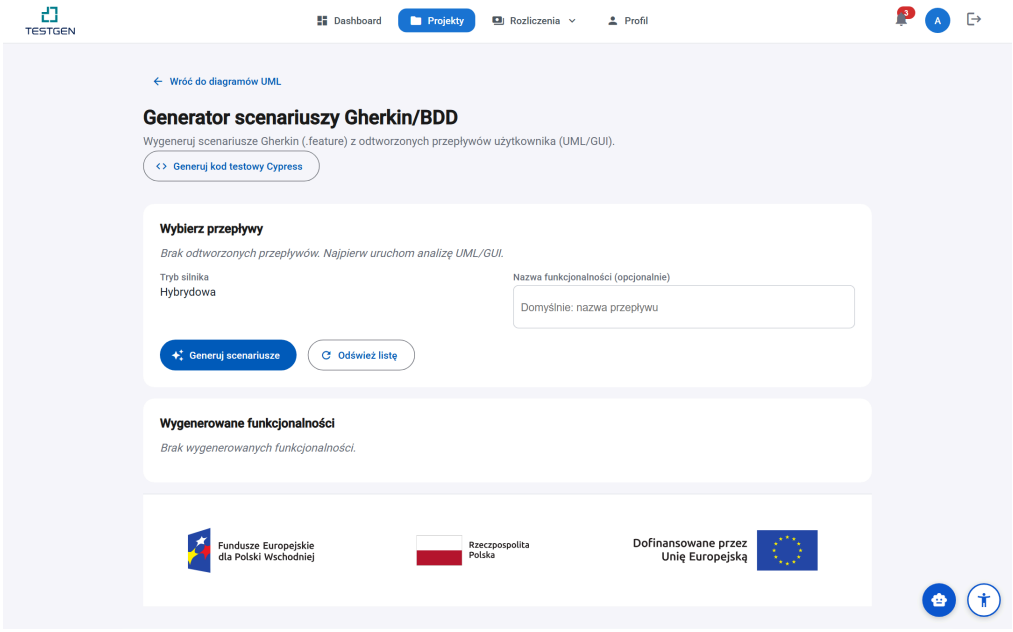
Komponenty z typem, etykietą, położeniem i pewnością rozpoznania; drzewo układu pokazuje zagnieżdżenie.

- **„Wykryte komponenty"** — typ (Przycisk, Pole tekstowe, Lista rozwijana, Pole wyboru, Przycisk opcji, Menu, Łącze, Tabela, Formularz, Okno modalne, Zakładka), etykieta z sąsiedztwa, położenie i pewność,
- **„Struktura układu"** — drzewo zagnieżdżenia (co zawiera co),
- **„Rozpoznany tekst (OCR)"** — bloki tekstu z położeniem.

Ekran ma też sekcję **źródeł HTML/DOM** („Wybierz pliki HTML") — wgrane widoki aplikacji pozwalają wiązać komponenty z realnymi elementami strony (pomocne przy samonaprawianiu testów, Część 3, rozdz. 11).

6.4. Generowanie testów UI (Gherkin/BDD)

Z kilku zrzutów (w kolejności numerów ekranów) i diagramów platforma buduje **przepływy użytkownika**, a generator **„Scenariuszy Gherkin/BDD"** zamienia je na scenariusze BDD: zaznacz przepływy („Wybierz przepływy"), nadaj opcjonalną nazwę funkcjonalności, **„Generuj scenariusze"** — lista pokaże pliki `.feature` ze statusem („Wygenerowano" → „Zweryfikowano" → „Zatwierdzono"), ostrzeżeniami walidacji i podglądem z **„Pobierz .feature"**. Z tej strony prowadzi też link do generatora kodu Cypress/SikuliX (rozdz. 8.5).



Przepływy → scenariusze Gherkin: kroki czyta cały zespół, nie tylko automatyzer.

6.5. Testy regresji GUI

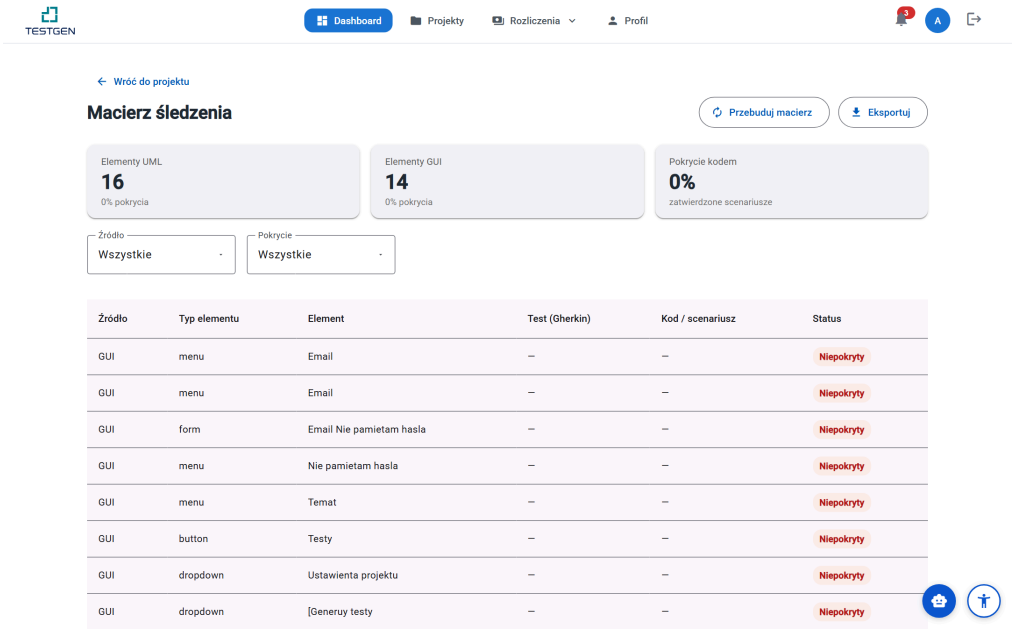
Dwa zrzuty tego samego ekranu — przed i po zmianie — porówna ekran **„Regresja wizualna"** (link na liście zrzutów GUI):

- **„Obraz różnicy"** — wizualna mapa różnic pikselowych,
- **„Wykryte zmiany"** — usunięte/dodane/przesunięte komponenty, zmiana układu lub tekstu,
- **„Wygenerowany kod testów"** — szkielet testu stabilności ekranu.

Wynik dostaje status: **istotna różnica**, **drobna różnica** lub **brak różnic** — szybki werdykt, czy zmiana interfejsu była zamierzona.

6.6. Macierz śledzenia

Kafelek **„Macierz śledzenia"** zbiera elementy UML i komponenty GUI → przepływy → zatwierdzone scenariusze Gherkin w jedną tabelę (filtry „Źródło" i „Pokrycie"; „Przebuduj macierz" odświeża; „Eksportuj" CSV/JSON/XLSX/PDF):



Przycisk „Przebuduj macierz" odświeża powiązania; eksport w czterech formatach.

⚠️ **Uwaga:** to macierz **źródeł UML/GUI** — odrębna od macierzy wymagań dokumentacji (5.6).

6.7. Dobre praktyki

- **Przesyłaj źródła, nie rysunki** (PlantUML/XMI > obraz).
- **Rysuj czytelnie** — duże, niekolidujące kształty, poziomy tekst.
- **Zrzuty GUI w naturalnej rozdzielczości** — bez przeskalowań i kompresji; ciemny i jasny motyw to dwa różne „ekrany".
- **Utrzymuj kolejność ekranów** — numery budują przepływy.
- **Etykietuj interfejs** — przycisk z tekstem rozpoznawalny pewniej niż ikona.

6.8. Ograniczenia

- **Z rysunku nie są odczytywane wnętrza klas** (atrybuty/metody) — wyłącznie z PlantUML/XMI.
- **Interfejs vs klasa vs wyliczenie** — z rysunku wszystkie prostokąty to klasy.
- **Diagramy stanów z rysunku** bywają brane za aktywności.
- **Detekcja komponentów heurystyczna** — najpewniej: przycisk, pole tekstowe, lista rozwijana, pole wyboru, menu, formularz; nietypowe kontrolki wypadną jako „Nieznany".
- **Brak minimalnej rozdzielczości** — formalny próg nie istnieje, ale jakość decyduje o wyniku.

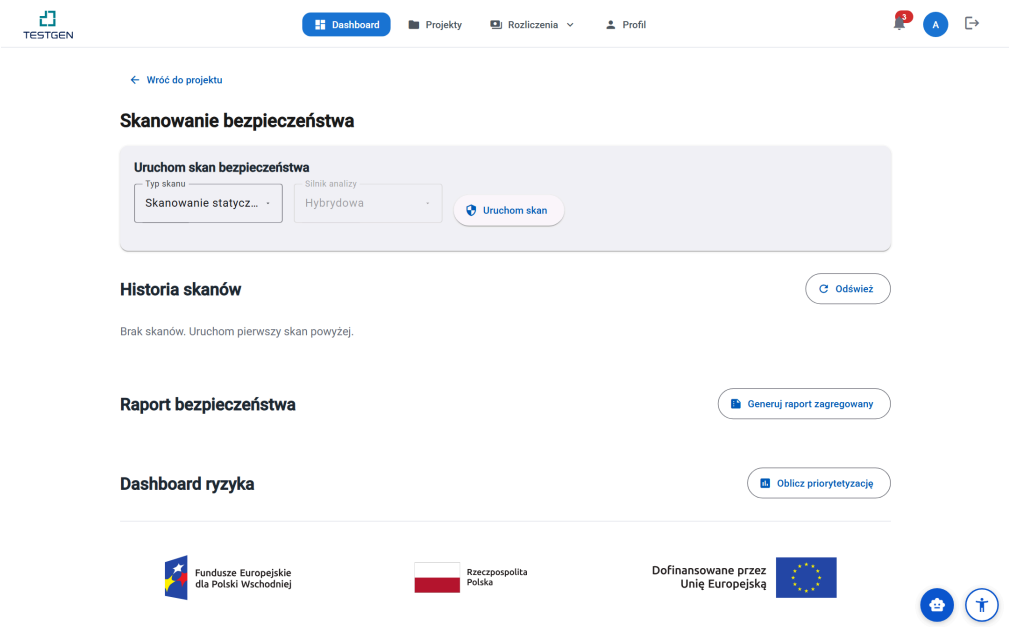
Rozdział 7. Analiza bezpieczeństwa

Moduł bezpieczeństwa (plan **Premium**) skanuje kod, zależności, konfigurację — a nawet działającą aplikację — w poszukiwaniu podatności, a następnie proponuje naprawy z gotowym kodem. Skany działają asynchronicznie i deterministycznie.

7.1. Typy skanów

Kafelek **„Skanowanie bezpieczeństwa"** otwiera kartę „Uruchom skan bezpieczeństwa" z listą **„Typ skanu"**:

Typ skanu	Co robi
„Skanowanie statyczne (SAST)"	przeszukuje kod regułami wzorców (SQL injection, XSS, CSRF, autoryzacja, sekrety…)
„Taint analysis (przepływ danych)"	śledzi dane od źródła do ujścia — czy niebezpieczne dane dochodzą do SQL/HTML/ścieżek bez oczyszczenia
„Zależności (CVE / SBOM)"	porównuje zależności z bazą znanych podatności
„Konfiguracja (Docker / K8s / Terraform)"	szuka błędnych konfiguracji
„Dynamiczny (DAST / OWASP ZAP)"	skanuje **działającą** aplikację pod adresem URL (do 30 minut)



Pięć typów skanów; dla DAST podajesz adres aplikacji i opcjonalną autentykację.

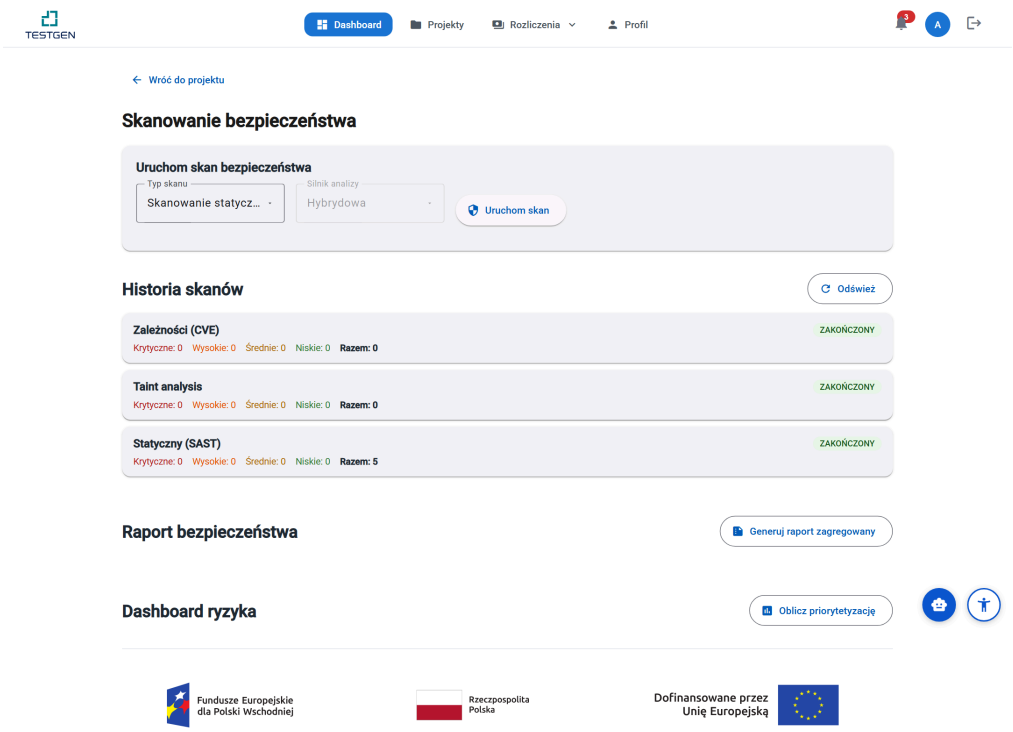
DAST wymaga **„URL aplikacji docelowej"** oraz — dla chronionych ścieżek — autentykacji („Bez autentykacji" / „Formularz (login)" / „Token (Bearer / JWT)" / „Basic auth"). Skanuj wyłącznie aplikacje, do których masz prawo testowania.

7.2. Uruchamianie

1. Wybierz **„Typ skanu"** i kliknij **„Uruchom skan"** (DAST: „Uruchom skan DAST").
2. **„Historia skanów"** pokaże kartę ze statusem „Oczekuje" → „W toku" → „Zakończony"/„Błąd" (przycisk „Odśwież").
3. Po zakończeniu karta pokazuje liczniki: „Krytyczne:", „Wysokie:", „Średnie:", „Niskie:", „Razem:".

Pod historią czeka tabela **„Podatności"**: Waga | Typ podatności | Plik | Linia | Pewność — z identyfikatorem CWE przy typie (np. „SQL Injection (CWE-89)").

7.3. Interpretacja wyników



Waga, lokalizacja w pliku i pewność — wpisy z niską pewnością traktuj ostrożnie.

Każde znalezisko opisują pola, które warto znać przy przeglądzie zgłoszeń:

Pole	Znaczenie
Waga	Krytyczna / Wysoka / Średnia / Niska / Info
CWE	identyfikator kategorii podatności
Kategoria OWASP	przypisanie do OWASP Top 10 (2021)
Pewność	0–100%; rośnie, gdy to samo wykrywa kilka skanów (korelacja SAST↔DAST to najmocniejszy sygnał)
CVSS	ocena 0–10 z wektorem; pasma: ≥9 krytyczna, 7–8,9 wysoka, 4–6,9 średnia, 0,1–3,9 niska
Status	„Otwarte" → „Potwierdzone" / „Naprawione" / „Fałszywe"

Dodatkowo: **taint** rysuje ścieżki `źródło → ujście` z ostrzeżeniami („Brak sanitizera"), **skan konfiguracji** pokazuje błędy z gotową „Poprawną konfiguracją" jako fragmentem kodu.

💡 **Wskazówka:** wysoka pewność + kilka źródeł = mała szansa na fałszywy alarm. Odwrotnie — niska pewność przy statycznym skanie to kandydat do weryfikacji przed naprawą.

7.4. Zależności i SBOM

Skan „Zależności (CVE / SBOM)" analizuje manifesty projektu (`requirements.txt` , `package.json` , `pom.xml` , `composer.json` , `.csproj`). Raport „Podatne zależności (CVE)" — Waga | Pakiet | Wersja | CVE | CVSS | Poprawka (wersja naprawiająca lub „brak"):

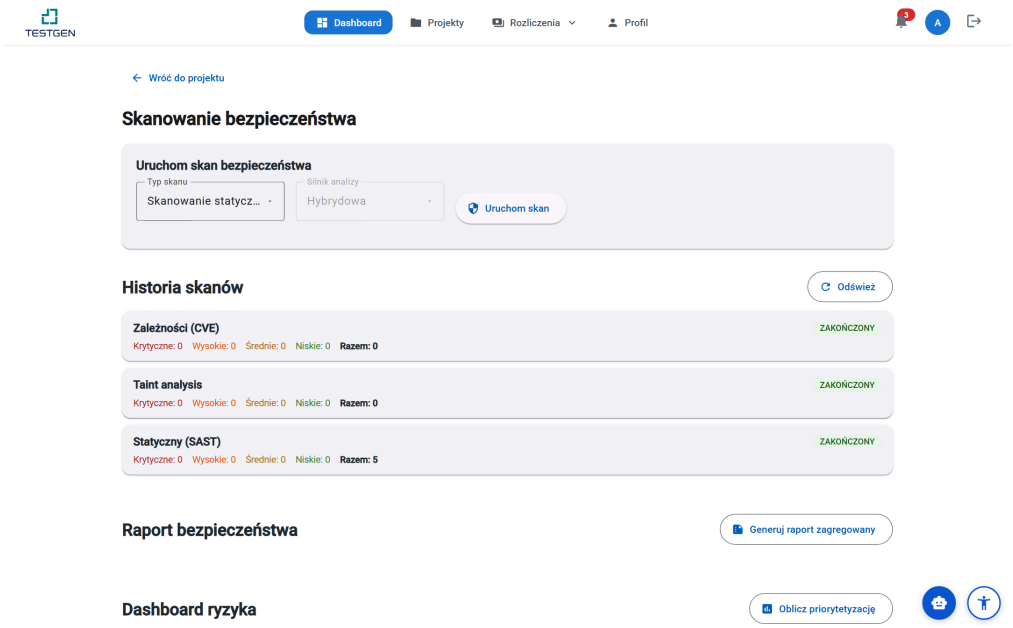


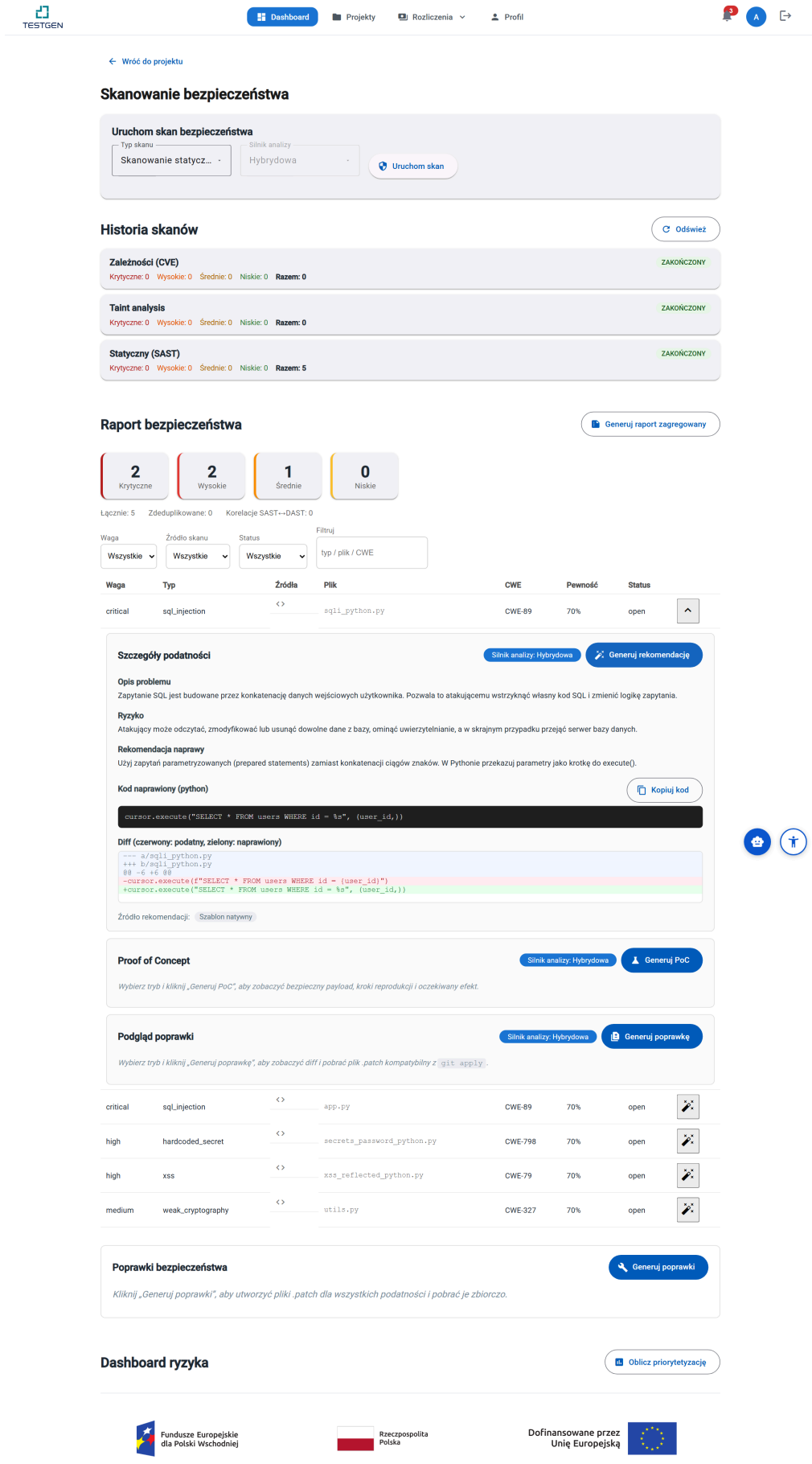
Tabela CVE z wersjami naprawiającymi; SBOM pobierzesz w dwóch standardach.

Przyciski **„SBOM (CycloneDX)"** i **„SBOM (SPDX)"** pobierają bilans materiałów projektu — pełną listę komponentów (CycloneDX 1.5 lub SPDX 2.3, JSON). Dane CVE pochodzą z NVD, OSV.dev i GitHub Advisory, synchronizowanych miesięcznie do lokalnej bazy — skan działa offline.

7.5. Rekomendacje naprawcze

Rozwiń wiersz podatności — dostaniesz trzy narzędzia:

- **Rekomendacja** — „Opis problemu", „Ryzyko", „Rekomendacja naprawy", „Kod naprawiony" (z „Kopiuj kod") i **diff** czerwono-zielony (podatny → naprawiony):



Rekomendacja z gotowym kodem naprawy i diffem — kopiujesz i commitujesz.

- **Proof of Concept** — dla klasycznych podatności: Payload, Kroki reprodukcji, Oczekiwany efekt; wzorce destrukcyjne blokuje bezpiecznik („Bezpieczny").
- **Poprawka** — plik `.patch` (unified diff) z podglądem i „Pobierz plik .patch"; sekcja „Poprawki bezpieczeństwa" generuje wszystkie naraz i ostrzega o konfliktach nakładających się linii (wtedy stosuj pojedynczo, przez `git apply`).

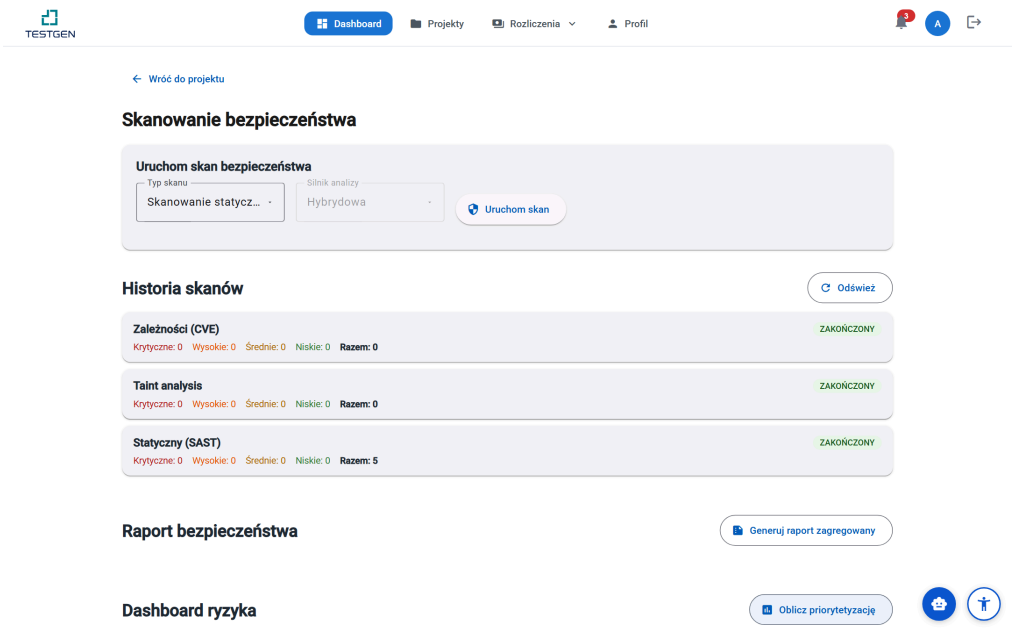
Testy bezpieczeństwa. Znaleziska zasilają też generowanie przypadków typu „Bezpieczeństwa" (rozdz. 8) — scenariusze weryfikujące poprawkę (np. próba wstrzyknięcia po naprawie).

7.6. Priorytetyzacja

Karta **„Dashboard ryzyka"** („Oblicz priorytetyzację") porządkuje naprawy:

- **„Rozkład CVSS"** — histogram ocen wg wag,
- **„Top 10 zagrożeń"** — najgroźniejsze znaleziska z CVSS,
- **„Rekomendowana kolejność napraw"** — tabela # | Typ | Waga | CVSS | Nakład pracy (Niski/Średni/Wysoki).

Kolejność liczy się jako: najpierw najpoważniejsze, najpewniejsze i najtańsze („szybkie zwycięstwa"). Podbite wagi: znalezisko potwierdzone przez SAST **i** DAST („Potwierdzona dynamicznie") albo zależność z publicznym exploitem (CVSS ≥ 9).



Rekomendowana kolejność napraw łączy wagę, pewność i koszt — realizuj od góry.

7.7. Dobre praktyki

- **Skanuj po każdej istotnej zmianie** — zwłaszcza nowej zależności.
- **Zacznij od „szybkich zwycięstw"** z kolejności napraw.
- **Oznaczaj fałszywe alarmy** — kolejne przeglądy krótsze, statystyki czystsze.
- **DAST tylko na środowisku testowym** — dynamiczny test strzela payloadami w Twoją aplikację.

7.8. Ograniczenia

- **Reguły SAST pokrywają popularne stosy** — egzotyczne języki przejdą bez znalezisk (fałszywy spokój).
- **DAST wymaga działającej aplikacji** (do 30 minut); głębokie przepływy za autoryzacją wymagają konfiguracji logowania.
- **Taint jest statyczny** — dynamiczne źródła danych widzi tylko DAST.
- **PoC jest dowodem, nie exploitem** — destrukcyjne wzorce blokowane.

Rozdział 8. Generowanie testów

Przypadki testowe i ich kod to najcenniejszy wynik pracy platformy. Ten rozdział opisuje pełny cykl: wybór frameworka, przegląd i edycję, statusy, operacje masowe, generowanie kodu i eksport.

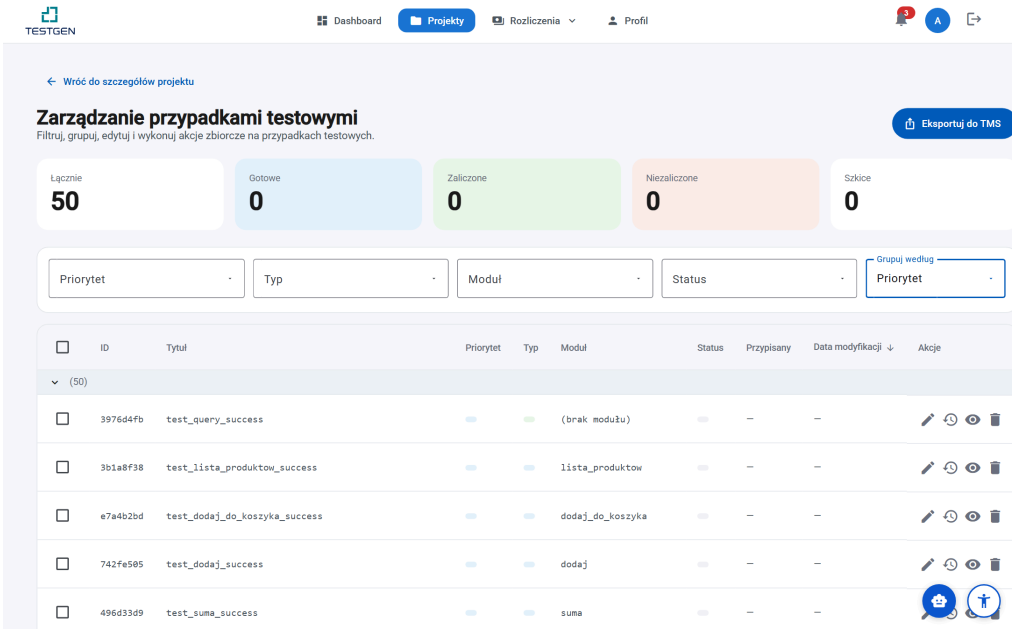
8.1. Wybór frameworka testowego

Framework wybierasz w **„Parametrach analizy"** (4.3) — „— Automatyczny —" wykrywa z plików konfiguracyjnych (projekty unittest/TestNG/MSTest dostają kod we własnym frameworku). Dostępne generatory kodu (pełna lista w 8.5): pytest, unittest, JUnit, TestNG, NUnit, MSTest, Mocha/Jest, Cypress, Selenium, Playwright, Appium, SikuliX, PHPUnit/Pest oraz testy bazodanowe (pytest).

Przypadki powstają z dwóch źródeł: **z kodu** (scenariusz pozytywny dla każdej funkcji, negatywne przy walidacjach, brzegowe dla parametrów liczbowych i kolekcji, integracyjne przy wywołaniach zewnętrznych) i **z wymagań** (5.7). Generator nadaje **typ** („Jednostkowy", „Integracyjny", „E2E", „Pozytywny", „Negatywny", „Brzegowy" i inne) oraz **priorytet** regułowo — np. funkcja z autoryzacją dostaje wyższy priorytet niż pomocnicza konwersja typu. Priorytet to wskazówka: w edytorze zmienisz go jednym kliknięciem.

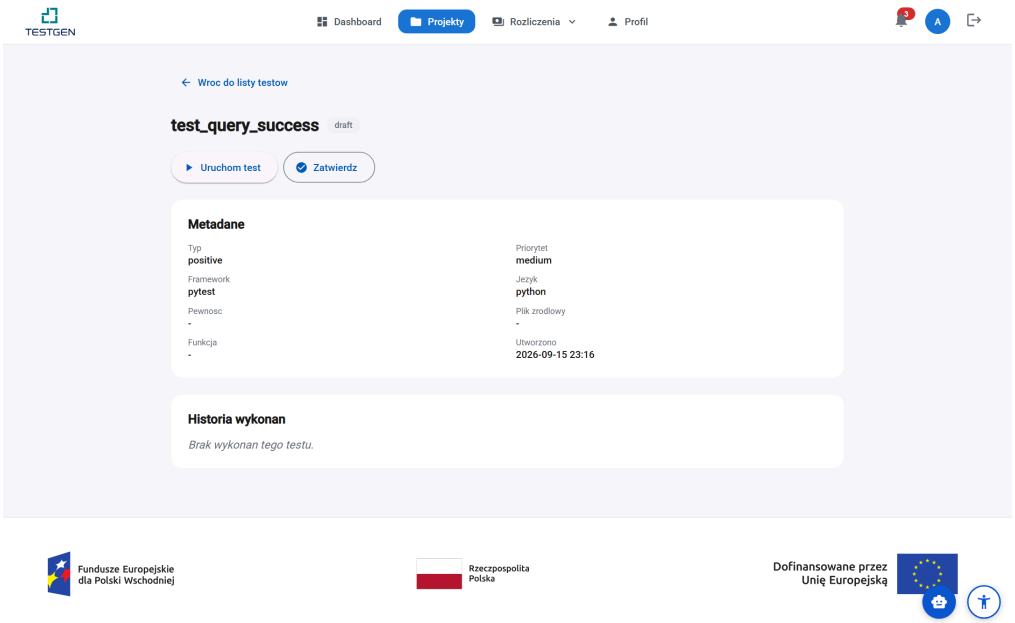
8.2. Przeglądanie i edycja przypadków

Kafelek **„Przypadki testowe"** otwiera tabelę **Tytuł | Typ | Priorytet | Status** z szukaczką i **„Grupuj według"** (Brak grupowania / Moduł / Priorytet / Typ):



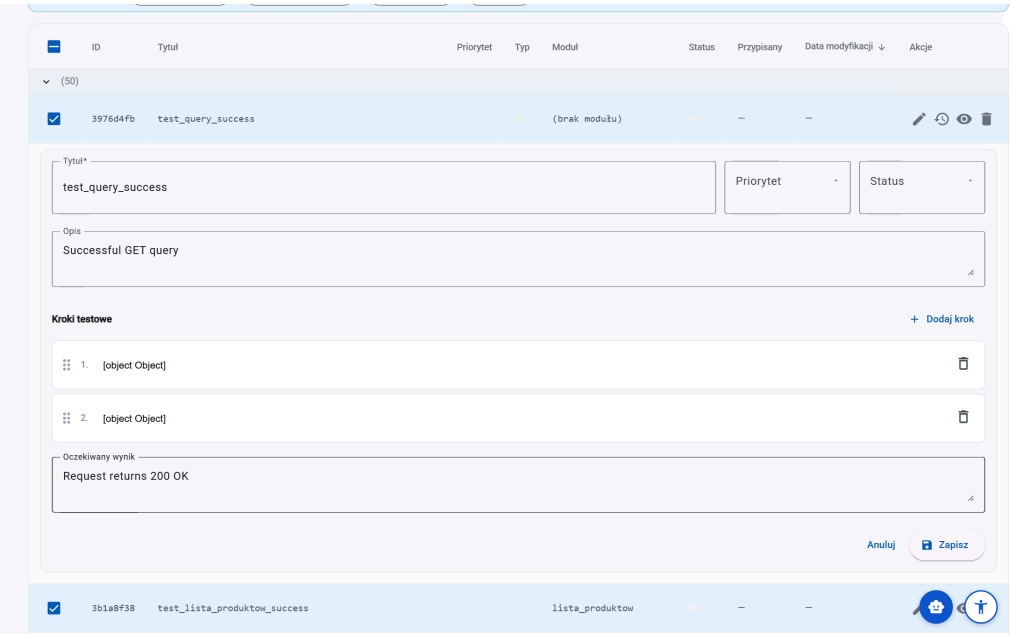
Grupowanie po priorytecie porządkuje przegląd — zacznij od grupy „Krytyczny".

Kliknięcie wiersza otwiera szczegóły: przyciski **„Uruchom test"** i **„Zatwierdź"**, kartę **„Metadane"** (Typ, Priorytet, Framework, Język, Pewność, Plik źródłowy, Funkcja, Utworzono — pełne powiązanie z kodem), kartę **„Kod testu"** i tabelę **„Historia wykonań"** (Data, Status, Czas ms, Pokrycie %):



Metadane mówią, skąd przypadek pochodzi; historia wykonań — co się stało po uruchomieniu.

Edytor (z listy) pozwala poprawić **Tytuł** (wymagany), **Priorytet** i **Status**, **Opis**, **Kroki testowe** („Dodaj krok", usuwanie, przeciąganie kolejności) oraz **Oczekiwany wynik** — „Zapisz" zapisuje każdą zmianę do historii zmian (dialog „Historia zmian — {tytuł}": Data, Autor, Pole, Przed, Po):



Kroki przeciągasz w kolejność rzeczywistej ścieżki testu; edytor zapisuje historię zmian.

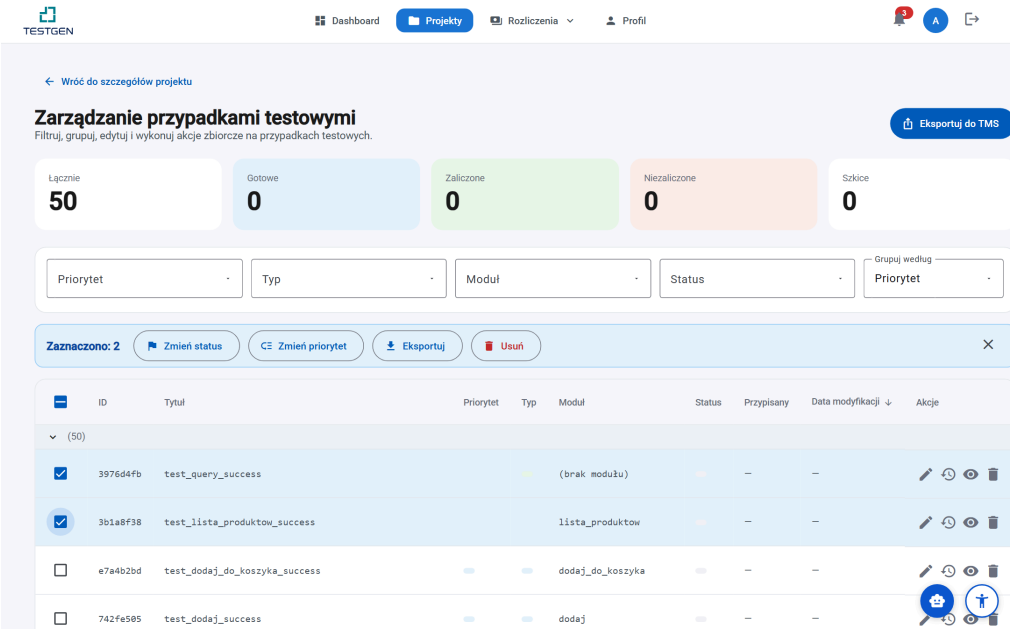
8.3. Statusy i przegląd

Status	Znaczenie
„Szkic"	świeżo wygenerowany — wymaga przeglądu
„Gotowy"	zatwierdzony („Zatwierdź")
„Zaliczony" / „Niezaliczony"	wynik uruchomienia („Uruchom test")
„Pominięty"	świadomie pominięty w biegu
„Wyłączony"	wyłączony z dalszych uruchomień

⚠️ **Uwaga:** nie eksportuj „Szkiców" do zespołu przed przeglądem — generator woli zaproponować więcej niż przeoczyć, więc część przypadków będzie zbędna. Po każdej analizie przejrzyj paczkę: zatwierdź sensowne, wyłącz śmieciowe.

8.4. Operacje masowe

Zaznacz wiele wierszy — pasek „Zaznaczono: N" z akcjami **„Zmień status"** (np. zatwierdzenie paczki), **„Zmień priorytet"**, **„Eksportuj"** (8.6) i **„Usuń"**:



Operacje masowe działają na zaznaczonych wierszach; „Wyczyść zaznaczenie" zamyka pasek.

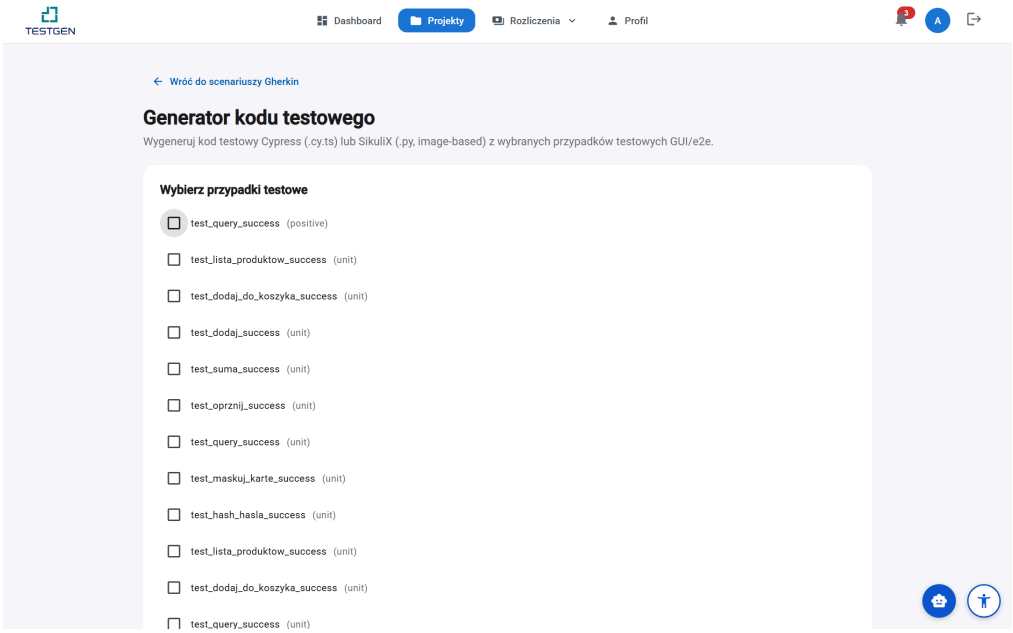
8.5. Generowanie kodu testów

Kod powstaje na dwa sposoby.

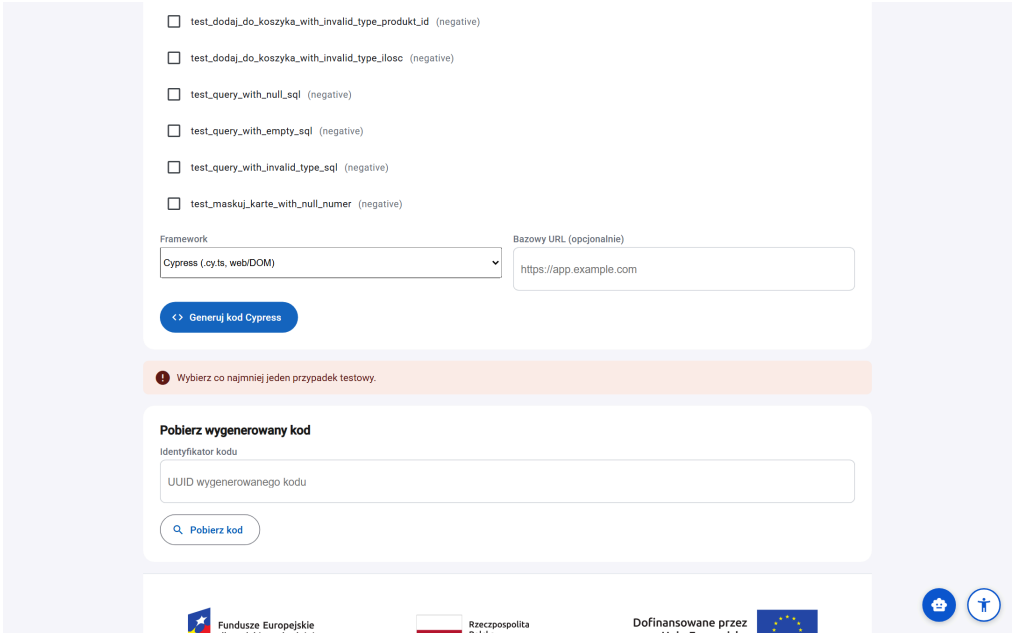
Automatycznie z analizy — w frameworku z „Parametrów analizy"; kod zobaczysz w karcie „Kod testu" szczegółów przypadku.

Generatory specjalistyczne (ścieżki `/codegen/...`, link ze strony Gherkin):

- **„Generator kodu testowego" (8 frameworków: Cypress/SikuliX/Selenium/ Playwright/Appium + unittest/TestNG/MSTest)** — zaznacz przypadki i wybierz framework z listy: „Cypress (.cy.ts, web/DOM)", „SikuliX (.py, image-based)", „Selenium (.py, E2E web)", „Playwright (.spec.ts, E2E web)", „Appium (.py, E2E mobilne)", „unittest (.py, TestCase)", „TestNG (.java, adnotacje)" lub „MSTest (.cs, [TestClass])"; dla Cypressa opcjonalnie „Bazowy URL", dla Appium „Platforma urządzenia" (Android — UiAutomator2 / iOS — XCUITest). Przycisk **„Generuj kod {nazwa frameworka}"** uruchamia zadanie, a wynik pobierasz jako „Pobierz .cy.ts" / „Pobierz .py" / „Pobierz .spec.ts" / „Pobierz .java" / „Pobierz .cs":



Jeden przycisk na przypadek: zaznacz, generuj, pobierz — kod w konwencji Twojego frameworka.



Wygenerowany plik `.cy.ts`: kroki przypadku przełożone na `cy.visit` / `cy.get` / asercje.

- **„Generator kodu testów bazodanowych"** — przypadki na testy **pytest** warstwy danych z izolacją rollback; „Pobierz .py".
- **„Generator kodu testów PHP"** — przełącznik PHPUnit/Pest, „Generuj kod", „Pobierz .php" (podpowiedź uruchomienia `./vendor/bin/phpunit` lub `./vendor/bin/pest`).

Każdy wygenerowany plik zostaje na serwerze — sekcja „Pobierz wygenerowany kod" przyjmuje „Identyfikator kodu" (UUID) do ponownego pobrania.

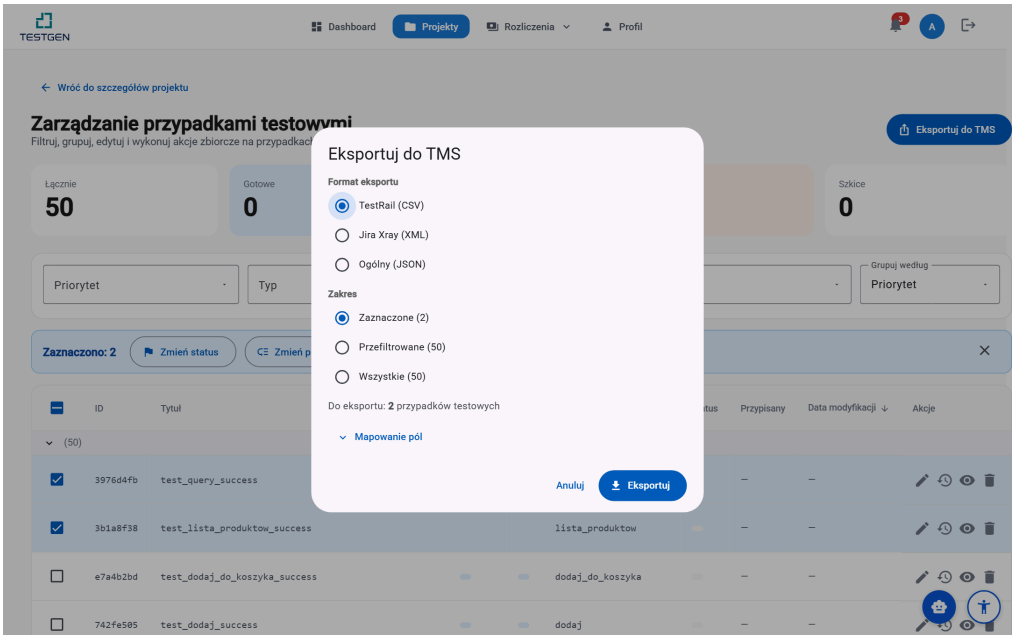
Frameworki i ich dostępność:

Framework	Język	Typ	Jak dostępny
pytest	Python	jednostkowy	framework analizy + generator bazodanowy
JUnit	Java	jednostkowy	framework analizy
NUnit	C#	jednostkowy	framework analizy
Mocha / Jest	JavaScript/TS	jednostkowy	framework analizy
unittest	Python	jednostkowy	framework analizy + generator (`python -m unittest`)
TestNG	Java	jednostkowy	framework analizy + generator (`@Test`)
MSTest	C#	jednostkowy	framework analizy + generator (`[TestClass]`)
Cypress	TypeScript	E2E web	framework analizy + generator
Selenium	Python	E2E web	framework analizy + generator
Playwright	TypeScript	E2E web	framework analizy + generator (`.spec.ts`)
Appium	Python	E2E mobilne	framework analizy + generator (+ platforma urządzenia)
SikuliX	Python (obrazy)	GUI desktop	generator z Gherkin (`.py` + PNG)
PHPUnit / Pest	PHP	jednostkowy	generator PHP

8.6. Eksport testów

Przycisk **„Eksportuj"** (pojedynczo lub masowo) otwiera dialog **„Eksportuj do TMS"**:

- **format**: „TestRail (CSV)", „Jira Xray (XML)", „Ogólny (JSON)",
- **zakres**: „Zaznaczone (n)" / „Przefiltrowane (n)" / „Wszystkie (n)",
- **„Mapowanie pól"** — tabela „Pole TMS → Pole TestGEN" z „Przywróć domyślne"; mapowanie zapamiętywane lokalnie.



Mapowanie pól dopasowuje nazwy kolumn TestGEN do schematu Twojego narzędzia TMS.

Plik (`testgen_{projekt}_{data}_{format}`) importujesz w narzędziu TMS. Bezpośrednie połączenia z trackerami (Jira, Azure DevOps) opisuje Część 3, rozdział 10.

8.7. Dobre praktyki

- **Przejrzyj „Szkice" po każdej analizie** — zatwierdź sensowne, wyłącz śmieciowe.
- **Priorytet to wskazówka** — krytyczne funkcje biznesowe zasługują na „Krytyczny" nawet, gdy generator dał „Wysoki".
- **Kroki językiem zespołu** — doprecyzuj zbyt techniczne sformułowania.
- **Nie usuwaj przypadków negatywnych** — łapią regresje obsługi błędów.
- **Jeden framework na projekt** — spójny styl kodu obniża koszty utrzymania.
- **Kod SikuliX wymaga obrazów** — przygotuj zrzuty dobrej jakości (6.7).

8.8. Ograniczenia

- Przypadki powstają **z przesłanych materiałów** — czego nie wgrasz, tego nie przetestujesz.
- **Regresja to zestaw, nie typ** — wybierany z porównania wersji (4.7).
- **Podglądy kodu to zwykły tekst** — podświetlanie i diff zobaczysz w raportach i rekomendacjach bezpieczeństwa.
- **Brak edycji kodu w miejscu** — poprawki wprowadzasz w repozytorium.
- **Generatory specjalistyczne jednorazowe** — po zmianie przypadku wygeneruj kod ponownie.

Rozdział 9. Raporty i eksport

Raporty domykają obieg: to, co wykryła platforma, trafia do zarządu, zespołu i narzędzi zewnętrznych — w formacie, którego każdy oczekuje.

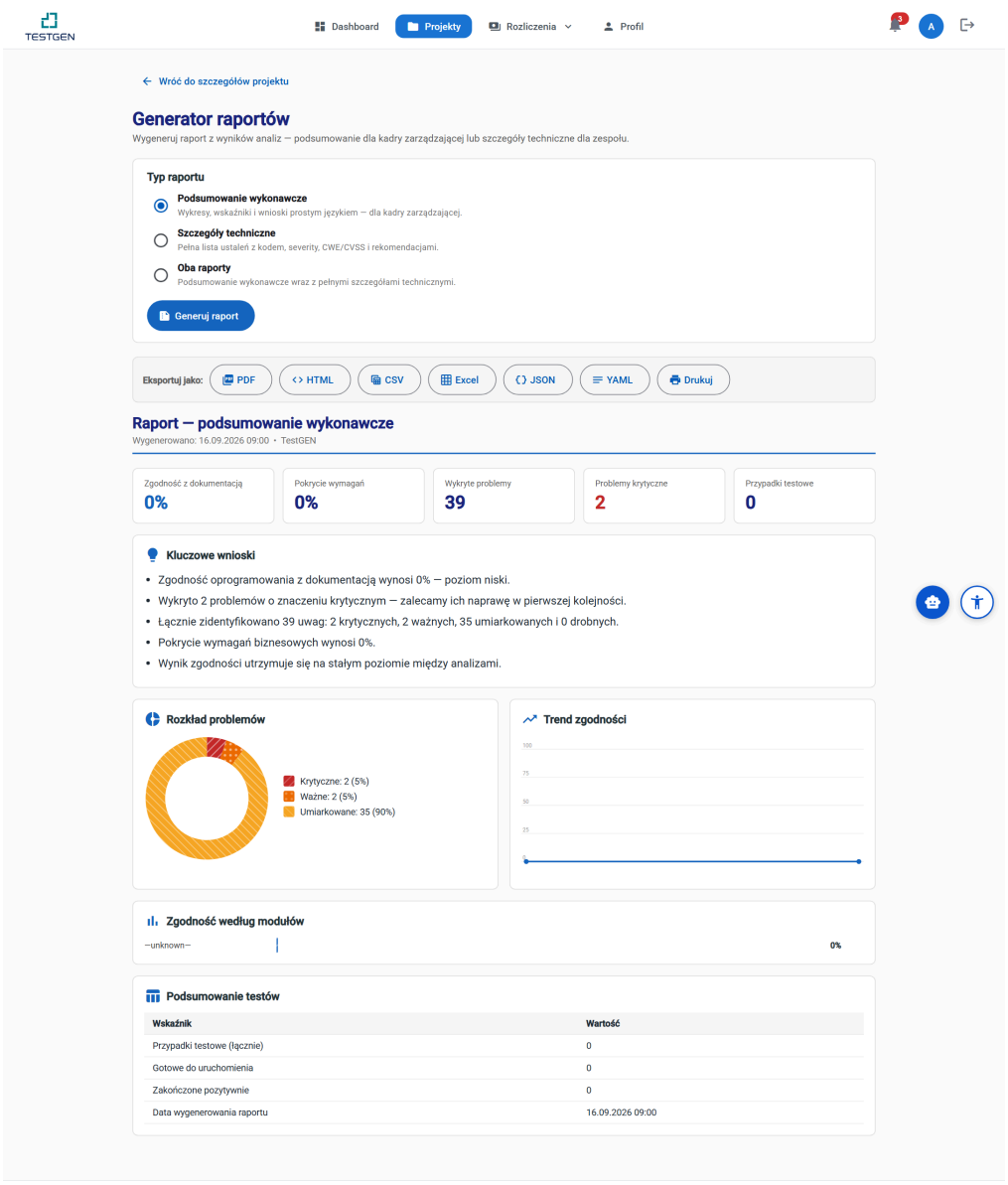
9.1. Poziomy raportów

Karta **„Raporty"** („Generator raportów") oferuje trzy warianty **„Typu raportu"**:

- **„Podsumowanie wykonawcze"** — „Wykresy, wskaźniki i wnioski prostym językiem — dla kadry zarządzającej": karty wskaźników (zgodność z dokumentacją, pokrycie wymagań, wykryte i krytyczne problemy, przypadki testowe), „Kluczowe wnioski" i wykresy (rozkład problemów, trend zgodności, zgodność według modułów);
- **„Szczegóły techniczne"** — „Pełna lista ustaleń z kodem, severity, CWE/CVSS i rekomendacjami" — dla zespołu: każde ustalenie z wagą, źródłem, lokalizacją, podświetlonym fragmentem kodu i rekomendacją;
- **„Oba raporty"** — raport pełny.

9.2. Generowanie i podgląd

1. Wybierz typ raportu i kliknij **„Generuj raport"**.
2. Postęp widać na czterech etapach: **Zbieranie danych** → **Analiza trendów** → **Generowanie treści** → **Renderowanie**.
3. Podgląd pokaże dokładnie to, co trafi do pliku.



Fundusze Europejskie dla Polski Wschodniej
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Podgląd na ekranie = zawartość pliku; typ raportu to jedyny wybór — reszta wynika z wyników projektu.

9.3. Formaty plików

Pasek **„Eksportuj jako:"** oferuje sześć formatów (plus „Drukuj"):

Format	Kiedy
PDF	wysyłka, archiwizacja, druk — stały układ
HTML	podgląd w przeglądarce, intranet
CSV / **Excel**	dalsza analiza w arkuszu
JSON / **YAML**	wrzutka do narzędzi i procesów automatycznych

⚠️ **Uwaga:** dłuższe eksporty pokazują dialog postępu (anulowalny) — plik pobiera się sam po zakończeniu i **pozostaje na serwerze 7 dni**. Wykresy renderują się niezależnie od przeglądarki — plik wygląda tak samo wszędzie.

9.4. Integracja z systemami zewnętrznymi

Trzy drogi wyników poza platformę:

- **Eksport przypadków do TMS** — pliki TestRail/Xray/JSON z mapowaniem pól (8.6).
- **Bezpośrednia synchronizacja z trackerami** — Jira i Azure DevOps: konfiguracja, „Test połączenia", synchronizacja luk i rekomendacji (Część 3, rozdział 10).
- **API platformy** — automatyzacja całego obiegu (uruchomienie analizy, pobranie wyników, eksport raportu) z przykładami curl i Python (Część 3, rozdział 11).

9.5. Dobre praktyki

- **Na zarząd — „Podsumowanie wykonawcze"**; szczegóły zostaw zespołowi.
- **Eksportuj po przeglądzie** — statusy przypadków i znalezisk w raporcie odzwierciedlają stan bieżący: najpierw selekcja, potem raport.
- **PDF do archiwum, JSON do automatyzacji** — raport w JSON porównasz między wydaniami.

9.6. Ograniczenia

- **Jedyną konfiguracją raportu jest jego typ** — własnych szablonów, sekcji ani brandingu nie ma; treść wynika z danych i filtrów projektu.
- **Raport dotyczy stanu bieżącego** — historię pokazuje tylko trend zgodności (5.6).
- **Pliki eksportowe żyją 7 dni** na serwerze.

Utrzymanie dokumentu: zmiany UI = aktualizacja zrzutów i opisów; zrzuty pochodzą z rzeczywistej aplikacji (stan na 2026-09-20). Spójność terminów: Glosariusz. Kontynuacja: Część 3 — Zaawansowane funkcje.

Podręcznik użytkownika — Część 3: Zaawansowane funkcje

> **Wersja:** 2.1 (2026-09-28) · **Odbiorcy:** użytkownicy platformy TestGEN korzystający z integracji, API i personalizacji · **Część 3 z 4** (całość: 01 — Wprowadzenie, 02 — Praca z systemem, 04 — Administracja i rozwiązywanie problemów, Spis treści)
>
> **Konwencje:** forma bezpośrednia („Ty"), nazwy przycisków i ekranów w cudzysłowie dokładnie tak, jak widzisz je w aplikacji. Ramki: 💡 **Wskazówka** (oszczędza czas) i ⚠️ **Uwaga** (częsta pułapka). Definicje pojęć: Glosariusz.

Spis treści części

- **Rozdział 10. Integracje** — 10.1. Co z czym łączy TestGEN · 10.2. Jira · 10.3. Azure DevOps · 10.4. Synchronizacja wyników · 10.5. GitHub i GitLab · 10.6. Status checki w pull requestach · 10.7. Konfiguracja CI/CD · 10.8. TestRail, Xray i Zephyr · 10.9. Synchronizacja dwukierunkowa z TMS · 10.10. Import wymagań z Jiry · 10.11. Ograniczenia
- **Rozdział 11. Webhooks, API i CLI** — 11.1. Zasady dostępu · 11.2. Uwierzytelnianie · 11.3. Limity żądań · 11.4. Format błędów · 11.5. Najczęściej używane endpointy · 11.6. Przykłady (curl i Python) · 11.7. Praca z CLI · 11.8. Webhooki wychodzące · 11.9. Ograniczenia
- **Rozdział 12. Dostępność i personalizacja** — 12.1. Dostępność (WCAG 2.1 AA) · 12.2. Tryb wysokiego kontrastu i animacje · 12.3. Skróty klawiszowe · 12.4. Wygląd na różnych ekranach · 12.5. Personalizacja interfejsu · 12.6. Ograniczenia

---

Rozdział 10. Integracje

TestGEN nie zastępuje Twoich narzędzi — wpisuje się w nie. Ten rozdział opisuje, z czym platforma łączy się bezpośrednio i gdzie znajdziesz ekran każdej integracji.

10.1. Co z czym łączy TestGEN

Cel	Sposób	Gdzie
Wysyłanie luk i rekomendacji do trackerów	**Jira** i **Azure DevOps**	karta projektu → „Integracje" (10.2–10.4)
Analiza po każdym pushu i pull requeście	**GitHub** i **GitLab** (OAuth + webhooki)	„Ustawienia" → „Integracja GitHub" / „Integracja GitLab" (10.5)

Cel	Sposób	Gdzie
Wynik analizy przy pull requeście	**status check** i komentarz w PR/MR	automatycznie po podłączeniu repozytorium (10.6)
Bramka jakości w pipeline'ie	**generator konfiguracji CI/CD**	karta projektu → „Konfiguracja CI/CD" (10.7)
Przeniesienie przypadków testowych do TMS	**TestRail**, **Jira Xray**, **Zephyr Scale** — bezpośrednio albo plikiem	„Integracje" → „Systemy zarządzania testami" (10.8)
Wspólne przypadki w TestGEN i TMS	**synchronizacja dwukierunkowa**	ekran konfiguracji TMS → „Synchronizacja dwukierunkowa" (10.9)
Wymagania prosto z Jiry	**import historyjek, epików i błędów**	karta projektu → „Import z Jiry" (10.10)
Powiadomienia Twoich systemów	**webhooki wychodzące**	„Ustawienia" → „Webhooki" (11.8)
Automatyzacja z kodu i terminala	**API REST**, **klucze API**, **CLI**	rozdz. 11

Kafelek **„Integracje"** na karcie projektu otwiera ekran z zakładkami **„Jira"** i **„Azure DevOps"** oraz sekcją **„Systemy zarządzania testami"** z odnośnikami do TestRaila, Jira Xray / Zephyr Scale i importu wymagań.

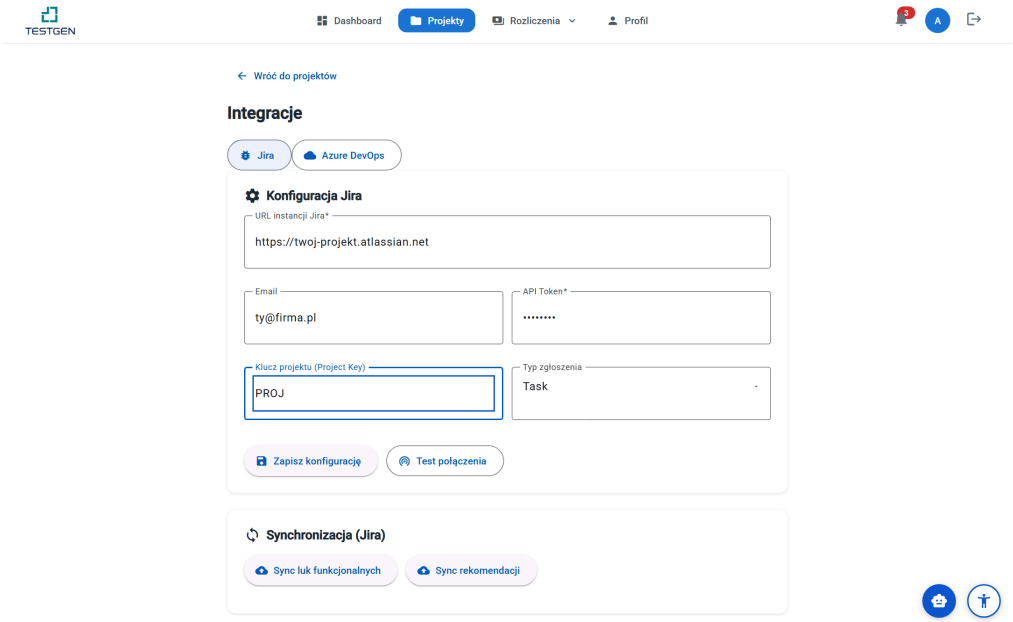
10.2. Jira

W zakładce „Jira" wypełnij „Konfigurację Jira":

Pole	Przykład	Skąd wziąć
„URL instancji Jira"	`https://twoj-projekt.atlassian.net`	adres Twojej organizacji Jira Cloud
„Email"	`ty@firma.pl`	konto z uprawnieniem tworzenia zgłoszeń
„API Token"	`ATATT3x...`	id.atlassian.com → Security → API tokens
„Klucz projektu (Project Key)"	`PROJ`	kod docelowego projektu Jira
„Typ zgłoszenia"	Task / Bug / Story / Sub-task	typ tworzonego issue

Przyciskiem **„Test połączenia"** sprawdzisz dane (platforma odpyta Jira o Twoje konto), a **„Zapisz konfigurację"** zapisuje je **zaszyfrowane** (poświadczenia szyfrowane w bazie — nie trzymane jawnym tekstem).

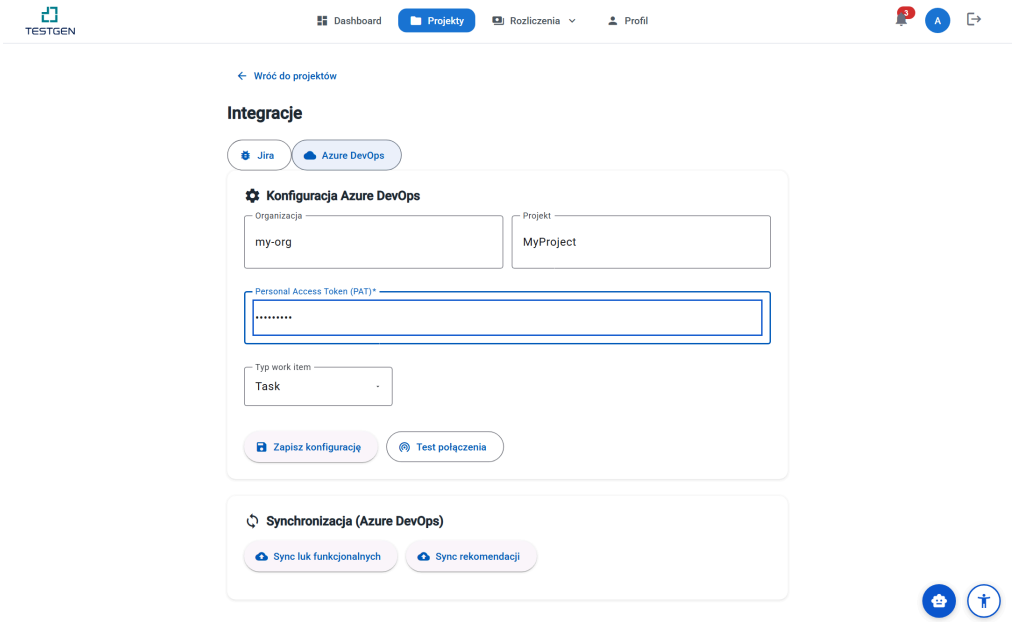
To samo połączenie wykorzystuje import wymagań (10.10). Eksport przypadków do wtyczek Xray i Zephyr ma **osobną** konfigurację (10.8) — wtyczki testowe wymagają innych uprawnień niż tworzenie zgłoszeń.



Test połączenia przed zapisem — błędny token wywali się tu, a nie przy pierwszej synchronizacji.

10.3. Azure DevOps

Zakładka „Azure DevOps" ma analogiczną formę: **„Organizacja"**, **„Projekt"**, **„Personal Access Token (PAT)"** (zakres *Work Items: Read & Write*) oraz **„Typ work item"** (Task / Bug / User Story / Issue). Adres organizacji składany jest automatycznie (`https://dev.azure.com/{organizacja}`), a „Test połączenia" odpytuje o listę projektów.



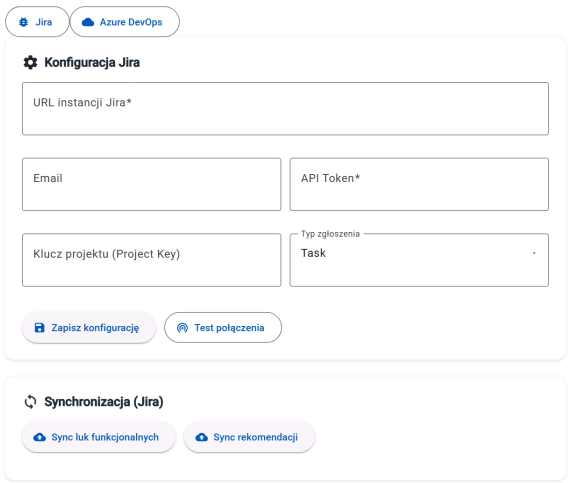
PAT generujesz w ustawieniach użytkownika Azure DevOps — wystarczy zakres do elementów roboczych.

10.4. Synchronizacja wyników

Sekcja „Synchronizacja" ma dwa przyciski:

- **„Sync luk funkcjonalnych"** — wysyła wykryte luki (Część 2, rozdz. 5.6) jako zgłoszenia w trackerze,
- **„Sync rekomendacji"** — wysyła rekomendacje napraw.

Synchronizacja jest **całościowa** (wszystkie luki albo wszystkie rekomendacje projektu) i przebiega w tle; wynik podsumowują chipy „Wysłano: {n} / {m}" i „Błędy: N". Sekcja **„Historia synchronizacji"** przechowuje przebiegi ze statusem (poprawny / częściowy / błąd) i licznikiem wysłanych pozycji.



Fundusze Europejskie dla Polski Wschodniej
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Historia pokazuje, ile pozycji realnie dotarło do trackera — przy „Błędach" sprawdź ważność tokenu.

Zgłoszenia dostają etykiety `testgen-gap` (luki) lub `testgen-rec` (rekomendacje) oraz priorytet przetłumaczony z wagi (P1 → Highest, P2 → High, P3 → Medium, P4 → Low).

💡 **Wskazówka:** synchronizuj po przeglądzie wyników — najpierw odrzuć fałszywe alarmy w TestGEN, a dopiero potem wyślij to, co naprawdę wymaga pracy zespołu. Limit bezpieczeństwa: 10 synchronizacji na godzinę na projekt.

⚠️ **Uwaga:** ponowna synchronizacja **tworzy nowe zgłoszenia** także dla już wysłanych pozycji — nie klikaj jej wielokrotnie „na wszelki wypadek". (Dotyczy luk i rekomendacji; eksport przypadków testowych do TMS deduplikuje — patrz 10.8.)

10.5. GitHub i GitLab

Po podłączeniu repozytorium TestGEN **sam** analizuje kod przy każdym pushu na gałąź główną i przy każdym pull requeście (w GitLabie: merge requeście). Nie musisz niczego wgrywać ręcznie.

Podłączenie GitHub:

1. Otwórz „Ustawienia" → **„Integracja GitHub"**.
2. Kliknij **„Połącz z GitHub"** i zatwierdź dostęp na stronie GitHuba.
3. W oknie **„Wybierz repozytorium"** wskaż repozytorium — TestGEN założy w nim webhook.

Podłączenie GitLab („Ustawienia" → **„Integracja GitLab"**) działa tak samo, z jedną różnicą: w polu **„Adres GitLaba"** podajesz adres instancji — `https://gitlab.com` albo firmowy, np. `https://git.twojafirma.pl`. Instancje self-hosted są obsługiwane.

Lista **„Podłączone repozytoria"** (w GitLabie: **„Podłączone projekty"**) pokazuje gałąź główną i stan webhooka, a sekcja **„Ostatnie zdarzenia"** — każdy push i pull request, który dotarł do TestGEN, wraz ze statusem analizy.

⚠️ **Uwaga:** komunikat „Integracja z GitHub nie jest skonfigurowana na tym serwerze" oznacza, że administrator platformy nie zarejestrował aplikacji OAuth — to nie jest błąd Twojego konta (Część 4).

Odłączenie repozytorium usuwa webhook i zatrzymuje nowe analizy; dotychczasowe wyniki zostają.

💡 **Wskazówka:** jednorazowy import kodu bez ciągłej integracji nadal zrobisz przez zakładkę „Połącz repozytorium" w projekcie (Część 2, rozdz. 4.1) — działa też z Bitbucketem.

10.6. Status checki w pull requestach

Każda analiza wyzwolona przez pull request kończy się **status checkiem** o nazwie `TestGEN / Code Analysis` i — domyślnie — komentarzem z podsumowaniem znalezisk. Status check jest:

- **zielony** — brak problemów na progu blokującym albo poniżej niego,
- **czerwony** — analiza znalazła problem co najmniej tak poważny jak próg,
- **w toku** — dopóki analiza trwa.

Zachowanie ustawisz osobno dla każdego repozytorium: na liście podłączonych repozytoriów kliknij ikonę **ustawień status checka**.

Ustawienie	Domyślnie	Znaczenie
„Próg blokowania"	Krytyczny	Krytyczny · Wysoki (krytyczne i wysokie) · Średni (krytyczne, wysokie i średnie)
„Blokuj merge po przekroczeniu progu"	włączone	gdy wyłączone, status zostaje zielony, a liczby i tak są widoczne
„Dodawaj komentarz na PR/MR"	włączone	podsumowanie znalezisk i link do raportu

Historię zgłoszonych wyników otworzysz ikoną **historii** przy repozytorium: ekran **„Historia status checków"** pokazuje zdarzenie, commit, status analizy, wysłany status check i czas analizy.

💡 **Wskazówka:** w regułach ochrony gałęzi na GitHubie/GitLabie oznacz `TestGEN / Code Analysis` jako wymagany — wtedy pull request z problemem powyżej progu nie da się scalić. Nazwa jest stała; nie zmieni się między wersjami.

10.7. Konfiguracja CI/CD

Kafelek **„Konfiguracja CI/CD"** na karcie projektu generuje gotowy plik pipeline'u, który uruchamia analizę TestGEN przy każdym buildzie i **przerywa build**, gdy analiza znajdzie problem powyżej wybranego progu.

Pole	Wartości
„Platforma"	Jenkins, GitHub Actions, Azure Pipelines, GitLab CI
„Próg blokowania builda"	Krytyczny / Wysoki / Średni
„Moduły analizy"	Analiza kodu, Analiza dokumentacji, Skan bezpieczeństwa, Analiza UML
„Tryb silnika"	Natywny (deterministyczny), LLM (semantyczny), Hybrydowy
„Limit czasu (minuty)"	od 1 do 180, domyślnie 30

Sekcja **„Podgląd"** pokazuje plik i nazwę, pod którą zapisać go w repozytorium (np. `.github/workflows/testgen.yml`, `Jenkinsfile`, `azure-pipelines.yml`, `.gitlab-ci.yml`). Przyciski **„Kopiuj"** i **„Pobierz plik"** — do wyboru.

Pipeline uwierzytelnia się **kluczem API** (11.2) zapisanym jako sekret o nazwie `TESTGEN_API_KEY` — podgląd podpowiada, gdzie go dodać na wybranej platformie.

Werdykt „przerwać build czy nie" liczy serwer TestGEN, nie plik pipeline'u. Dzięki temu build i status check w pull requeście zawsze mówią to samo — zmiana progu w pliku nie rozjedzie ich ze sobą.

10.8. TestRail, Xray i Zephyr

Przypadki testowe wysyłasz do TMS **bezpośrednio** z listy testów: przyciski **„Eksportuj do TestRail"**, **„Eksportuj do Xray"** i **„Eksportuj do Zephyr"**. Najpierw jednak skonfiguruj połączenie: „Integracje" → sekcja „Systemy zarządzania testami".

TestRail (**„Integracja z TestRail"**):

Pole	Skąd wziąć
„Adres instancji TestRail"	adres Twojej instancji (wymagany HTTPS)
„E-mail konta TestRail"	konto z prawem tworzenia przypadków

Pole	Skąd wziąć
„Klucz API"	TestRail: My Settings → API Keys
„Identyfikator projektu TestRail"	liczba z adresu projektu w TestRailu
„Suite (opcjonalnie)"	tylko dla projektów w trybie multi-suite

Jira Xray i Zephyr Scale (**„Integracja z Jira"**): wybierz **„Wtyczkę"**, a następnie podaj „Adres instancji Jira", „E-mail konta Atlassian", „Token API" i „Klucz projektu Jira". Opcjonalne **„Powiązane zgłoszenie"** to historyjka, z którą związesz eksportowane testy. Projekt korzysta z jednej wtyczki naraz.

Przed zapisem sprawdź dane przyciskiem **„Testuj połączenie"**, potem **„Zapisz konfigurację"**. Sekcja **„Mapowanie pól"** pozwala dopasować priorytety i typy testów do tego, jak skonfigurowana jest Twoja instancja.

Deduplikacja. Ponowny eksport **aktualizuje** przypadki wysłane wcześniej, zamiast tworzyć duplikaty. TestRail i Zephyr rozpoznają je po polu własnym `testgen_id`, Xray — po etykiecie.

⚠️ **Uwaga:** w TestRailu i Zephyrze pole własne `testgen_id` (typ tekstowy) musisz założyć sam, zanim wyeksportujesz pierwszy przypadek. Bez niego każdy eksport utworzy nowe przypadki.

Eksport idzie w tle — TMS-y ograniczają tempo żądań, więc setka przypadków to kilka minut. Dialog eksportu pokazuje identyfikator zadania, którym śledzisz postęp.

💡 **Wskazówka:** eksport **plikiem** (TestRail CSV, Jira Xray XML, JSON — dialog „Eksportuj do TMS", Część 2, rozdz. 8.6) nadal działa i przydaje się tam, gdzie TestGEN nie ma sieciowego dostępu do Twojego TMS-a.

10.9. Synchronizacja dwukierunkowa z TMS

Eksport działa w jedną stronę. Synchronizacja dokłada kierunek powrotny: zmiany wprowadzone w TestRailu, Xrayu lub Zephyrze wracają do TestGEN, a zmiany z TestGEN trafiają do TMS-a — samoczynnie, w stałym rytmie.

Otwórz ekran konfiguracji TMS i kliknij **„Synchronizacja dwukierunkowa"** (odnośnik pojawia się po zapisaniu połączenia). W sekcji **„Ustawienia"** wybierz:

Pole	Opcje
„Kierunek"	TestGEN → TMS · TMS → TestGEN · W obie strony
„Jak często"	od co 5 minut do co 24 godziny
„Gdy zmiana nastąpi po obu stronach"	Wygrywa nowsza zmiana · Rozstrzygam ręcznie

„Zapisz ustawienia" zapisuje konfigurację, ale **nie uruchamia** synchronizacji — do tego służy przycisk **„Włącz"**. Przycisk **„Synchronizuj teraz"** uruchamia przebieg bez czekania na odstęp.

Z TMS-a wracają tytuł, opis, kroki, oczekiwany wynik i priorytet. Kod testu zostaje nietknięty. Puste pole po stronie TMS-a **nie kasuje** wartości w TestGEN.

Konflikty. Gdy ten sam przypadek zmienił się po obu stronach od ostatniej synchronizacji, sekcja **„Konflikty"** pokazuje obie wersje. Przy „Wygrywa nowsza zmiana" rozstrzygnięcie jest automatyczne, ale konflikt i tak zostaje zapisany — przegrana wersja jest jedynym śladem po nadpisanych danych. Przy „Rozstrzygam ręcznie" wybierasz **„Zostaw wersję TestGEN"** albo **„Przyjmij wersję TMS"**.

Stan. Karta stanu pokazuje, czy synchronizacja jest **„Włączona"**, datę ostatniego przebiegu, liczbę zsynchronizowanych przypadków i termin następnego. Po trzech nieudanych przebiegach z rzędu stan zmienia się na **„Wymaga uwagi"**, a właściciel firmy dostaje e-mail. Synchronizacja wraca do działania sama, gdy połączenie z TMS-em zostanie przywrócone.

Na liście przypadków testowych każdy zsynchronizowany przypadek ma plakietkę z identyfikatorem po stronie TMS-a (np. `#1234`); datę ostatniej synchronizacji zobaczysz po najechaniu na plakietkę.

10.10. Import wymagań z Jiry

Zamiast pobierać dokumentację z Jiry i wgrywać ją ręcznie, zaimportuj historyjki, epiki, błędy i podzadania wprost jako wymagania. Kafelek **„Import z Jiry"** na karcie projektu otwiera ekran importu.

Import korzysta z połączenia Jira z zakładki „Jira" (10.2) — skonfiguruj je najpierw.

1. Wybierz **„Projekt Jiry"** — lista pochodzi z Jiry i zawiera projekty, które widzi konto z konfiguracji.
2. Zaznacz **„Typy zgłoszeń"**: Historyjki, Epiki, Błędy, Podzadania.
3. Wybierz **„Zakres"**: **„Tylko zmiany"** (domyślnie — zgłoszenia zmienione od ostatniego importu), **„Wszystko"** albo **„Od daty"**.
4. Kliknij **„Importuj"**.

Z Jiry	W TestGEN
historyjka, epik, podzadanie	wymaganie funkcjonalne
błąd	wymaganie **negatywne** (czego system robić nie powinien)
epik	moduł grupujący swoje historyjki
sekcja „Acceptance Criteria" w opisie	lista kryteriów akceptacji
załączniki	pliki projektu, analizowane jak każdy wgrany dokument

Wymagania trafiają do dokumentu `jira-<KLUCZ>.md` na liście dokumentów projektu i od razu uruchamia się dla nich analiza luk (Część 2, rozdz. 5). **„Historia importów"** podsumowuje każdy przebieg: „Zaimportowano 12 wymagań i 3 załączniki".

Ponowny import **aktualizuje** wymagania zamiast je dublować i nie cofa Twoich decyzji z przeglądu (akceptacji albo odrzucenia). Ten sam załącznik dołączony do kilku historyjek zapisuje się raz.

💡 **Wskazówka:** kryteria akceptacji zostaną rozpoznane, gdy w opisie zgłoszenia są pod nagłówkiem `h2. Acceptance Criteria` (albo `h3. Kryteria akceptacji`) jako lista punktowana.

10.11. Ograniczenia

- **Trackery zgłoszeń to Jira Cloud i Azure DevOps** — GitHub Issues i GitLab Issues nie są obsługiwane (integracja GitHub/GitLab dotyczy repozytoriów i pull requestów, nie zgłoszeń).
- **Synchronizacja luk i rekomendacji jest jednokierunkowa i całościowa** — bez wyboru pojedynczych pozycji i bez deduplikacji (etykiety `testgen-*` pomogą je odnaleźć). Nie dotyczy to przypadków testowych, które synchronizują się dwukierunkowo (10.9).
- **Synchronizacja z TMS nie zakłada nowych przypadków w TestGEN** — przypadek utworzony tylko w TMS-ie zostaje pominięty; najpierw musi pochodzić z TestGEN.
- **Usunięcie przypadku w TestGEN nie usuwa go w TMS-ie.**
- **Import z Jiry: do 200 załączników na przebieg, do 25 MB każdy.** Kryteria akceptacji rozpoznajemy tylko z opisu, nie z pól własnych.
- **Trackery i TMS konfigurujesz osobno dla każdego projektu** — każdy projekt łączy się z własną Jirą, Azure DevOps i TMS-em (zaleta przy wielu klientach, uwaga przy kopiowaniu ustawień).
- **GitHub, GitLab, klucze API i webhooki wychodzące to ustawienia firmy** („Ustawienia"), nie projektu. Każde podłączone repozytorium zasila własny projekt, zakładany automatycznie przy pierwszym zdarzeniu.

Rozdział 11. Webhooks, API i CLI

Wszystko, co robisz w interfejsie, da się wykonać programistycznie — platforma wystawia spójne REST API. Ten rozdział to praktyczne minimum: jak się uwierzytelnić, na jakie limity uważać, które endpointy wykorzystasz najczęściej, jak działa narzędzie wiersza poleceń i jak TestGEN powiadamia Twoje systemy webhookami.

11.1. Zasady dostępu

- Wszystkie endpointy żyją pod prefiksem `/api/v1` — wersjonowanie w ścieżce; zmiany łamiące zgodność wejdą jako `/api/v2`.

- Odpowiedzi i żądania to **JSON**.
- Operacje długotrwałe (analiza, eksport raportu, eksport do TMS, import z Jiry) zwracają `202 Accepted` od razu z identyfikatorem zadania — wynik pobierasz osobno (status + pobranie).
- Interaktywna dokumentacja (`/docs`, `/openapi.json`) dostępna jest w środowiskach deweloperskich; w produkcji wyłączona. Pełna referencja wszystkich endpointów, z przykładami w curl, Pythonie i JavaScripcie, znajduje się w dokumentacji API.

11.2. Uwierzytelnianie

Do skryptów, pipeline'ów i integracji używaj **klucza API**. Sesja z logowania (e-mail i hasło) służy interfejsowi.

Utworzenie klucza (tylko **właściciel firmy**):

1. Otwórz „Ustawienia" → **„Klucze API"**.
2. Kliknij **„Utwórz klucz API"**, nadaj **„Nazwę klucza"** i wybierz **„Zakres uprawnień"**:
   - **„Tylko odczyt"** — pobieranie wyników i raportów,
   - **„Pełny dostęp"** — także uruchamianie analiz i zapis danych.
3. Opcjonalnie ustaw **„Datę wygaśnięcia"** (maksymalnie 2 lata; puste — bezterminowy).
4. **Skopiuj klucz od razu** — po zamknięciu okna zobaczysz już tylko jego pierwsze znaki.

Klucz ma postać `tgn_…` i przekazujesz go w nagłówku:

```
Authorization: Bearer tgn_…
```

Lista kluczy pokazuje zakres, datę wygaśnięcia i **ostatnie użycie** — po tym poznasz klucz, którego nic już nie używa. **„Odwołaj klucz"** wyłącza go natychmiast; pipeline'y, które go używają, zaczną dostawać `401`.

⚠️ **Uwaga:** klucz o zakresie „Tylko odczyt" dostanie `403` przy próbie uruchomienia analizy. Kluczem API **nie** zarządzisz też innymi kluczami ani webhookami — to wymaga zalogowania w interfejsie, żeby wyciek jednego klucza nie pozwolił wystawić sobie kolejnych.

Sesja (interfejs). API akceptuje też token JWT z logowania: nagłówek `Authorization: Bearer <token>` albo ciasteczko `access_token` (HttpOnly) — przy ciasteczku żądania zmieniające stan wymagają dodatkowo nagłówka `X-XSRF-TOKEN` z treścią ciasteczka `XSRF-TOKEN`. Logowanie: `POST /api/v1/auth/login` z `{email, password}` (przy wielu firmach — najpierw `POST /auth/select-tenant`; przy włączonej MFA — `POST /auth/mfa/verify`). Sesja wygasa, więc do automatyzacji się nie nadaje.

11.3. Limity żądań

Limity zależą od planu: **Freemium 100, Basic 100, Premium 1000 żądań na minutę** (okno pełnej minuty). Każda odpowiedź zwraca nagłówki `X-RateLimit-Limit`, `X-RateLimit-Remaining` i `X-RateLimit-Reset`; po wyczerpaniu API odpowiada **429** z `Retry-After` i komunikatem, po ilu sekundach spróbować ponownie. Interfejs, skrypty i klucze API **współdzielą** limit firmy.

11.4. Format błędów

Błędy zwracane są w ustandaryzowanym formacie **problem+json** (RFC 7807): `type` (identyfikator, np. `…/errors/validation-error`), `title`, `status` i `detail` po polsku. Najczęstsze kody:

Kod	Kiedy
400/422	nieprawidłowe dane żądania (walidacja pól)
401	brak/nieważna sesja albo odwołany lub wygasły klucz API
402	funkcja wymaga wyższego planu (z linkiem do zmiany planu)
403	brak uprawnień do zasobu albo klucz „Tylko odczyt" przy operacji zapisu
404	nieistniejący zasób (lub projektu innej firmy)
429	przekroczony limit żądań (albo limit planu — np. analiza)

11.5. Najczęściej używane endpointy

Co	Metoda i ścieżka	Uwagi
Kim jestem	`GET /users/me`	szybki test klucza API
Lista projektów	`GET /projects`	paginacja + nagłówek `X-Total-Count`
Utworzenie projektu	`POST /projects`	`{name, description?}`; 201 + `Location`
Wgranie kodu	`POST /projects/{id}/upload`	multipart, także archiwum ZIP
Uruchomienie analizy	`POST /projects/{id}/analyze`	202; `{modules, test_framework…}`
Stan zadania	`GET /tasks/{task_id}`	postęp w procentach i bieżący krok
Werdykt bramki CI	`GET /projects/{id}/ci-cd/summary?threshold=`	`should_fail` — przerwać build czy nie
Lista przypadków	`GET /test-cases?project_id=`	filtry: typ, status, priorytet, framework
Eksport raportu	`POST /projects/{id}/reports/export`	202 → `GET /exports/{id}/status` → `/download`

Co	Metoda i ścieżka	Uwagi
Eksport do TMS	`POST /projects/{id}/export/testrail` (`xray`, `zephyr`)	202; pusta lista = wszystkie przypadki

11.6. Przykłady (curl i Python)

Test klucza i lista projektów (curl):

```
export TESTGEN_API_KEY=tgn_…

curl -s -H "Authorization: Bearer $TESTGEN_API_KEY" \
  https://app.testgen.pl/api/v1/users/me

curl -s -H "Authorization: Bearer $TESTGEN_API_KEY" \
  https://app.testgen.pl/api/v1/projects
```

Uruchomienie analizy i czekanie na wynik (Python + requests):

```
import os, time, requests

base = "https://app.testgen.pl/api/v1"
s = requests.Session()
s.headers["Authorization"] = f"Bearer {os.environ['TESTGEN_API_KEY']}"

r = s.post(f"{base}/projects/{PID}/analyze",
           json={"modules": ["code_analysis", "test_generation"]})
task_id = r.json()["task_id"]

while True:
    status = s.get(f"{base}/tasks/{task_id}").json()
    if status["status"] not in ("pending", "running"):
        break          # completed, failed, cancelled, timeout,
         partial_failure
    time.sleep(10)

verdict = s.get(f"{base}/projects/{PID}/ci-cd/summary",
                params={"threshold": "critical"}).json()
print(verdict["description"], "— przerwać build:",
         verdict["should_fail"])
```

Eksport raportu (curl, trzy kroki):

```
AUTH="Authorization: Bearer $TESTGEN_API_KEY"

EXP=$(curl -s -H "$AUTH" -H "Content-Type: application/json" \
  -d '{"format":"pdf","report_type":"executive","report":{}}' \
  https://app.testgen.pl/api/v1/projects/$PID/reports/export | jq -r
        .export_id)

curl -s -H "$AUTH" https://app.testgen.pl/api/v1/exports/$EXP/status
curl -s -H "$AUTH" -o raport.pdf \
  https://app.testgen.pl/api/v1/exports/$EXP/download
```

💡 **Wskazówka:** zamiast odpytywać status w pętli, możesz zasubskrybować zdarzenie „Analiza zakończona" webhookiem (11.8).

11.7. Praca z CLI

Narzędzie wiersza poleceń `testgen` wykonuje typowe operacje jednym poleceniem, bez pisania skryptów. Instalacja z repozytorium platformy:

```
pip install ./cli        # wymagany Python 3.11+
testgen login --api-key tgn_…
testgen whoami
```

W CI nie loguj się — ustaw zmienne `TESTGEN_API_KEY` i `TESTGEN_PROJECT_ID`, a CLI użyje ich samo.

Polecenie	Co robi
`testgen analyze . --wait`	pakuje katalog, wysyła kod, uruchamia analizę i czeka na werdykt
`testgen generate --framework pytest -o tests`	zapisuje wygenerowane testy na dysk
`testgen scan --type sast --severity-threshold high`	skan bezpieczeństwa z progiem
`testgen export --format pdf -o raport.pdf`	pobiera raport projektu
`testgen status <task_id> --watch`	śledzi zadanie co 5 s
`testgen projects`	lista projektów

Ustawienia projektu trzymasz w pliku `.testgen.yaml` w repozytorium (`project_id`, `engine_mode`, `default_framework`, `severity_threshold`, `api_url`) — CLI szuka go w bieżącym katalogu i wyżej, jak `git`. Argumenty polecenia mają pierwszeństwo przed zmiennymi środowiskowymi, a te — przed plikiem.

Kody wyjścia to kontrakt z pipeline'em:

Kod	Znaczenie
0	sukces, brak problemów powyżej progu

Kod	Znaczenie
1	znaleziska powyżej progu — build ma się zatrzymać
2	błąd narzędzia: brak sieci, zły klucz, zły argument

Format wyniku wybierasz opcją `--json` albo `--yaml` (domyślnie tabele); `--quiet` wyłącza wyjście i zostawia sam kod. Komunikaty postępu idą na stderr, więc `testgen analyze . --wait --json | jq` działa bez sztuczek.

Pełny opis: `cli/README.md` w repozytorium platformy.

11.8. Webhooki wychodzące

Webhooki powiadamiają Twoje systemy o zdarzeniach w TestGEN — nie musisz ich odpytywać. Konfiguruje je **właściciel firmy**: „Ustawienia" → **„Webhooki"** → **„Dodaj webhook"**.

Pole	Uwagi
„Adres endpointu"	wymagany HTTPS — payload niesie dane Twoich analiz
„Sekret"	minimum 16 znaków; nim podpisujemy każdy payload
„Opis (opcjonalny)"	żeby po miesiącu wiedzieć, co to za adres
„Subskrybowane zdarzenia"	jedno lub więcej z listy poniżej

Zdarzenie	Kiedy
„Analiza zakończona" (`analysis.completed`)	po każdym przebiegu analizy projektu
„Podatność krytyczna lub wysoka" (`security.finding`)	tylko te dwa poziomy — niższe zostają w raporcie
„Wykryta luka funkcjonalna" (`gap.detected`)	rozbieżność między dokumentacją a kodem
„Raport gotowy" (`export.ready`)	plik eksportu czeka do pobrania
„Zmiana planu" (`subscription.changed`)	upgrade lub downgrade subskrypcji

Każde żądanie to `POST` z ciałem JSON (zawsze `event_type` i `timestamp` plus pola zdarzenia) oraz nagłówkami:

- `X-TestGEN-Event` — typ zdarzenia,
- `X-TestGEN-Delivery` — identyfikator dostawy (do deduplikacji po Twojej stronie),
- `X-TestGEN-Signature` — `sha256=<HMAC-SHA256 ciała z Twoim sekretem>`.

⚠️ **Uwaga:** zawsze weryfikuj podpis, zanim uwierzysz w treść — adres webhooka nie jest tajemnicą, sekret jest.

Dostarczanie. Za sukces uważamy odpowiedź 2xx w ciągu 10 sekund. Nieudana dostawa jest ponawiana po 1 minucie, 5 minutach i 30 minutach — razem cztery próby. **„Historia dostaw"** pokazuje każdą próbę z kodem odpowiedzi i filtrem

„Dostarczone / Oczekujące / Nieudane". Ikoną **testu połączenia** wyślesz próbne zdarzenie, zanim podłączysz prawdziwy system.

Webhooki **przychodzące** dotyczą rozliczeń (tpay) oraz GitHuba i GitLaba (10.5) — konfiguruje je platforma, niczego tu nie ustawiasz.

11.9. Ograniczenia

- **Dokumentacja OpenAPI niewidoczna w produkcji** — korzystaj z dokumentacji API.
- **CLI nie jest opublikowane w PyPI** — instalujesz je z repozytorium platformy.
- **CLI loguje się hasłem tylko bez MFA** — konta z MFA używają klucza API.
- **Limity planu obowiązują także przez API i CLI** — wyczerpany limit analiz zwróci 429 niezależnie od tego, czy klikniesz przycisk, czy wyślesz żądanie.
- **Webhook jest wyłączany ręcznie, nie automatycznie** — przy stale nieudanych dostawach sprawdź „Historię dostaw" i wyłącz go przełącznikiem.

Rozdział 12. Dostępność i personalizacja

TestGEN ma być używalny dla każdego — także przy nawigacji klawiaturą, czytnikiem ekranu, na małym ekranie i w trudnych warunkach oświetleniowych. Ten rozdział zbiera wszystkie pokrętła, które masz do dyspozycji.

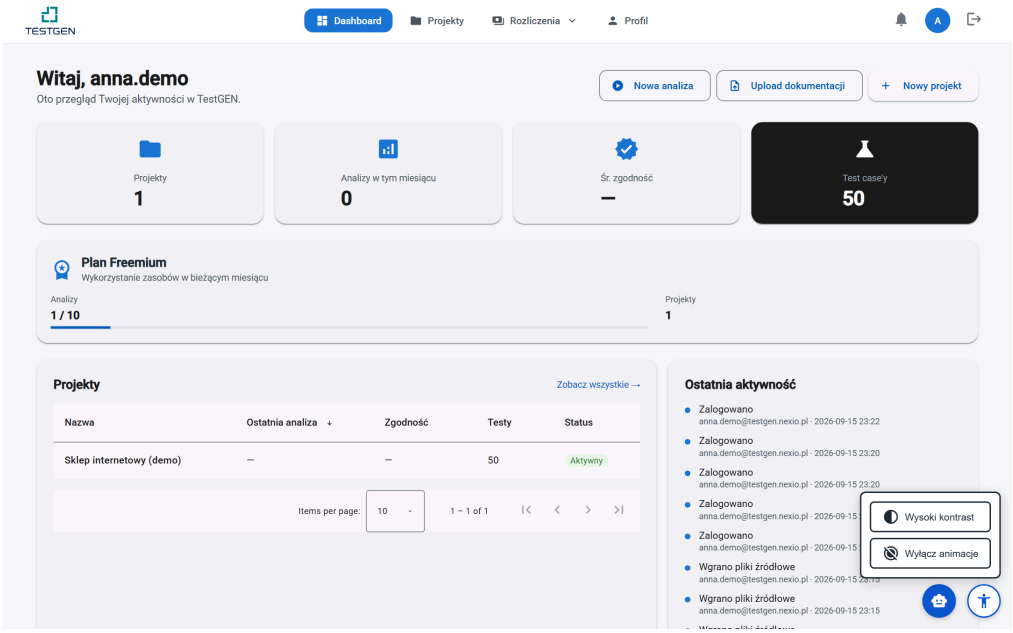
12.1. Dostępność (WCAG 2.1 AA)

Całość interfejsu obsłużysz **klawiaturą i czytnikiem ekranu** (standard WCAG 2.1 AA, testowane z NVDA). Elementy mechaniki:

- **Przycisk „Przejdź do treści głównej"** — pierwszy element strony, widoczny po naciśnięciu Tab; pomija menu i przenosi od razu do treści.
- **Okruszki i linki powrotu** („Wróć do…") — czytelne dla czytników, klikalne na każdym poziomie.
- **Etyykiety pól i komunikaty** — snackbar z potwierdzeniem operacji ogłasza się także czytnikowi ekranu.
- **Focus widoczny** — aktywny element ma wyraźną obwódkę; Escape zamyka okno dialogowe, a fokus wraca tam, skąd przyszedł.

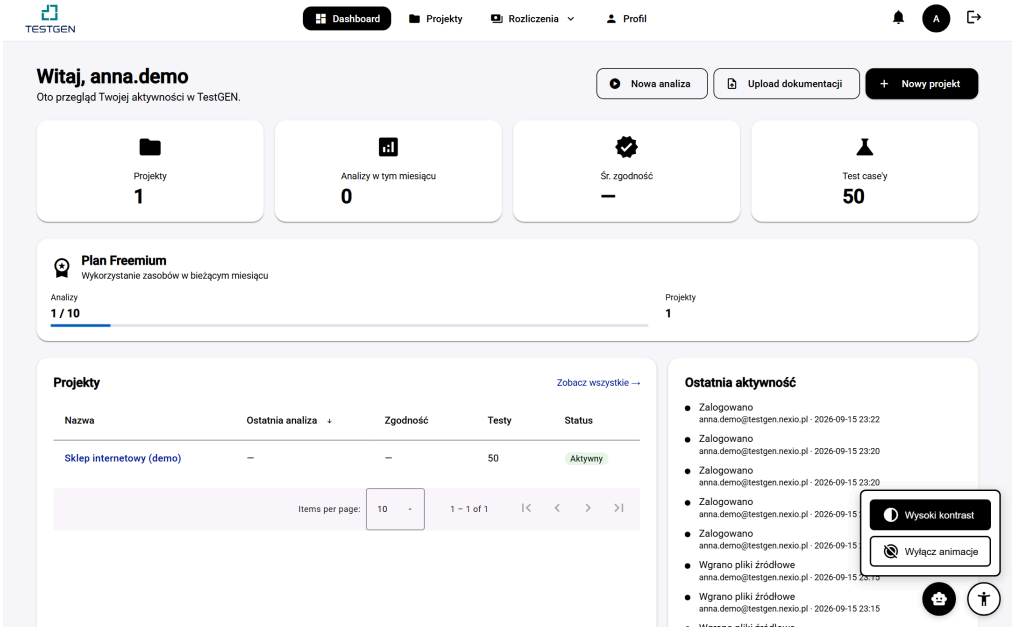
12.2. Tryb wysokiego kontrastu i animacje

Dwie funkcje włączysz bezpośrednio w aplikacji — przycisk paska dostępności znajduje się w menu bocznym:



Pasek dostępności: tryb wysokiego kontrastu i wyłączenie animacji.

- **„Wysoki kontrast"** — czarno-biały motyw o podwyższonym kontraście (przydatny przy słabym wzroku i jasnym świetle):



Tryb wysokiego kontrastu — wybór zapamiętywany jest w przeglądarce.

- **„Wyłącz animacje"** — wyłącza przejścia i ruch (pomocny przy zawrotach głowy i migrenach). Ustawienie systemowe „redukuj ruch" jest respektowane automatycznie.

Szczegóły praktyczne opisuje poradnik Dostępność.

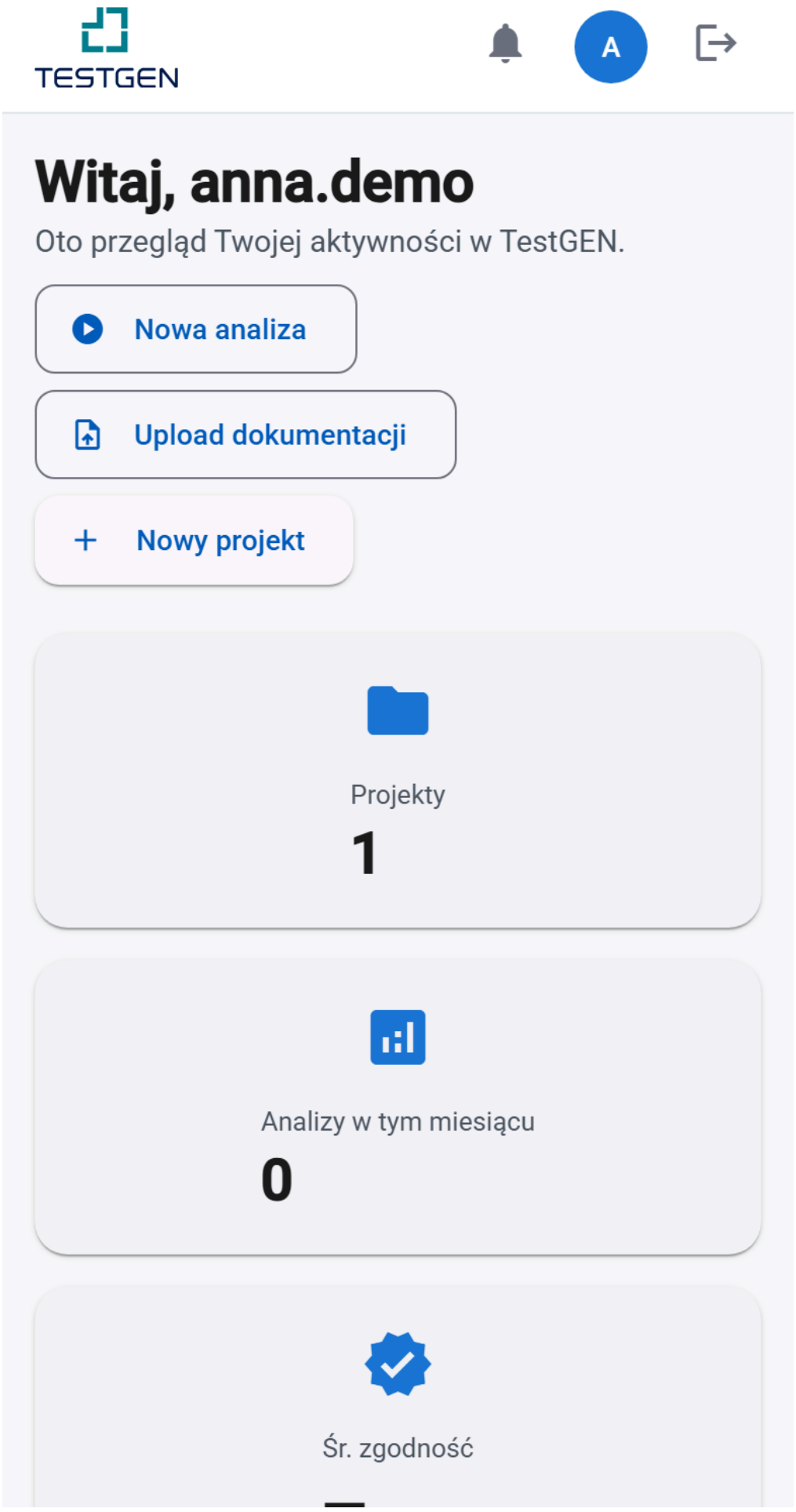
12.3. Skróty klawiszowe

TestGEN nie wprowadza własnych skrótów aplikacyjnych — świadomie korzysta z tych, które znasz z przeglądarki i czytników ekranu. Najczęściej używane:

Klawisz / kombinacja	Działanie
Tab / **Shift+Tab**	przejście do następnego / poprzedniego elementu
Enter / **spacja**	aktywacja przycisku lub linku
Escape	zamknięcie okna dialogowego (fokus wraca tam, skąd przyszedł)
strzałki	poruszanie się po listach i tabelach z rozwiniętym menu

12.4. Wygląd na różnych ekranach

Interfejs dopasowuje się do ekranu: na tablecie i telefonie układ składa się do jednej kolumny, menu boczne chowa się pod przyciskiem, tabele przewijają się w miejscu bez psucia strony. Przy powiększeniu do 200% i szerokości 320 pikseli treść pozostaje czytelna — nic się nie „gubi" poza ekranem.


TESTGEN
A
Witaj, anna.demo
Oto przegląd Twojej aktywności w TestGEN.
Nowa analiza
Upload dokumentacji
Nowy projekt
Projekty
1
Analizy w tym miesiącu
0
Śr. zgodność



Ten sam pulpit na telefonie: jedna kolumna, menu pod przyciskiem.

12.5. Personalizacja interfejsu

Poza funkcjami dostępności (12.2) interfejs jest wspólny dla wszystkich użytkowników: brak własnych motywów kolorystycznych, układów ani języka interfejsu do wyboru (aplikacja jest po polsku). Nazwę wyświetlaną zmienisz w profilu (Część 4, rozdz. 13); powiadomienia dostosujesz w preferencjach kanałów.

12.6. Ograniczenia

- **Brak własnych motywów** — tylko tryb wysokiego kontrastu i wyłączanie animacji.
- **Brak wyboru języka interfejsu** — aplikacja jest po polsku (materiały analizujesz w dowolnym języku).
- **Skróty ograniczone do przeglądarki** — własne kombinacje klawiszy nie są konfigurowalne.

Utrzymanie dokumentu: zmiany UI = aktualizacja zrzutów i opisów; zrzuty pochodzą z rzeczywistej aplikacji (stan na 2026-09-20). Spójność terminów: Glosariusz. Kontynuacja: Część 4 — Administracja i rozwiązywanie problemów.

Podręcznik użytkownika — Część 4: Administracja i rozwiązywanie problemów

> **Wersja:** 2.0 (2026-09-20) · **Odbiorcy:** użytkownicy platformy TestGEN (analitycy QA, deweloperzy, kierownicy projektów) · **Część 4 z 4** — obejmuje Część IV konspektu (Administracja i zarządzanie) oraz Część V (Rozwiązywanie problemów). Całość: 01 — Wprowadzenie, 02 — Praca z systemem, 03 — Zaawansowane funkcje, Spis treści
>
> **Konwencje:** forma bezpośrednia („Ty"), nazwy przycisków i ekranów w cudzysłowie dokładnie tak, jak widzisz je w aplikacji. Ramki: 💡 **Wskazówka** (oszczędza czas) i ⚠️ **Uwaga** (częsta pułapka). Definicje pojęć: Glosariusz, nawigacja: Indeks terminów.

Spis treści części

Część IV — Administracja i zarządzanie

- **Rozdział 13. Zarządzanie kontem i subskrypcją** — 13.1. Profil · 13.2. Użytkownicy firmy · 13.3. Plany i zmiana subskrypcji · 13.4. Płatności (tpay) · 13.5. Faktury · 13.6. Zużycie limitów i powiadomienia · 13.7. Usuwanie konta · 13.8. Ograniczenia
- **Rozdział 14. Bezpieczeństwo i prywatność** — 14.1. Hasło · 14.2. Weryfikacja dwuetapowa (MFA) · 14.3. Izolacja danych firmy · 14.4. RODO i prywatność danych · 14.5. Dostęp API i klucze · 14.6. Ograniczenia

Część V — Rozwiązywanie problemów

- **Rozdział 15. FAQ** — 15.1. Konto i dostęp · 15.2. Projekty · 15.3. Analiza kodu · 15.4. Dokumentacja, UML i GUI · 15.5. Generowanie testów · 15.6. Bezpieczeństwo · 15.7. Eksport i raporty · 15.8. Integracje · 15.9. Płatności i plany · 15.10. Błędy i problemy
- **Rozdział 16. Troubleshooting** — 16.1. Problemy z analizą · 16.2. Problemy z generowaniem testów · 16.3. Problemy z integracjami · 16.4. Problemy z wydajnością · 16.5. Problemy z dostępem i kontem
- **Rozdział 17. Wsparcie techniczne** — 17.1. Jak uzyskać pomoc · 17.2. Kanały wsparcia · 17.3. Zgłaszanie błędów · 17.4. Propozycje nowych funkcji

---

Część IV — Administracja i zarządzanie

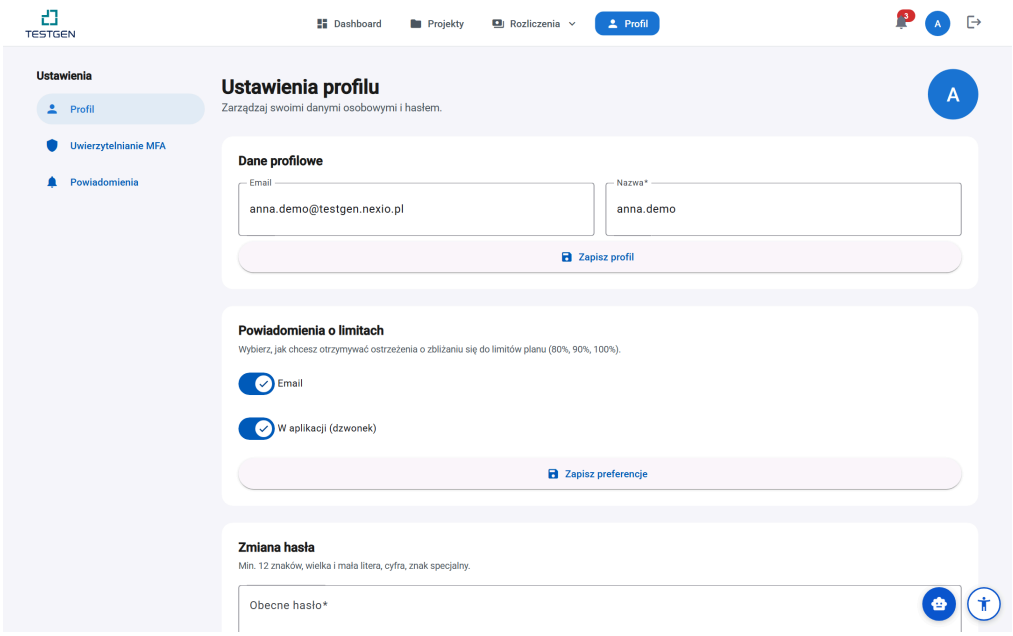
Rozdział 13. Zarządzanie kontem i subskrypcją

Menu **„Profil"** prowadzi do ustawień konta, a menu **„Rozliczenia"** — do subskrypcji. Ten rozdział zbiera całość w jednym miejscu: od danych profilu, przez użytkowników firmy, po faktury i limity.

13.1. Profil

Ekran **„Ustawienia profilu"** („Zarządzaj swoimi danymi osobowymi i hasłem") pokazuje:

- **awatar** — generowany automatycznie z inicjałów (przesyłanie własnego obrazka nie jest obsługiwane),
- **„Email"** — tylko do odczytu (to identyfikator logowania),
- **„Nazwa"** — edytowalna, zapis przyciskiem **„Zapisz profil"**.

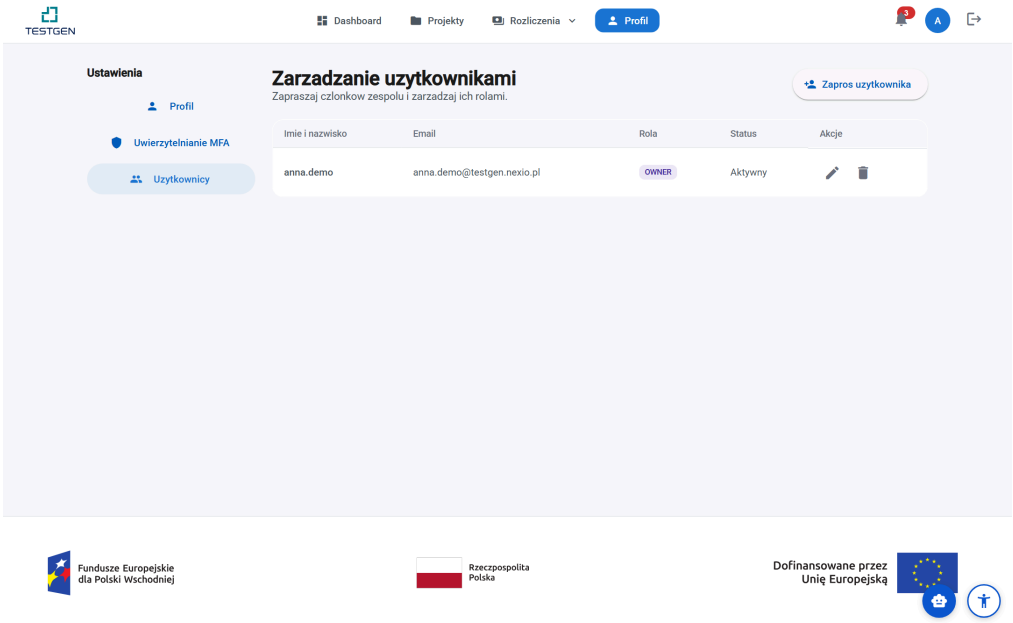


Email jest stały, nazwę zmienisz w każdej chwili; hasło — rozdział 14.

Zmiany nazwy widzą inni członkowie firmy (m.in. na liście użytkowników i w historii zmian przypadków testowych). Zmianę hasła opisuje rozdział 14.1; preferencje powiadomień — 13.6.

13.2. Użytkownicy firmy

Ekran **„Zarządzanie użytkownikami"** (tylko Właściciel) listuje firmę: imię i nazwisko, e-mail, rola, status („Aktywny" / „Oczekuje") oraz akcje — zmiana roli i usunięcie. Przycisk **„Zaproś użytkownika"** otwiera formularz z adresem e-mail i rolą („Użytkownik" lub „Właściciel"); zaproszona osoba ustawia hasło z linku w wiadomości i od razu widzi projekty firmy.



Role działają na całą firmę — zaproszony użytkownik widzi wszystkie projekty.

Role przypomnienie (Część 2, rozdz. 3.4):

Rola	Uprawnienia
Użytkownik	projekty, pliki, analizy, wyniki, przypadki testowe
Właściciel	to co Użytkownik + zarządzanie projektami i ludźmi, subskrypcja, dane fakturowe
Administrator platformy	obsługa TestGEN (m.in. blokady kont, przywracanie projektów)

Blokowanie kont i resetowanie haseł członków firmy wykonuje administrator platformy — Właściciel w aplikacji może usuwać użytkowników i zmieniać role.

13.3. Plany i zmiana subskrypcji

Nowa firma startuje na **Freemium**; zmiany planu dokonuje Właściciel w menu **„Rozliczenia"**:

- **upgrade** (Freemium → Basic → Premium) — natychmiast, z dopłatą proporcjonalną do pozostałego okresu; moduły odblokowują się od razu,
- **downgrade** — od końca bieżącego okresu (do tego czasu możesz go odwołać); po okresie łaski plan wraca do Freemium bez utraty danych,
- **anulowanie** — dostęp do końca opłaconego okresu, potem automatyczny Freemium; reaktywacja możliwa do końca okresu.

Pełny opis (w tym macierz modułów w planach): Rozliczenia — plany i zakup oraz zarządzanie subskrypcją.

13.4. Płatności (tpay)

Płatności obsługuje **tpay** — polski operator, bez konfiguracji po Twojej stronie:

- **BLIK** z Płatnościami Powtarzalnymi (comiesięczne odnowienia bez wpisywania kodu),
- **karta** (3-D Secure) z możliwością zapisania,
- **przelew** (PBL) — wybór banku w panelu tpay.

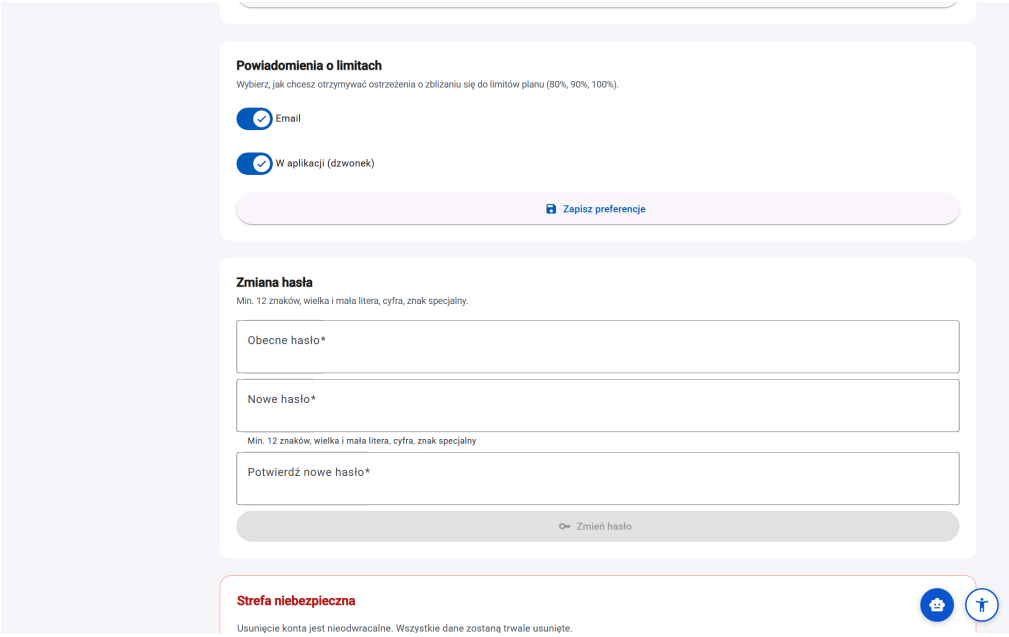
⚠️ **Uwaga:** bez zapisanej metody (karta lub aktywny alias BLIK) miesięczne odnowienie nie powiedzie się — system ponawia próby, a po ich wyczerpaniu włącza 7 dni łaski (z limitami płatnego planu), po których plan wraca do Freemium. Zaktualizuj metodę w menu „Rozliczenia" → „Metody płatności" (poradnik).

13.5. Faktury

Faktura VAT powstaje automatycznie po każdej opłaconej płatności — dostaje ją e-mailem Owner (PDF) i znajdziesz ją na liście w menu „Rozliczenia" → „Faktury". Faktury trzymają **snapshot danych fakturowych** z chwili wystawienia — poprawki danych zadziałają od następnej faktury (poradnik: faktury). Listy faktur i transakcji widzą Owner i User; dane fakturowe i metody płatności — wyłącznie Owner.

13.6. Zużycie limitów i powiadomienia

Liczniki planu (analizy, przypadki testowe, miejsce) sprawdzisz w menu **„Rozliczenia" → „Zużycie zasobów"** — karty z paskami postępu i prognozą wyczerpania. Ostrzeżenia przychodzą przy **80%, 90% i 100%** zużycia; kanały konfigurujesz w profilu (karta **„Powiadomienia o limitach"** — przełączniki **„Email"** i **„W aplikacji (dzwonek)"**):



Dwa kanały ostrzeżeń o limitach — w aplikacji zawsze, e-mailem opcjonalnie.

O zdarzeniach rozliczeniowych (nieudane obciążenie, zmiana planu, gotowa faktura) platforma informuje mailem automatycznie — tych powiadomień nie konfiguruje się osobno.

13.7. Usuwanie konta

Samodzielne usunięcie konta **nie jest obecnie dostępne** — przycisk w „Strefie niebezpiecznej" profilu jest zablokowany. Jeśli chcesz zamknąć konto: wyeksportuj najpierw raporty i dane (Część 2, rozdz. 9), a potem skontaktuj się z obsługą TestGEN — konta i dane firmowe są usuwane z zachowaniem okresu retencji wymaganego przepisami (faktury).

13.8. Ograniczenia

- **Email niezmienny** — zmiana wymaga kontaktu z obsługą.
- **Brak awatara z pliku, strefy czasowej i języka interfejsu** — profil jest szczątkowy z natury (aplikacja jest po polsku).
- **Usuwanie konta przez obsługę** — bez samodzielnej opcji i harmonogramu retencji w aplikacji.

Rozdział 14. Bezpieczeństwo i prywatność

14.1. Hasło

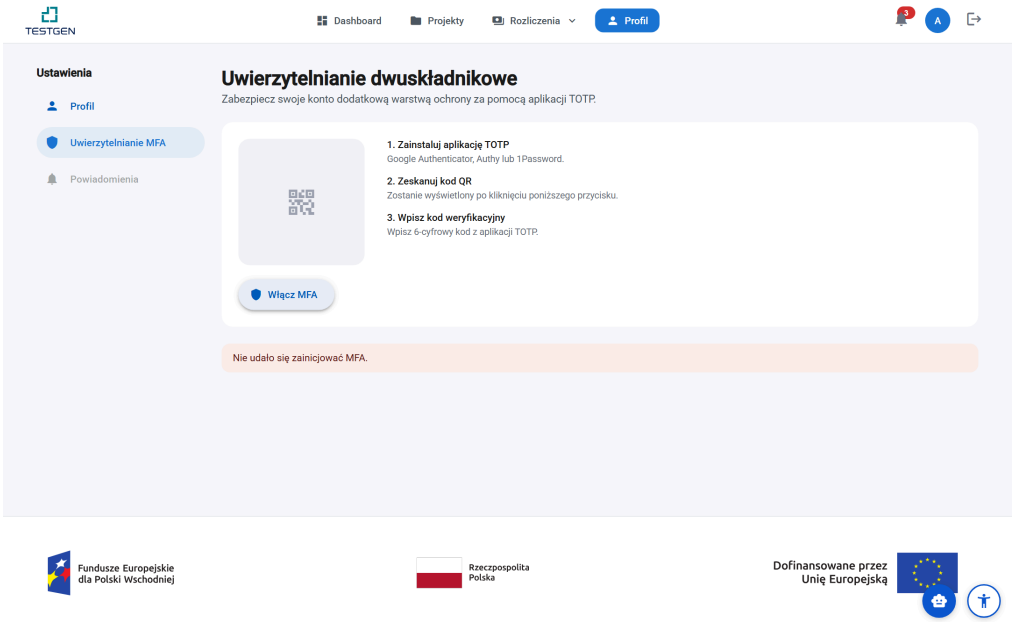
Formularz zmiany hasła (profil) przyjmuje **„Obecne hasło"**, **„Nowe hasło"** i **„Potwierdź nowe hasło"**. Nowe hasło musi mieć **minimum 12 znaków**, wielką i małą literę, cyfrę oraz znak specjalny — platforma hasła przechowuje w formie skrótu (Argon2id), nigdy jawnym tekstem.

💡 **Wskazówka:** po zmianie hasła pozostałe sesje (np. druga przeglądarka) wylogują się przy następnym żądaniu — to zamierzona obrona przed zapomnianymi sesjami. Reset zapomnianego hasła: link „Nie pamiętasz hasła?" na ekranie logowania.

14.2. Weryfikacja dwuetapowa (MFA)

Ekran **„Uwierzytelnianie dwuskładnikowe"** dodaje drugą barierę logowania — kod z aplikacji TOTP. Włączenie w trzech krokach:

1. **Zainstaluj aplikację TOTP** — Google Authenticator, Authy lub 1Password.
2. **Zeskanuj kod QR** (alternatywnie przepisz pokazany klucz ręcznie).
3. **Wpisz kod weryfikacyjny** — sześciocyfrowy z aplikacji — i zatwierdź przyciskiem „Weryfikuj i włącz MFA".



QR i klucz ręczny prowadzą do tego samego konta — skanowanie wygodniejsze, przepisywanie pewniejsze.

Po włączeniu platforma pokaże **kody zapasowe** — zachowaj je w bezpiecznym miejscu, bo **wyświetlane są tylko raz**: każdy z nich jednokrotnie zastąpi kod z aplikacji (gdy np. wymienisz telefon). Wyłączenie MFA wymaga podania hasła.

⚠️ **Uwaga:** utrata telefonu **i** kodów zapasowych oznacza kontakt z administratorem platformy — nie ma samodzielnej drogi odzyskania. Zrób kopię kodów w dniu włączenia. MFA jest opcjonalne, ale zalecane dla Właścicieli (subskrypcja, dane fakturowe).

14.3. Izolacja danych firmy

Twoje dane widzi wyłącznie Twoja firma: projekty, pliki, wyniki i integracje są izolowane między organizacjami na poziomie bazy danych, a uprawnienia wewnątrz firmy wynikają z ról (13.2). Poświadczenia integracji (tokeny Jira/ADO) szyfrowane są w bazie i odszyfrowywane tylko na czas synchronizacji; operacje wrażliwe (zmiany ról, synchronizacje, eksporty) trafiają do rejestru audytowego platformy.

14.4. RODO i prywatność danych

W skrócie, co ważne dla codziennej pracy:

- **Przetwarzane dane** to przede wszystkim materiały, które sam wgrywasz (kod, dokumentacja, zrzuty) oraz dane kont i rozliczeń.
- **Materiały projektowe należą do Twojej firmy** — nie są używane do trenowania modeli; analizy deterministyczne działają offline, a tryby korzystające z modeli zewnętrznych konfiguruje administrator platformy (Część 3, 11.1).

- **Prawa osób** (dostęp, sprostowanie, usunięcie) realizujesz przez obsługę — samodzielny eksport danych robisz raportami i eksportami przypadków, a zamknięcie konta opisano w 13.7.
- **Retencja:** faktury i dane rozliczeniowe są przechowywane przez okres wymagany przepisami; pliki projektu możesz usunąć w każdej chwili (odzyskując limit miejsca).

Szczegółowe kwestie prawne reguluje umowa i polityka prywatności platformy — ten podręcznik opisuje mechanizmy techniczne.

14.5. Dostęp API i klucze

Do skryptów, pipeline'ów CI/CD i narzędzia wiersza poleceń używaj **kluczy API** zamiast hasła (Część 3, 11.2). Klucze tworzy i odwołuje **właściciel firmy** w „Ustawienia" → „Klucze API".

Zasady bezpiecznego użycia:

- **Najmniejszy potrzebny zakres.** Pipeline, który tylko pobiera wyniki, dostaje klucz „Tylko odczyt" — wyciek takiego klucza nie pozwoli uruchamiać analiz ani zmieniać danych.
- **Osobny klucz na każdy system.** Nazwij go po pipeline'ie lub integracji; gdy trzeba odciąć jeden system, odwołujesz jeden klucz, a nie wszystkie.
- **Termin wygaśnięcia.** Ustaw go dla kluczy wydanych na czas projektu albo dla wykonawcy zewnętrznego.
- **Klucz trzymaj w sekretach**, nie w repozytorium — np. jako zmienna `TESTGEN_API_KEY` w sekretach CI. Pełny klucz zobaczysz tylko raz, przy tworzeniu; platforma przechowuje wyłącznie jego skrót.
- **Sprawdzaj „Ostatnie użycie".** Klucz, którego nic nie używa od miesięcy, odwołaj.

Klucz API nie daje dostępu do zarządzania kluczami ani webhookami — te operacje wymagają zalogowania w interfejsie. Utworzenie, odwołanie i użycie klucza trafiają do rejestru audytowego firmy.

14.6. Ograniczenia

- **Kody zapasowe MFA tylko raz** — regeneracja wymaga kontaktu z obsługą; zapisz je przy włączeniu.
- **Brak widoku historii logowań** dla użytkownika (rejestr audytowy prowadzi platforma).
- **Klucza API nie da się odzyskać** — zgubiony utwórz od nowa, a stary odwołaj (14.5).
- **Kluczami API i webhookami zarządza tylko właściciel firmy.**

Część V — Rozwiązywanie problemów

Rozdział 15. FAQ

Odpowiedzi oparliśmy na pytaniach z poradników modułowych (rozliczenia, bezpieczeństwo, luki funkcjonalne) i na problemach, które najczęściej przeszkadzają w codziennej pracy z platformą. Jeśli Twojego pytania tu nie ma — zajrzyj do rozdziału 16 i indeksu terminów.

15.1. Konto i dostęp

P: Czy do rejestracji potrzeba czegoś więcej niż e-mail? O: Nie — imię, nazwisko, adres e-mail, hasło (min. 12 znaków) oraz nazwa firmy i NIP, jeśli chcesz otrzymywać faktury.

P: Wiadomość aktywacyjna nie przyszła. O: Sprawdź folder oferty/spam. Po wygaśnięciu linku poproś o ponowne wysłanie ze strony logowania.

P: Jak zresetować hasło? O: Link „Nie pamiętasz hasła?" na ekranie logowania wyśle wiadomość z jednorazowym linkiem. Zmiana w aplikacji: profil → zmiana hasła (wymaga obecnego hasła).

P: Zmieniłem telefon z aplikacją TOTP i nie mam kodów zapasowych. O: To sytuacja do obsługi TestGEN — samodzielnej drogi nie ma. Dlatego kody zapasowe zapisz w dniu włączania MFA (wyświetlane tylko raz).

P: Czy mogę zmienić adres e-mail konta? O: Nie samodzielnie — e-mail jest identyfikatorem logowania (pole tylko do odczytu). Zmiana wymaga kontaktu z obsługą.

P: Pracuję w dwóch firmach — jedno konto wystarczy? O: Tak, jedno konto może należeć do wielu firm; przy logowaniu wybierasz organizację, na której chcesz pracować.

P: Kto może blokować konta członków firmy? O: W aplikacji Właściciel może usuwać użytkowników i zmieniać role; blokady i reset haseł wykonuje administrator platformy.

P: Czy muszę włączyć MFA? O: Nie, jest opcjonalne — ale zalecane zwłaszcza dla Właścicieli (subskrypcja, dane fakturowe).

15.2. Projekty

P: Ile projektów mogę mieć? O: Bez limitu w każdym planie — limitowane jest miejsce na pliki (0,5/5/10 GB) i liczba plików na analizę (5/25/50).

P: Jak zarchiwizować projekt? O: Samodzielna archiwizacja nie istnieje — możesz projekt usunąć (odwracalnie, przywraca administrator) albo poprosić obsługę o zamknięcie na stanie.

P: Usunąłem projekt przez przypadek — da się odzyskać? O: Tak — usuwanie jest miękkie; administrator platformy może projekt przywrócić.

P: Czy mogę ograniczyć współpracownika do jednego projektu? O: Nie — uprawnienia działają na poziomie firmy.

P: Komunikat „Projekt o tej nazwie już istnieje". O: Nazwy są unikalne w firmie. Zmień nazwę albo otwórz istniejący projekt.

P: Czym różni się status „Utworzony" od „Aktywny"? O: „Utworzony" = brak plików; pierwszy upload lub repozytorium daje „Aktywny", analiza w trakcie „W analizie", jej koniec „Zakończony".

P: Lista projektów jest wspólna dla całej firmy? O: Tak — zespół pracuje na tych samych materiałach; własne porządkowanie najlepiej rozwiązać konwencją nazewniczą.

15.3. Analiza kodu

P: Jakie języki rozpoznaje platforma? O: Jedenastu; pełna analiza struktury obejmuje osiem (Python, Java, JavaScript, TypeScript, PHP, C, C++, C#) — pełna tabela w Części 2, 4.8.

P: Czy wyniki analizy są powtarzalne? O: Tak — silniki są deterministyczne: te same dane wejściowe dają ten sam wynik.

P: Ile trwa analiza? O: Małe projekty 1–2 minuty; duże zależą od liczby plików — postęp widać na etapach.

P: Anulowana analiza zwraca limit? O: Nie — licznik rośnie o każde uruchomienie; resetuje się co okres rozliczeniowy.

P: Jak często uruchamiać analizę? O: Po istotnych zmianach lub scaleniach; przed wydaniem pełną i z porównaniem wersji.

P: Wykryty język jest błędny. O: Popraw klasyfikację w „Raporcie preprocessingu" — analiza policzy struktury poprawnie dopiero po tym.

P: Framework testowy: zostawić „— Automatyczny —"? O: Tak — wykrywanie z plików konfiguracyjnych wystarcza w przeważającej liczbie projektów.

15.4. Dokumentacja, UML i GUI

P: Jakie formaty dokumentacji są obsługiwane? O: PDF, DOCX, Markdown, HTML, TXT; PDF do 500 stron, skany przez OCR.

P: Dlaczego tabele z PDF nie są widoczne w wynikach? O: Tabele z PDF nie są odczytywane — prześlij DOCX/HTML/Markdown.

P: Diagram UML: źródło czy rysunek? O: Zawsze źródło (PlantUML/XMI) — pełna struktura; z rysunku tylko nazwy i kształty.

P: Jaka rozdzielczość zrzutów GUI jest wymagana? O: Formalnego minimum nie ma; naturalna rozdzielczość bez kompresji, limit 20 MB.

P: Czy kolejność zrzutów GUI ma znaczenie? O: Tak — numery ekranów budują przepływy użytkownika.

P: Diagramy stanów z obrazu rozpoznane jako aktywności. O: Znane ograniczenie geometrii — dla stanów przesyłaj źródło.

15.5. Generowanie testów

P: W jakich frameworkach powstanie kod testów? O: Dziesięć generatorów (pytest, JUnit, NUnit, Mocha/Jest, Cypress, Selenium, Playwright, Appium, SikuliX, PHPUnit/Pest) + testy bazodanowe; unittest/TestNG/MSTest tylko wykrywane.

P: Czy wygenerowane przypadki są gotowe do użycia? O: Mają status „Szkic" — przejrzyj, dopracuj i zatwierdź („Gotowy"); kod działa bez poprawek po zatwierdzeniu.

P: Jak nadawane są priorytety? O: Regułowo (sygnatura funkcji, rodzaj logiki) — to wskazówka do zmiany w edytorze.

P: Co to „przypadek brzegowy"? O: Scenariusz na wartościach granicznych parametrów (zero, pusta kolekcja, maksimum).

P: Jak zrobić zestaw testów regresyjnych? O: Regresja to wybór przypadków z porównania wersji — szczegóły w Części 2, 4.7.

P: Czy mogę usunąć zbędne przypadki hurtowo? O: Tak — zaznacz wiersze i „Usuń" z paska operacji masowych.

P: Czy przypadki powstaną z dokumentacji? O: Tak — z zaakceptowanych wymagań; odrzucone nie generują testów.

15.6. Bezpieczeństwo

P: Czym różni się SAST od DAST? O: SAST bada kod statycznie; DAST działającą aplikację pod adresem URL (do 30 minut). Potwierdzenie przez oba podbija wagę.

P: Czy skany wymagają internetu? O: SAST, taint, konfiguracja i zależności działają offline (lokalna baza CVE synchronizowana co miesiąc z NVD, OSV.dev i GitHub Advisory). DAST testuje żywą aplikację.

P: Mam fałszywy alarm. O: Ustaw status znaleziska na „Fałszywe" — przeglądy będą krótsze.

P: Jak zastosować poprawkę? O: Pobierz `.patch` i zastosuj `git apply`; przy wielu naprawach w jednym pliku sprawdź konflikty.

P: Czy PoC jest bezpieczny? O: Filtr blokuje operacje destrukcyjne; używaj tylko na systemach, do których masz prawa.

P: Co to SBOM i po co mi? O: Bilans materiałów projektu (CycloneDX/SPDX) — lista komponentów; pobierzesz z raportu zależności.

15.7. Eksport i raporty

P: Jakie formaty raportów są dostępne? O: PDF, HTML, CSV, Excel, JSON, YAML oraz druk.

P: Czy mogę dodać logo firmy do raportu? O: Nie — własne szablony i branding obecnie nie istnieją.

P: Jak przenieść przypadki do TestRail? O: „Eksportuj do TMS" → „TestRail (CSV)" z mapowaniem pól — plik importujesz w TestRail.

P: Jak długo plik eksportu czeka na serwerze? O: 7 dni od wygenerowania.

P: Który raport wybrać na spotkanie z zarządem? O: „Podsumowanie wykonawcze"; „Szczegóły techniczne" zostaw zespołowi.

15.8. Integracje

P: Z jakimi trackerami działa bezpośrednia integracja? O: Jira Cloud i Azure DevOps — dla luk i rekomendacji (Część 3, 10.2–10.4). GitHub Issues i GitLab Issues nie są obsługiwane.

P: A TestRail, Xray i Zephyr? O: Tak, bezpośrednio — przyciski „Eksportuj do TestRail / Xray / Zephyr" na liście testów. Połączenie konfigurujesz w „Integracje" → „Systemy zarządzania testami" (Część 3, 10.8).

P: Eksport do TestRaila zgłasza „Integracja z TestRail nie jest skonfigurowana". O: Konfiguracja TMS jest osobna dla każdego projektu. Otwórz „Integracje" w tym projekcie i uzupełnij „TestRail" (albo „Jira Xray / Zephyr Scale").

P: Każdy eksport do TestRaila lub Zephyra tworzy nowe przypadki zamiast aktualizować. O: Brakuje pola własnego `testgen_id` w TMS-ie. Załóż je (typ tekstowy) — od następnego eksportu przypadki będą aktualizowane. Duplikaty powstałe wcześniej usuń ręcznie.

P: Zmiany z TestRaila nie wracają do TestGEN. O: Sprawdź na ekranie „Synchronizacja dwukierunkowa", czy synchronizacja jest „Włączona" i czy kierunek to „TMS → TestGEN" albo „W obie strony". Sam zapis ustawień jej nie uruchamia (Część 3, 10.9).

P: Synchronizacja ma stan „Wymaga uwagi". O: Trzy przebiegi z rzędu zakończyły się błędem — powód znajdziesz na karcie stanu. Najczęściej wygasł klucz API TMS-a; popraw go w konfiguracji. Synchronizacja wróci do działania sama.

P: Ponowna synchronizacja luk utworzyła duplikaty zgłoszeń. O: Tak działa — sync luk i rekomendacji nie deduplikuje. Sprzątaj po etykietach `testgen-gap` / `testgen-rec` i synchronizuj po przeglądzie.

P: „Test połączenia" wyrzuca błąd uwierzytelnienia. O: Sprawdź ważność tokenu/PAT/klucza API i uprawnienia konta w projekcie docelowym.

P: Pull requesty nie dostają status checka od TestGEN. O: Sprawdź w „Ustawienia" → „Integracja GitHub" (albo „GitLab"), czy repozytorium jest podłączone i czy w „Ostatnich zdarzeniach" pojawiają się pull requesty. Brak zdarzeń oznacza, że webhook nie dociera — odłącz i podłącz repozytorium ponownie (Część 3, 10.5).

P: Import z Jiry nie pobrał żadnych zgłoszeń. O: Przy zakresie „Tylko zmiany" to poprawne, gdy od ostatniego importu nic się nie zmieniło. Wybierz „Wszystko", aby pobrać całość (Część 3, 10.10).

P: Czy integracja dotyczy jednego projektu? O: Jira, Azure DevOps i TMS — tak, konfiguruje się je osobno dla każdego projektu. GitHub, GitLab, klucze API i webhooki są ustawieniami całej firmy.

15.9. Płatności i plany

P: Czy mogę płacić rocznie? O: Nie — jedyny cykl to miesięczny (Freemium 0 zł, Basic 30 zł, Premium 50 zł).

P: Jakie metody płatności obsługuje tpay? O: BLIK (płatności powtarzalne), karta (3-D Secure, z zapisaniem) i przelew.

P: Czy muszę zapisać kartę? O: Nie — ale bez zapisanej metody odnowienie nie powiedzie się i po 7 dniach łaski plan wróci do Freemium.

P: Czy downgrade zwraca pieniądze? O: Nie — obowiązuje od końca okresu, bez zwrotów; do tego czasu możesz go odwołać.

P: Co się dzieje po anulowaniu subskrypcji? O: Dostęp do końca opłaconego okresu, potem Freemium — bez utraty danych; reaktywacja możliwa do końca okresu.

P: Kto widzi faktury i transakcje? O: Listy — Owner i User; zakupy, dane fakturowe i metody płatności — wyłącznie Owner.

15.10. Błędy i problemy

P: Komunikat o funkcji dostępnej w wyższym planie (402). O: Moduł jest poza planem — komunikat prowadzi na stronę planów; po zmianie planu moduł odblokuje się bez ponownego konfigurowania.

P: Dostaję błędy 429 — przekroczony limit żądań. O: Plan ma limit 100 (Freemium/Basic) lub 1000 (Premium) żądań na minutę — łącznie interfejs i skrypty. Odczekaj; jeśli to limit analiz — sprawdź „Zużycie zasobów".

P: Analiza zakończyła się błędem. O: Ekran pokaże przyczynę (najczęściej: brak plików, limit plików, uszkodzony plik wejściowy); kontekst w rozdz. 16.

P: Upload odrzuca pliki. O: Sprawdź rozszerzenie i rozmiar (100 MB Freemium / 500 MB Basic+; zrzuty GUI 20 MB); archiwa mają dodatkowe limity rozpakowania.

P: Gdzie zgłaszać problemy techniczne? O: Przez kanały wsparcia z rozdziału 17; Premium obejmuje wsparcie priorytetowe.

Rozdział 16. Troubleshooting

Problemy pogrupowane tematycznie; każdy z objawem, przyczyną i rozwiązaniem. Jeśli twojego przypadku nie ma — rozdział 17.

16.1. Problemy z analizą

Analiza kończy się statusem błędu. Sprawdź komunikat na ekranie analizy. Najczęstsze przyczyny: projekt bez plików, przekroczony limit plików na analizę (5/25/50 — zmniejsz paczkę albo zmień plan), uszkodzony plik wejściowy (zobacz, który plik ma status błędu na liście) lub chwilowe obciążenie kolejki zadań — spróbuj ponownie za kilka minut.

Analiza dokumentacji zwraca „Brak wymagań". Analiza luk wymaga wcześniej: (1) analizy dokumentacji (wymagania) i (2) analizy kodu (byty). Uruchom obie — dopiero wtedy „Luki funkcjonalne" mają co porównywać.

Wyniki analizy puste przy niepustym projekcie. Sprawdź „Raport preprocessingu": jeśli język wykryty błędnie (albo pliki czekają na klasyfikację), struktury się nie policzą. Popraw klasyfikację i uruchom analizę ponownie.

Wynik compliance spadł po dodaniu funkcjonalności. To normalne: nowy kod bez wymagań rodzi luki „Nadmiarowe", częściowe nakładanie się — „Różne". Jeśli kod jest poprawny, dopisz wymagania do dokumentacji i przeanalizuj ponownie.

Luka ma niski wynik dopasowania, ale wymaganie jest zaimplementowane. Niski wynik oznacza słabą korelację nazwy funkcji/docstring z tekstem wymagania. Rozwiązania: lepszy docstring (np. z tekstem wymagania), przemianowanie funkcji, albo odrzucenie pozycji z powodem.

Skan bezpieczeństwa ma status „Błąd". Sprawdź komunikat przy karcie skanu. Dla DAST najczęstsze przyczyny: niedostępny adres aplikacji (sprawdź `target_url`), przekroczenie limitu 30 minut albo wymóg autentykacji na chronionych ścieżkach (skonfiguruj „Formularz (login)" / „Token" / „Basic auth").

16.2. Problemy z generowaniem testów

Wygenerowany kod testów nie przechodzi. Najpierw sprawdź, czy przypadek był zatwierdzony po przeglądzie („Szkic" może zawierać błędy generatora). Jeśli kroki przypadku są poprawne, a kod dalej nie działa — popraw go w repozytorium (generator tworzy szkielet zgodny ze scenariuszem, nie gwarantuje semantyki Twojej aplikacji) i zgłoś przypadek do wsparcia (rozdz. 17).

Brak przypadków z kodu w Go/Ruby/Swift/Kotlin. Pełna analiza struktury obejmuje osiem języków (Część 2, 4.8); dla pozostałych generuj testy z wymagań i diagramów.

Szkielet (skeleton) ma niefortunną nazwę funkcji. Nazwa pochodzi z tekstu wymagania; przy długich wymaganiach bywa nieoptymalna. Popraw nazwę w generowanym pliku i dodaj docstring.

Przypadki „Nadmiarowe" pokazują funkcje pomocnicze. Klasyfikator pomija `test_*`, `__init__`, `setup/teardown`, `main` i akcesory; pozostałe pomocniki oznacz `@internal` lub prefiksem `_` albo dodaj im wymagania.

16.3. Problemy z integracjami

„Test połączenia" — błąd uwierzytelnienia (Jira/ADO). Token wygasł albo konto nie ma uprawnień tworzenia zgłoszeń w projekcie docelowym. Wygeneruj nowy token (Jira: id.atlassian.com → Security; ADO: PAT z zakresem *Work Items: Read & Write*), zapisz konfigurację i przetestuj ponownie.

Synchronizacja zwraca „Błędy: N". Najczęstsze przyczyny: wygasłe poświadczenia (patrz wyżej), przekroczony limit 10 synchronizacji na godzinę (odczekaj) albo tymczasowa niedostępność trackera. Historia synchronizacji pokazuje status każdego przebiegu.

„Błąd połączenia" przy zapisie konfiguracji. Sprawdź adres instancji (Jira Cloud: `https://…atlassian.net`) i kompletność pól — platforma waliduje URL przed zapisem.

Po synchronizacji brak zgłoszeń w trackerze. Sprawdź, czy synchronizowałeś właściwy typ („Sync luk funkcjonalnych" vs „Sync rekomendacji") i czy w projekcie istnieją luki/rekomendacje (bez nich wysyłka ma pustą paczkę). Etykiet `testgen-gap` / `testgen-rec` użyj do odszukania zgłoszeń.

16.4. Problemy z wydajnością

Analiza trwa bardzo długo. Skala decyduje: duże projekty (setki plików) liczą się dłużej — postęp na etapach pokazuje, czy praca idzie. Jeśli etap stoi bez ruchu kilka minut, sprawdź komunikat błędu po zakończeniu; przy powtarzających się problemach zgłoś przez wsparcie (dołącz nazwę projektu i czas zdarzenia).

Analiza luk trwa > 60 sekund. Porównanie wymagań z bytami rośnie z iloczynem ich liczby (1000 × 1000 = milion par). W bardzo dużych projektach rozważ podział na moduły (albo ogranicz dokumenty w dialogu uruchomienia).

Eksport raportu się wlecze. Duże raporty z wykresami renderują się dłużej — dialog postępu pokazuje etapy i pozwala anulować. Plik pozostaje dostępny 7 dni, więc jednorazowy koszt wystarcza.

Interfejs zamula przy dużych listach. Listy stronicują (domyślnie 20–50 wierszy) — zawęź filtry zamiast przewijać tysiące wierszy; szukajka i „Grupuj według" redukują liczbę pozycji.

16.5. Problemy z dostępem i kontem

Nie mogę się zalogować — „Wymagana weryfikacja MFA". Wpisz sześciocyfrowy kod z aplikacji TOTP (albo użyj kodu zapasowego). Pięć błędnych prób blokuje na chwilę — odczekaj przed kolejną.

Utracony telefon z TOTP i brak kodów zapasowych. Skontaktuj się z obsługą — samodzielnej ścieżki odzyskania nie ma; przygotuj się na weryfikację tożsamości.

403 przy próbie edycji projektu. Edycja i usuwanie wymagają roli Właściciela — poproś Właściciela firmy o zmianę roli (13.2).

402 przy uruchomieniu analizy dokumentacji/UML/GUI. Moduł poza planem: dokumentacja/UML/GUI wymagają Basic, bezpieczeństwo i luki — Premium. Zmień plan (upgrade działa natychmiast) albo użyj modułów planu.

Blokada konta po nieudanych próbach logowania. Chwilowa blokada bezpieczeństwa mija sama; upewnij się, że caps lock wyłączony i używasz właściwego hasła (reset — „Nie pamiętasz hasła?").

Rozdział 17. Wsparcie techniczne

17.1. Jak uzyskać pomoc

Zanim napiszesz: sprawdź FAQ (rozdz. 15) i troubleshooting (rozdz. 16) — najczęstsze problemy mają tam gotowe rozwiązania. Jeśli temat jest merytoryczny (jak działa funkcja), indeks terminów i poradniki modułowe (spis) odpowiedzą szybciej niż kanał wsparcia.

17.2. Kanały wsparcia

Kanał	Dla kogo	Cechy
Wsparcie priorytetowe	plan Premium	pierwszeństwo w kolejce, krótszy czas reakcji

Kanał	Dla kogo	Cechy
Wsparcie standardowe	wszystkie plany	zgłoszenia obsługiwane w kolejce
Dokumentacja	wszyscy	podręcznik + poradniki modułowe, dostępne od ręki

Dane kontaktowe (adres e-mail, portal zgłoszeń) znajdziesz na stronie platformy — ten podręcznik celowo nie zapisuje ich na stałe, żeby nie zdezaktualizowały się przy zmianie.

17.3. Zgłaszanie błędów

Dobre zgłoszenie skraca czas naprawy o połowę. Dołącz:

1. **Co zrobiłeś/łaś** — kroki jedno po drugim (ekran, przycisk),
2. **Co się stało** — komunikat błędu (przepisz lub zrzut ekranu),
3. **Co oczekiwałeś/łaś** — zamierzony efekt,
4. **Kiedy** — data i godzina (lub przybliżony przedział),
5. **Kontekst** — projekt (nazwa), przeglądarka, plan.

⚠️ **Uwaga:** nie wklejaj do zgłoszenia haseł, tokenów ani kluczy — wystarczy opis. Jeśli błąd zawiera wrażliwy fragment kodu, ogranicz go do minimum potrzebnego do reprodukcji.

17.4. Propozycje nowych funkcji

Pomysły są mile widziane i trafiają na listę rozwojową — to z niej wyrasta roadmapa rozwoju platformy. Opisz: problem, który funkcja ma rozwiązać (ważniejsze niż sam pomysł na rozwiązanie), jak dziś obchodzisz brak funkcji oraz minimalny zestaw, który byłby dla Ciebie użyteczny. Propozycje z największą liczbą podobnych zgłoszeń mają pierwszeństwo w planowaniu.

Utrzymanie dokumentu: zmiany UI = aktualizacja zrzutów i opisów; zrzuty pochodzą z rzeczywistej aplikacji (stan na 2026-09-20; rozdział 14.5 zaktualizowany 2026-09-28 bez nowych zrzutów). Spójność terminów: Glosariusz i Indeks terminów. To ostatnia część podręcznika — zacznij od Części 1 i spisu treści.

Przewodnik — nawigacja i ekrany główne

> Dokumentacja P2, Epic 6. Frontend TestGEN (przeglądarka lub aplikacja mobilna — ten sam interfejs).

1. Logowanie i start

1. `/login` — email + hasło; przy włączonym MFA podajesz kod jednorazowy (`/mfa-verify`).
2. Należysz do więcej niż jednej firmy? Wybierz tenant na `/select-tenant`.
3. Lądujesz na **dashboardzie** (`/dashboard`): lista projektów, statystyki aktywności, skróty do ostatnich analiz.

Pierwsze kroki w nowej firmie: zaproś współpracowników (`/users/invite`), utwórz projekt i wgraj pliki (02 — Projekty i upload).

2. Górny pasek (navbar)

- **Menu główne** — projekty, wyniki, raporty, rozliczenia (zależnie od roli),
- **dzwonek powiadomień** — powiadomienia o progach limitów (80/90/100%), licznik nieprzeczytanych, „oznacz wszystkie" (szczegóły: `docs/final/user-guide/billing/`),
- **pasek dostępności** — przełączniki wysokiego kontrastu i wyłączenia animacji (05 — Dostępność),
- **profil** — dane konta, MFA, wylogowanie.

3. Ekrany i ich adresy

Ekran	Adres	Kto widzi
Dashboard	`/dashboard`	wszyscy
Projekty	`/projects`	wszyscy w tenancie
Szczegóły projektu	`/projects/{id}` + zakładki (pliki, analiza, wyniki, testy, raporty)	wszyscy w tenancie
Test case'y	`/projects/{id}/test-cases`	wszyscy
Raporty	`/projects/{id}/reports`	wszyscy
Rozliczenia	`/billing/...` (plany, subskrypcja, faktury, zużycie)	plany/faktury — wszyscy; zakupy — Owner
Ustawienia	`/settings/profile`, `/settings/mfa`, `/settings/users`	profil/MFA — ja; użytkownicy — Owner
Panel administratora	`/admin`	Admin platformy

Niektóre zakładki modułów analiz mogą być **zablokowane planem** (kłódka) — opis w `docs/final/user-guide/billing/`.

4. Błędy i stany brakowe

- `/403` — brak uprawnień do ekranu (np. panel admina bez roli Admin),
- 404 — nieistniejący adres,
- bannery limitów — wyczerpany limit operacji (co robić: 05 — Zużycie i dokumentacja billingowa).

5. Aplikacja mobilna/desktopowa

Ta sama aplikacja działa w natywnej powłoce (Android, iOS, Windows; Linux otwiera przeglądarkę). Linki `testgen://…` z maili i powiadomień otwierają właściwy ekran aplikacji. Bez sieci powłoka pokazuje komunikat offline i wraca po odzyskaniu połączenia.

Przewodnik — projekty i upload plików

> Dokumentacja P2, Epic 6 — Stories 6-1, 6-2, 6-3.

1. Tworzenie projektu

Na `/projects` przycisk nowego projektu: nazwa i opis. Projekt jest przestrzenią roboczą dla analiz jednej aplikacji — pliki, wyniki, test case'y i raporty żyją w obrębie projektu. Usunięcie projektu (wymaga potwierdzenia w dialogu) usuwa powiązane dane.

2. Upload plików (drag & drop)

Ekran plików projektu (`/projects/{id}/files` i upload dokumentacji):

1. **Przeciągnij pliki** na obszar przesyłania albo wybierz je przyciskiem — można wiele plików naraz.
2. Pliki są sprawdzane od razu: **dozwolone rozszerzenia** i **limit rozmiaru** (błędne pliki są oznaczane przed startem przesyłania — nic nie „wisi" na serwerze).
3. Upload biegnie **równolegle**, każdy plik ma własny pasek postępu i status: oczekuje → przesyłanie → OK / błąd / anulowano.
4. Podsumowanie pokazuje ile plików wgrano, ile nie udało się — i dlaczego; błędne można powtórzyć bez wgrywania od nowa całości.

Uwagi: limit plików na analizę zależy od planu (5/25/50 — `docs/final/user-guide/billing/`); upload może zostać odrzucony przez limit storage tenanta (banner z wyjaśnieniem).

3. Dokumenty i źródła projektu

Zakładki projektu prowadzą do poszczególnych źródeł analizy: kod (pliki), dokumentacja (`/documents` — upload i podgląd), diagramy UML, zrzuty GUI. Każde źródło pokazuje status przetworzenia (np. raport preprocessingu dokumentacji przed uruchomieniem analizy).

4. Uruchomienie analizy

1. Zakładka **Analiza** projektu — wybierz źródła i rodzaj analizy.
2. Uruchom — postęp zadania widać na ekranie; zakończenie ogłasza komunikat (działa też z czytnikiem ekranu).
3. Wyniki analizy pojawiają się w przeglądarce wyników — patrz 03 — Wyniki i test case'y.

Analiza zużywa limit planu (licznik „Analizy"); przy braku limitu system poinformuje błędem z podpowiedzią zmiany planu.

5. Porządkowanie

- Filtr listy projektów na dashboardzie (nazwa, status, data),
- historia analiz projektu zachowuje wyniki do porównań między uruchomieniami,
- stare pliki można usuwać, by odzyskać limit storage.

Przewodnik — wyniki analiz i test case'y

> Dokumentacja P2, Epic 6 — Stories 6-4, 6-5.

1. Przeglądarka wyników

Zakładka **Wyniki** projektu (`/projects/{id}/results`):

- **Panel filtrów** — zawężanie po typie znaleziska, wadze (severity), statusie, module; filtry działają bez przeładowania,
- **szczegóły znaleziska** — opis, kontekst, źródło,
- **podświetlanie kodu** — fragmenty kodu z zachowaną składnią,
- **diff viewer** — porównanie wersji obok siebie (np. rozbieżność kod ↔ dokumentacja).

Te same komponenty obsługują wyniki wszystkich modułów analiz (dokumentacja, UML/GUI, bezpieczeństwo, luki funkcjonalne) — uczysz się ich raz.

2. Zarządzanie test case'ami

Zakładka **Test case'y** (`/projects/{id}/test-cases`) — kompletny widok zarządzania wygenerowanymi przypadkami testowymi:

Czynność	Jak
Przegląd	tabela TC projektu z filtrowaniem grupowym; wejście do szczegółów
Edycja	edytor TC — kroki, oczekiwane rezultaty, priorytet
Operacje masowe	pasek akcji masowych: zmiana statusu, eksport, usunięcie zaznaczonych
Historia	dialog historii zmian TC (kto, co, kiedy)
Eksport do TMS	dialog eksportu — **CSV lub XML** do zaimportowania w narzędziu TMS (TestRail, Xray…)
Import	mapowanie pól przy imporcie TC z zewnątrz

Zmiany zapisują się automatycznie; komunikat potwierdzenia czyta się także w czytniku ekranu.

3. Statusy i priorytety

Test case'y mają statusy (np. szkic → zaakceptowany → archiwalny) — przydają się przy przeglądzie wygenerowanych treści przed użyciem. Operacje masowe pozwalają zaakceptować całą paczkę po weryfikacji.

4. Limity

Generowanie test case'ów zżyna limit planu (50/250/bez limitu miesięcznie). Licznik zużycia — dashboard zużycia (`/billing/usage` , `docs/final/user-guide/billing/`).

Przewodnik — raporty i eksport

> Dokumentacja P2, Epic 6 — Stories 6-6, 6-7 (FR43–FR45).

1. Raporty dwupoziomowe

Zakładka **Raporty** projektu (`/projects/{id}/reports`) generuje raport z wyników analizy w dwóch warstwach:

- **Executive summary** — dla zarządu/interesariuszy: podsumowanie jakości, metryki zagregowane, najważniejsze ryzyka; bez żargonu,
- **szczegóły techniczne** — dla zespołów QA/dev: pełne listy znalezisk z kontekstem, odwołania do kodu.

Obie warstwy powstają z tego samego zbioru wyników — różni je układ i poziom szczegółowości. Podgląd na ekranie pokazuje dokładnie to, co trafi do pliku.

2. Formaty eksportu

Przyciski eksportu generują plik w formacie: **PDF, HTML, CSV, Excel, JSON, YAML**. Dłuższe eksporty pokazują dialog postępu (anulowalny); ukończenie jest ogłaszane komunikatem. Wykresy w raportach renderują się niezależnie od przeglądarki — plik wygląda tak samo wszędzie.

Wybór formatu w praktyce:

Format	Kiedy
PDF	do wysyłki/archiwizacji, wersja do druku
HTML	podgląd w przeglądarce, intranet
CSV / Excel	dalsza analiza w arkuszu
JSON / YAML	wrzutka do narzędzi/pipeline'ów

3. Eksport test case'ów do TMS

Osobny eksport z widoku test case'ów (CSV/XML — patrz 03 — Wyniki i test case'y) — plik importujesz w swoim narzędziu do zarządzania testami.

4. Powiązania

- Wyniki w raportach pochodzą z przeglądarki wyników — te same filtry i statusy,
- raporty można porównywać między analizami (zakładka porównań projektu) — szczegóły w dokumentacjach modułów (np. luki funkcjonalne),
- ekrany raportów spełniają WCAG 2.1 AA — eksport PDF zachowuje kontrasty i strukturę nagłówków.

Przewodnik — dostępność w praktyce

> Dokumentacja P2, Epic 6 — Stories 6-9, 6-10, 6-11 (WCAG 2.1 AA).

1. Co gwarantuje platforma

Cały interfejs TestGEN spełnia **WCAG 2.1 poziom AA**, m.in.:

- pełna obsługa **klawiatury** (Tab, Enter, Escape, strzałki w listach),
- widoczny **fokus** na każdym elemencie interaktywnym,
- **skip-link** „Przejdź do treści" jako pierwszy element strony,
- kontrasty tekstu ≥ 4,5:1 (wysoki kontrast — nawet 7:1+),
- poprawne etykiety pól i komunikaty błędów (tekstowo, nie tylko kolorem),
- działanie przy **zoomie 200%** i szerokości 320 px,
- komunikaty asynchroniczne ogłaszane czytnikom ekranu.

2. Wysoki kontrast

Pasek dostępności (ikona na każdym ekranie) → **Wysoki kontrast**: czarno-biały motyw z kontrastem tekstu co najmniej 7:1 (poziom AAA). Wybór zapamiętuje się w przeglądarce i działa na wszystkich kolejnych sesjach. Jeśli system operacyjny ma włączony tryb podwyższonego kontrastu, TestGEN włączy go automatycznie.

3. Redukcja animacji

Pasek dostępności → **Wyłącz animacje**: przejścia i animacje zostają zastąpione natychmiastowymi zmianami stanu (pomocne przy zawrotach głowy/migrenach). Ustawienie systemowe „redukuj ruch" jest respektowane automatycznie, bez żadnej akcji w aplikacji.

4. Nawigacja klawiaturą — skróty i wzorce

- **Tab / Shift+Tab** — kolejne elementy (kolejność = kolejność wizualna),
- **Enter/Spacja** — aktywacja przycisku/linku,
- **Escape** — zamknięcie okna dialogowego (fokus wraca na element, który je otworzył),
- **strzałki** — poruszanie się po listach, menu i wierszach tabel,
- formularze: błąd pola czytany jest po opuszczeniu pola; lista błędów na górze formularza prowadzi do pól.

5. Czytniki ekranu

Testowane z NVDA na Windows. Wskazówki:

- struktura nagłówków (poziomy H1–H3) odzwierciedla układ ekranu — nawigacja po nagłówkach jest najszybsza,

- landmarki (nawigacja/treść/stopka) dostępne przez skróty regionów,
- zmiany stanu („Analiza zakończona", „Zaznaczono 12 przypadków") są ogłaszane automatycznie — nie trzeba szukać ich wzrokiem,
- paginacja i etykiety tabel czytane po polsku.

6. Zgłaszanie problemów z dostępnością

Bariera dostępności = błąd produktu. Zgłoś przez obsługę (plan Premium ma priorytetowe wsparcie); każde zgłoszenie wchodzi na listę Napraw WCAG i jest weryfikowane testem axe-core po naprawie.

Przewodnik — plany i zakup subskrypcji

> Dokumentacja P2, Epic 7. Ekran: `/billing/plans` (menu „Rozliczenia").

1. Porównanie planów

Ekran **Plany subskrypcji** (`plan-comparison.component`) pokazuje trzy plany z cenami miesięcznymi: Freemium (0 PLN), Basic (30 PLN), Premium (50 PLN). Macierz różnic:

- limity: analizy/miesiąc (10/50/bez limitu), test case'y (50/250/bez limitu), storage (0,5/5/10 GB), pliki na analizę (5/25/50),
- moduły analizy: analiza kodu i generowanie testów — wszystkie plany; dokumentacja, UML, GUI, testy integracyjne, CI/CD — Basic; bezpieczeństwo, rekomendacje bezpieczeństwa, spójność kod↔dokumentacja, luki funkcjonalne — Premium,
- priorytetowe wsparcie — Premium.

Aktualny plan tenanta jest wyróżniony. Ceny są miesięczne — nie ma płatności rocznych.

2. Zakup planu (Owner)

1. Upewnij się, że masz rolę **Owner** (zakup wymaga Ownera; User widzi plany, ale nie kupuje).
2. Na `/billing/plans` kliknij **Wybierz Basic/Premium**.
3. Zostaniesz przekierowany na **Panel Transakcyjny tpay**:
   - BLIK, karta (z 3-D Secure) albo przelew/PBL — metoda wybierasz w panelu tpay,
   - regulamin i polityka prywatności są dostępne na stronach zakupu (linki test-gen.pl).
4. Po płatności wracasz na `/billing/checkout/success` — strona sama odpytuje status transakcji i pokazuje potwierdzenie; aktywnacja planu następuje po webhooku od tpay (zwykle w ciągu sekund).
5. Potwierdzenie zakupu przychodzi też mailem; faktura VAT — patrz 04 — Faktury i dane fakturowe.

Pierwsza płatność BLIK = metoda odnowień

Płacąc BLIK, wyrażasz zgodę na **Płatności Powtarzalne** — tworzony jest alias BLIK, którym odnowienia będą obciążane automatycznie co miesiąc (limit pojedynczej płatności = cena planu). Jeśli Twój bank nie obsługuje tej usługi, system zaproponuje płatność kartą i zapisanie jej jako metody odnowień.

Gdy coś pójdzie nie tak

- Odrzucenie płatności → `/billing/checkout/error` z przyczyną i przyciskiem ponowienia (transakcja lokalna zostaje anulowana — nic nie kosztuje).
- Dłuższe „wisi" statusu → strona sama dopyta tpay o rzeczywisty stan (reconciliation); ostatecznym źródłem prawdy jest webhook.
- Błąd techniczny (502) → spróbuj ponownie za chwilę.

3. Moduły zablokowane planem

Moduły analizy poza planem są widoczne w UI, ale z kłódką i podpisem, który plan je odblokowuje — po wejściu pojawia się komunikat 402 z CTA do porównania planów. Po zakupie wyższego planu moduły odblokowują się bez przeładowania strony. Szczegóły: 05 — Zużycie i powiadomienia.

4. Zmiana planu

Zmiana planu (upgrade/downgrade) przy aktywnej subskrypcji płatnej odbywa się przez ekran **Subskrypcja** — patrz 02 — Zarządzanie subskrypcją. Bezpośredni zakup (`/billing/checkout`) dotyczy tylko przejścia z Freemium na plan płatny.

Przewodnik — zarządzanie subskrypcją

> Dokumentacja P2, Epic 7 — Stories 7.5, 7.7. Ekran: `/billing/subscription` (navbar → „Subskrypcja", rola **Owner**).

1. Szczegóły subskrypcji

Ekran pokazuje: plan z ceną, status, okres rozliczeniowy (start/koniec), metodę płatności (wizerunek karty/aliasu), datę najbliższego obciążenia oraz — gdy liczy się czas — zaplanowaną zmianę planu albo anulowanie.

2. Zmiana planu (FR55)

Przycisk zmiany planu prowadzi przez potwierdzenie (`plan-change-confirm.component`) z **podglądem kwoty** — obliczonej proration:

- **Upgrade** (np. Basic → Premium): dopłata proporcjonalna do pozostałych dni bieżącego okresu. Przy kwocie > 0 następuje przekierowanie na panel tpay (płatność jak przy zakupie); przy kwocie 0,00 (np. ostatni dzień okresu) — zmiana natychmiastowa, bezpłatna. Nowy plan obowiązuje od razu po opłaceniu.
- **Downgrade** (np. Premium → Basic): **bez płatności** — nowy plan zaczyna obowiązywać **na koniec bieżącego okresu**; do tego czasu korzystasz z obecnego planu. Rozliczenie następuje na koniec okresu, bez zwrotów.
- Z Freemium: pełna cena pierwszego miesiąca nowego planu.

Zaplanowany downgrade możesz **odwołać** do końca okresu przyciskiem rezygnacji — subskrypcja wraca do bieżącego planu i odnowienia.

Historia zmian

Sekcja/ekran historii (`subscription/history` w API) pokazuje pełny audyt: typ zmiany, plany z→na, status (zaplanowana / oczekuje na płatność / zakończona / nieudana / odwołana), kwotę i szczegóły proration. Nieudany upgrade (odrzucona płatność) nie zmienia planu — rekord z statusem „nieudana" + email wyjaśniający.

3. Anulowanie subskrypcji

1. Na `/billing/subscription` kliknij **Anuluj subskrypcję**.
2. Dialog potwierdzenia (`cancel-confirm-dialog.component`) wyjaśnia konsekwencje: dostęp do planu **do końca opłaconego okresu**, potem automatyczny powrót do Freemium (limit 10 analiz/mies., 0,5 GB…).
3. Potwierdzenie jest operacją lokalną — karta/alias nie są obciążane, żadne wywołanie do tpay nie następuje.

Anulowanie przy zaplanowanym downgrade'u: działa jak zwykle — na koniec okresu obowiązuje **plan niższy** z downgrade'u (nie Freemium). Anulowanie na Freemium → 409 (nie ma czego anulować); ponowne anulowanie → 409.

4. Reaktywacja

Przed końcem okresu po anulowaniu możesz kliknąć **Reaktywuj**: zdjęcie flagi anulowania, przywrócenie najbliższego obciążenia na koniec bieżącego okresu — kontynuacja bez przerwy i bez ponownej płatności. Reaktywacja przy oczekującej zmianie planu → 409 (najpierw rozstrzygnij zmianę: odwołaj ją lub zaczekaj na jej wykonanie).

5. Nieudane odnowienie (dunning)

Gdy odnowienie nie może obciążyć metody płatności:

1. System ponawia próby w kolejnych dniach (`dunning`); na bieżąco informuje mailem.
2. Po wyczerpaniu prób: downgrade do **Freemium** + **7 dni okresu łaski (grace)** z zachowaniem limitów płatnego planu — czas na aktualizację metody płatności (patrz 03 — Metody płatności).
3. Po upływie grace obowiązują limity Freemium. Konto ani dane nie są blokowane; powrót na plan płatny = normalny zakup.

Najczęstsze przyczyny: wygasła karta, przekroczony limit aliasu BLIK (12 płatności — po jego wyczerpaniu zapłać ponownie, co utworzy nowy alias), usunięcie metody płatności (system blokuje usunięcie jedynej metody przy aktywnej subskrypcji — 409).

Przewodnik — metody płatności

> Dokumentacja P2, Epic 7 — Stories 7.2, 7.7. Ekran: `/billing/payment-methods` (rola **Owner**).

1. Lista metod

Ekran pokazuje zapisane metody płatności:

- **karty** — wizerunek (ostatnie 4 cyfry, marka, np. `•••• 4242 Visa`), data ważności, status (aktywna / w trakcie rejestracji / usunięta), metoda domyślna odnowień,
- **aliasy BLIK** (Płatności Powtarzalne) — tworzony automatycznie przy pierwszej płatności BLIK; ważność do 730 dni, limit 12 płatności.

Statusy kart odświeżają się na żywo z tpay (krótki cache 5 min); gdy tpay jest chwilowo niedostępny, pokazujemy ostatni znany stan.

2. Dodanie karty (bez obciążenia)

1. Kliknij **Dodaj kartę** — zostaniesz przekierowany do panelu tpay. Karta jest zapisywana **bez żadnej płatności** (rejestracja tokena).
2. Po powrocie (`/billing/payment-methods/return`) strona sama sprawdza potwierdzenie — nowa karta pojawia się na liście jako aktywna.
3. Kartę ustawisz jako domyślną metodę odnowień.

Karta przechowywana jest wyłącznie u tpay (token); TestGEN trzyma token zaszyfrowany i wizerunek karty — nigdy pełny numer.

3. Usuwanie metody

Przycisk usuwania przy metodzie: karta/alias jest usuwany także u tpay.
Ostrzeżenie 409: nie można usunąć jedynej aktywnej metody, gdy subskrypcja odnawia się cyklicznie — najpierw dodaj inną metodę.

4. Która metoda płaci przy odnowieniu

- Jeśli ustawiona jest metoda domyślna (karta) — odnowienie obciąża ją.
- Alias BLIK obciążany jest automatycznie, dopóki jest aktywny i ma niewyczerpany limit płatności.
- Brak aktywnej metody przy odnowieniu → dunning (ponawiane próby, ostatecznie Freemium + 7 dni grace — patrz 02 — Zarządzanie subskrypcją, sekcja 5).

5. Bank nie obsługuje Płatności Powtarzalnych

Przy zakupie BLIK część banków odmawia (błąd `refusenopayid`) — wtedy system podpowie: zapłać **kartą** i **zapisz ją jako metodę płatności**; odnowienia będą obciążać kartę. Płatność jednorazowa BLIK nadal działa — bez funkcji automatycznego odnowienia.

Przewodnik — faktury i dane fakturowe

> Dokumentacja P2, Epic 7 — Story 7.6. Ekrany: `/billing/invoices` (Owner i User), `/billing/invoice-data` (Owner).

1. Faktury

Po **każdej opłaconej płatności** (zakup, upgrade, odnowienie) system automatycznie wystawia fakturę VAT i wysyła ją mailem do Ownera z PDF w załączniku. Lista faktur (`/billing/invoices`) zawiera: numer (`FV/rok/miesiąc/sekwencja`), datę wystawienia, kwotę brutto, metodę płatności i status. Przycisk pobrania zwraca PDF.

Faktura zawiera: numer, daty wystawienia i sprzedaży, dane sprzedawcy (TestGEN) i nabywcy (snapshot danych fakturowych z chwili wystawienia), pozycję z opisem planu i okresem, kwoty netto/VAT 23%/brutto w PLN.

2. Dane fakturowe (Owner)

Ekran `/billing/invoice-data` pozwala wpisać dane do faktur: nazwa firmy, NIP, adres. Przy pierwszym wejściu formularz inicjalizuje się danymi profilu tenanta.

Ważne: zmiana danych dotyczy wyłącznie **następnych** faktur — wystawione faktury przechowują snapshot danych z chwili wystawienia (wymóg księgowy). Chcesz poprawić dane na już wystawionej fakturze — skontaktuj się z obsługą (faktura korygująca po stronie księgowości).

3. Kiedy faktura może nie przyjść mailem?

Faktura powstaje w tle po webhooku płatności — email z PDF może dotrzeć z krótkim opóźnieniem (do kilku minut). Email nie dotarł, a płatność się udała? PDF jest zawsze dostępny na liście faktur — wysyłka mailowa jest tylko dodatkiem, nie warunkiem wystawienia.

4. History transakcji

Ekran `/billing/transactions` (Owner i User) — pełna historia płatności: data, kwota, plan, status (np. oczekująca/zakończona/nieudana), metoda. Filtry statusu i zakresu dat + paginacja. Tu sprawdzisz też płatności, które nie powiodły się (np. odrzucony upgrade) — z kwotą i statusem.

Przewodnik — zużycie zasobów i powiadomienia

> Dokumentacja P2, Epic 7 — Stories 7.9, 7.11. Ekran: `/billing/usage` (wszyscy użytkownicy tenanta).

1. Dashboard zużycia

Ekran pokazuje karty liczników z paskami postępu:

Licznik	Co mierzy	Limit Freemium/Basic/Premium
Analizy	analizy uruchomione w bieżącym okresie	10 / 50 / bez limitu
Test case'y	wygenerowane przypadki testowe	50 / 250 / bez limitu
Storage	miejsce na pliki	0,5 / 5 / 10 GB
Pliki na analizę	pliki w jednej analizie	5 / 25 / 50

Na każdej karcie: zużycie, limit, procent, dni do końca okresu rozliczeniowego. Rozwinięcie karty pokazuje **wykres historii** dziennego zużycia (natywny wykres SVG).

2. Prognoza wyczerpania limitu

Gdy system ma co najmniej 7 dni danych o Twoim zużyciu, na karcie pojawia się **ostrzeżenie prognozy**: przy obecnej średniej dziennego zużycia limit zostanie wyczerpany **przed końcem okresu rozliczeniowego** (z informacją, na który dzień). To sygnał, by zawczasu podnieść plan — upgrade działa natychmiast (patrz 02 — Zarządzanie subskrypcją).

3. Powiadomienia o progach

Gdy zużycie któregoś licznika przekroczy **80%**, **90%** lub **100%** limitu:

- **w aplikacji** — dzwonek w navbarze z licznikiem nieprzeczytanych; lista powiadomień z akcją „oznacz wszystkie jako przeczytane",
- **mailem** — jeśli nie wyłączysz tego kanału w ustawieniach profilu (preferencje kanałów: in-app / email).

Każdy próg informuje maksymalnie raz w okresie rozliczeniowym — bez spamowania przy kolejnych godzinowych sprawdzeniach.

4. Co się dzieje po wyczerpaniu limitu

- Liczniki analiz/test case'ów/plików: kolejna operacja zwraca komunikat o wyczerpanym limicie (HTTP 402/429) z podpowiedzią zmiany planu lub odczekania do końca okresu. Liczniki **resetują się** na początku nowego okresu rozliczeniowego.
- Storage: upload nowych plików zablokowany do zwolnienia miejsca (usunięcie starych analiz) lub zmiany planu.
- Zużycie liczników nie jest cofane po downgrade — obowiązują limity nowego planu od momentu jego wejścia w życie.

5. Widoczność

Dashboard zużycia i powiadomienia dotyczą **całego tenanta** (limity są per firma, nie per użytkownik) — każdy użytkownik widzi te same wartości.

FAQ — moduł billingowy

> Dokumentacja P2, Epic 7.

Płatności

Czy mogę płacić rocznie? Nie. Jedyny cykl to miesięczny (30/50 PLN), decyzja produktowa z 2026-08-28.

Jakie metody płatności obsługuje tpay w TestGEN? BLIK (z Płatnościami Powtarzalnymi), karta (3-D Secure, z możliwością zapisania) oraz przelew/PBL — wybór następuje w Panelu Transakcyjnym tpay.

Czy muszę zapisywać kartę? Nie — ale bez zapisanej metody (karta lub aktywny alias BLIK) odnowienie miesięczne nie powiedzie się i po dunningu (7 dni łaski) plan wróci do Freemium.

Płacę BLIK, ale mój bank odmawia. Twój bank nie obsługuje Płatności Powtarzalnych BLIK. Zapłać kartą i zapisz ją jako metodę — odnowienia będą obciążać kartę.

Dlaczego nie mogę kupić planu, mając już Basic? Z aktywnym planem płatnym zakup jest zablokowany (409) — zmiany planu dokonuje się przez ekran Subskrypcja (upgrade z dopłatą proporcjonalną albo downgrade na koniec okresu).

Subskrypcja

Co się dzieje po anulowaniu? Dostęp do planu do końca opłaconego okresu, potem automatyczny Freemium. Bez utraty danych i bez blokady konta. Do końca okresu możesz reaktywować.

Czy downgrade zwraca pieniądze? Nie — downgrade obowiązuje od końca bieżącego okresu, rozliczenie kończy się z bieżącym okresem (bez zwrotów).

Zaplanowałem downgrade — mogę go odwołać? Tak, do końca bieżącego okresu (przycisk rezygnacji na ekranie subskrypcji).

Nie pomyśliła płatność przy odnowieniu — co teraz? System ponawia próby w kolejnych dniach; po ich wyczerpaniu: Freemium

- 7 dni łaski z limitami płatnego planu. W tym czasie zaktualizuj metodę płatności albo kup plan ponownie.

Limity i faktury

Kiedy resetują się limity? Na początku każdego okresu rozliczeniowego (miesięcznego).

Anulowałem analizę — czy licznik wraca? Nie — licznik analiz rośnie o uruchomioną analizę; licznik plików dotyczy plików w analizie.

Kiedy dostanę fakturę? Automatycznie po każdej opłaconej płatności — mailem z PDF (Owner) i na liście faktur w portalu.

Poprawiłem dane fakturowe — czy stara faktura się zmieni? Nie. Wystawione faktury trzymają snapshot danych z chwili wystawienia; nowe dane zadziałają od następnej faktury.

Kto widzi faktury i transakcje? Owner i User — lista faktur oraz historia transakcji. Zakupy, zmiany planu, dane fakturowe i metody płatności — wyłącznie Owner.

Przewodnik użytkownika — uruchamianie skanu

1. Typy skanów

scan_type	Co analizuje
SAST	kod źródłowy (reguły wzorców)
TAINT	przepływ danych source→sink
DEPENDENCY	zależności / CVE / SBOM
CONFIG	Docker / Kubernetes / Terraform
DAST	uruchomiona aplikacja (OWASP ZAP)

2. Uruchomienie skanu (API)

```
POST /projects/{project_id}/security-scan
```

Body (`SecurityScanRequest`):

```
{
  "scan_type": "SAST",
  "target_url": null,              // wymagany dla DAST
  "auth_config": null              // opcjonalny dla DAST
}
```

Odpowiedź: `202 Accepted` z `scan_id`, `task_id`, `status`. Skan działa asynchronicznie (Celery) na deterministycznych silnikach — offline, z powtarzalnymi wynikami.

3. Śledzenie statusu

```
GET /projects/{project_id}/security-scans              # lista skanów
GET /projects/{project_id}/security-scans/{scan_id}    # status + severity
```

Status przechodzi: PENDING → RUNNING → COMPLETED / FAILED.

4. Skan DAST — wymagania

DAST wymaga uruchomionej aplikacji pod `target_url`. Dla chronionych ścieżek podaj `auth_config` (form/token/basic) — patrz autentykacja DAST.

5. SBOM (tylko DEPENDENCY)

```
GET /projects/{project_id}/security-scans/{scan_id}/sbom?
sbom_format=cyclonedx
```

Format: `cyclonedx` lub `spdx`.

6. Dostęp z UI

Skany uruchamiasz i przeglądasz z poziomu widoku projektu (sekcja bezpieczeństwo). Wyniki agregowane są w dashboardzie bezpieczeństwa.

Przewodnik użytkownika — interpretacja wyników

1. Pola znaleziska (SecurityFinding)

Pole	Znaczenie
`vulnerability_type`	typ podatności (np. sql_injection)
`severity`	CRITICAL / HIGH / MEDIUM / LOW / INFO
`cwe_id`	identyfikator CWE
`owasp_category`	kategoria OWASP Top 10 (2021)
`file_path` / `line_start`	lokalizacja
`confidence`	zaufanie 0.0–1.0
`source`	silnik, który wykrył znalezisko
`cvss_score` / `cvss_vector`	ocena CVSS v3.1
`priority`	priorytet naprawy (1 = najwyższy)
`status`	OPEN / ACKNOWLEDGED / FIXED / FALSE_POSITIVE

2. Severity vs CVSS

Severity wyrównywane jest z pasmem CVSS (bierze wyższą wartość):

CVSS	Severity
9.0–10.0	critical
7.0–8.9	high
4.0–6.9	medium
0.1–3.9	low
0.0	info

3. Confidence — jak czytać

Confidence bazuje na narzędziu (DEPENDENCY 0.90 > DAST 0.85 > TAINT 0.80 > CONFIG 0.75 > SAST 0.70), z premią za:

- wiele źródeł wykrywających to samo (+do 0.15),
- korelację SAST↔DAST (+0.20) lub SAST↔TAINT (+0.10).

Wysoki confidence + wiele źródeł = silny sygnał (mała szansa na false positive).

4. Raport zagregowany

```
GET /projects/{project_id}/security-report
```

Zwraca zdeduplikowane, skorelowane znaleziska + `dedup_count` i `correlation_count`.

5. Dashboard

```
GET /projects/{project_id}/security-dashboard
```

Zawiera rozkład CVSS, top 10 znalezisk i sekwencję napraw — patrz priorytetyzacja.

6. Filtrowanie znalezisk

```
GET /projects/{project_id}/security-findings?
severity=HIGH&scan_type=SAST&cwe_id=CWE-89
```

Filtry: `scan_id`, `severity`, `scan_type`, `status`, `vulnerability_type`, `file_path`, `cwe_id`, `page`, `page_size`.

Przewodnik użytkownika — priorytetyzacja napraw

1. Jak liczony jest priorytet

Risk engine sortuje znaleziska po kluczu:

```
(-severity, -confidence, effort, -cvss_score)
```

i przypisuje `priority` (1 = najwyższy). Oznacza to: najpierw najpoważniejsze, najpewniejsze i najtańsze w naprawie podatności.

2. Szacowanie wysiłku (effort)

Mapowanie CWE → effort (`_EFFORT_BY_CWE`):

Effort	Przykładowe CWE	Charakter naprawy
low	89, 79, 352, 22, 798, 1104, 611	sanityzacja / aktualizacja
medium	78, 94, 502, 668, 918, 200	refaktoryzacja
high	639, 862	przeprojektowanie autoryzacji

3. Eskalacja severity

Risk engine podbija severity o 1 poziom, gdy:

- znalezisko ma źródła SAST **i** DAST (potwierdzenie dynamiczne) → „Potwierdzona dynamicznie”,
- zależność (DEPENDENCY) ma CVSS ≥ 9.0 → „Zależność z publicznym exploitem”.

Powód eskalacji zapisany jest w `escalation_reason` .

4. Sekwencja napraw (dashboard)

```
GET /projects/{project_id}/security-dashboard
```

`repair_sequence` to uporządkowana lista kroków naprawczych — realizuj od priorytetu 1. Strategia: najpierw critical/high z niskim effort („szybkie zwycięstwa”), potem kosztowne przeprojektowania.

5. Praca z patchami zbiorczymi

Przy naprawie wielu znalezisk w jednym pliku sprawdź konflikty:

```
GET /projects/{project_id}/security-patches/conflicts
POST /projects/{project_id}/security-patches/batch
```

Konflikty oznaczane są jako `critical` (te same linie) lub `warning` (sąsiadujące) — patrz rekomendacje i patche.

6. Triage statusu

```
PATCH /projects/{project_id}/security-findings/{finding_id}/status
```

Ustaw `ACKNOWLEDGED`, `FIXED` lub `FALSE_POSITIVE`, aby utrzymać czysty backlog.

Przewodnik użytkownika — rekomendacje, patche i PoC

1. Rekomendacje naprawcze

```
POST /projects/{project_id}/security-findings/{finding_id}/recommendation
```

`RecommendationEngine` zwraca: opis problemu, opis ryzyka, opis naprawy, naprawiony snippet kodu i unified diff. Rekomendacje pochodzą z szablonów (`recommendation_templates/`) per typ podatności i język.

2. Patche (git apply)

```
POST /projects/{project_id}/security-findings/{finding_id}/generate-patch
GET  /projects/{project_id}/security-findings/{finding_id}/patch
GET  /projects/{project_id}/security-findings/{finding_id}/patch/preview
GET  /projects/{project_id}/security-findings/{finding_id}/patch/download
```

`PatchGenerator` tworzy unified diff (`diff_generator.py`), waliduje go względem kodu źródłowego (`is_valid`) i nadaje nazwę `fix_{typ}_{plik}_{id}.patch`. `preview` pokazuje naprawę bez zapisu.

3. Patche zbiorcze i konflikty

```
POST /projects/{project_id}/security-patches/batch
GET  /projects/{project_id}/security-patches/conflicts
```

Wykrywanie konfliktów grupuje znaleziska po pliku i porównuje zakresy linii:

- `critical` — nakładające się linie,
- `warning` — linie sąsiadujące (koniec+1 == początek).

4. Proof of Concept (PoC)

```
POST /projects/{project_id}/security-findings/{finding_id}/generate-poc
GET  /projects/{project_id}/security-findings/{finding_id}/poc
```

`PoCGenerator` zwraca: payload, kroki reprodukcji, oczekiwany efekt, flagę `is_safe` i disclaimer.

5. Bezpieczeństwo PoC (poc_safety)

Każdy PoC przechodzi przez `validate_text_safety()`. Blokowane są wzorce destrukcyjne, m.in.: `DROP TABLE/DATABASE`, `DELETE FROM`, `TRUNCATE`, `UPDATE ... SET`, `INSERT INTO`, `rm -rf`, `format c:`, `mkfs`, `shutdown/reboot`, fork bomb,

`wget` / `curl -O`, `curl | sh`.

PoC z wykrytym wzorcem destrukcyjnym ma `is_safe = False` i listę `blocked_reasons` — NIE jest udostępniany do wykonania.

Disclaimer: „PoC służy wyłącznie do weryfikacji podatności. Użycie na systemach bez autoryzacji jest zabronione."

Przewodnik użytkownika — FAQ bezpieczeństwa

Pytania ogólne

P: Czy wyniki skanu są powtarzalne? O: Tak. Silniki są deterministyczne (reguły i analiza przepływu danych) — te same dane wejściowe dają ten sam wynik, bez zależności od usług zewnętrznych.

P: Czy skan wymaga dostępu do internetu? O: SAST, TAINT, CONFIG i DEPENDENCY (z lokalną bazą CVE) działają offline. DAST wymaga uruchomionej aplikacji do testowania.

P: Skąd biorą się dane CVE? O: Z NVD, OSV.dev i GitHub Advisory, synchronizowanych miesięcznie do lokalnej tabeli `cve_database` (patrz źródła danych).

Jakość wyników

P: Mam false positive — co zrobić? O: Ustaw status znaleziska na `FALSE_POSITIVE` (`PATCH .../security-findings/{id}/status`). Reguły są strojone tak, by utrzymać FP < 30% na korpusie golden (Story 4.14).

P: Dlaczego to samo znalezisko ma różny confidence? O: Confidence zależy od narzędzia i potwierdzeń. Znalezisko wykryte przez wiele narzędzi lub potwierdzone dynamicznie (SAST↔DAST) ma wyższy confidence.

P: Co oznacza eskalacja severity? O: Risk engine podnosi severity, gdy podatność jest potwierdzona dynamicznie lub dotyczy zależności z publicznym exploitem (CVSS ≥ 9.0). Powód w `escalation_reason`.

Naprawy

P: Czy mogę zastosować patch automatycznie? O: Patch to unified diff walidowany względem kodu (`is_valid`). Pobierz przez `/patch/download` i zastosuj `git apply`. Przy wielu naprawach w jednym pliku sprawdź konflikty (`/security-patches/conflicts`).

P: Czy PoC jest bezpieczny do uruchomienia? O: PoC przechodzi przez filtr `poc_safety` blokujący operacje destrukcyjne. PoC z `is_safe=False` nie jest udostępniany. Uruchamiaj wyłącznie na autoryzowanych systemach.

Troubleshooting

P: Skan ma status FAILED. O: Sprawdź `error_message` w statusie skanu. Dla DAST najczęstsze przyczyny to niedostępny `target_url` lub timeout (limit 30 min).

P: Skan DAST nie pokrywa chronionych ścieżek. O: Podaj `auth_config` (form/token/basic), aby skaner działał jako uwierzytelniony użytkownik.

P: Aktualizacja CVE nie zadziałała. O: Sprawdź `CVEUpdateLog` i `SourceStatus` per źródło. Zadanie ma 3 retry z backoffem; błąd jednego źródła nie przerywa całego przebiegu.

Uruchamianie analizy luk

> Przewodnik — uruchamianie analizy luk funkcjonalnych.

Wymagania wstępne

1. Projekt z co najmniej jednym dokumentem (SRS, SDD, user stories) — patrz moduł dokumentów.
2. Projekt z przeskanowanym kodem źródłowym — patrz moduł analizy.
3. Uprawnienia `write:projects` w tenant.

Krok 1: Otwórz Project Detail

1. Zaloguj się do TestGEN.
2. Wybierz projekt z listy.
3. W sidebarze kliknij **Projekty** → **{twój projekt}** — otworzy się Project Detail z kafelkami akcji.

Krok 2: Uruchom analizę luk

1. W sekcji akcji kliknij przycisk **"Luki funkcjonalne"**.
2. Otworzy się widok analizy luk (route: `/projects/:id/gap-analysis`).
3. Kliknij przycisk **"Uruchom analizę"** w prawym górnym rogu.
4. W oknie dialogowym wybierz:
   - **Dokumenty**: lista dokumentów SRS/SDD do analizy (auto-wykryte).
5. Kliknij **Potwierdź**.

Krok 3: Monitoruj postęp

- Po uruchomieniu otrzymasz `task_id` (HTTP 202 Accepted).
- W nagłówku widoku pojawi się status: `pending` → `running` → `success` / `failed`.
- Średni czas analizy: < 5 s (małe projekty) / 30 s (duże).
- Po zakończeniu tabela findings odświeża się automatycznie.

Krok 4: Interpretuj status

Status	Znaczenie	Akcja
`pending`	Zadanie w kolejce	Czekaj
`running`	Analiza w toku	Czekaj
`success`	Ukończono pomyślnie	Tabela findings pokazuje wyniki
`failed`	Błąd	Sprawdź logi w `/projects/{id}/tasks`

Analiza działa na deterministycznych algorytmach dopasowania — wyniki są powtarzalne, a analiza nie wymaga dostępu do usług zewnętrznych. Nadaje się do CI i szybkich iteracji.

Krok 5: Automatyzacja (opcjonalne)

- **Celery beat**: skonfiguruj harmonogram w `app/workers/scheduler.py` aby uruchamiać analizę np. co noc.
- **API**: `POST /api/v1/projects/{project_id}/gap-analysis` z body `{"document_ids": [...]}`.

Rozwiązywanie problemów

"Brak wymagań"

Analiza wymaga `AnalysisResult(module="requirements")`. Najpierw uruchom analizę dokumentu w module analizy.

"Brak bytów kodu"

Analiza wymaga `AnalysisResult(module="logic_structures")`. Najpierw uruchom skanowanie kodu.

Następne kroki

- 02 — Interpretacja wyników
- 03 — Compliance i priorytetyzacja

Interpretacja wyników analizy luk

> Przewodnik P2 — jak czytać tabela findings i podejmować decyzje.

Widok tabeli findings

Po uruchomieniu analizy w `projects/:id/gap-analysis` pojawi się tabela:

Kolumna	Opis
Typ luki	Chip koloru: czerwony (missing), pomarańczowy (different), żółty (extra), fioletowy (inconsistent)
Wymaganie	ID wymagań (np. REQ-ANL-004)
Treść wymagania	Skrócony tekst wymagania
Byt kodu	Nazwa funkcji / metody / klasy
Lokalizacja	`file_path:line`
Score	0–100% (jak silne dopasowanie)
Metoda	lev, jw, sem, combined
Severity	info / warning / critical

Filtry

Górny panel filtrów:

- **Typ luki** (MultiSelect) — zawęź do wybranych typów
- **Severity** (MultiSelect) — critical / warning / info
- **Min score** (Slider 0–100) — ukryj findingi poniżej progu
- **Metoda** (Select) — combined / lev / jw / sem

Klasyfikacje luk — kiedy się pojawiają

`missing` (czerwony)

Co znaczy: wymaganie istnieje, ale kod nie zawiera odpowiadającego bytu.

Decyzja:

- Priorytet high/critical → zaplanuj sprint implementation
- Priorytet medium → dodaj do backlog
- Priorytet low → discuss z PO (może nieaktualne wymaganie)

Akcja: Kliknij finding → "Generuj skeleton" (jeśli dostępny) → skopiuj kod do repo → implementuj według spec.

`different` (pomarańczowy)

Co znaczy: kod częściowo spełnia wymaganie (score 30–70).

Decyzja:

- Przeczytaj `context` aby zobaczyć, czego dokładnie brakuje
- Jeśli kod jest niekompletny → zaplanuj rozszerzenie
- Jeśli kod jest poprawny, ale nazwa myląca → rename lub popraw docstring

Akcja: Kliknij finding → rozwiń panel "Context" → przejrzyj excerpt.

`extra` (żółty)

Co znaczy: byt kodu nie ma odpowiadającego wymagania.

Decyzja:

- Kod pomocniczy (utils, helpers) → dodaj tag `@internal` lub pomiń
- Kod z feature creep → discuss z PO: dodać wymaganie czy usunąć?
- Dead code → rozważ usunięcie

Akcja: Kliknij finding → "Oznacz jako @internal" (pomija w przyszłych analizach) lub "Dodaj wymaganie" (tworzy draft wymagania).

`inconsistent` (fioletowy)

Co znaczy: ten sam byt kodu jest mapowany do sprzecznych wymagań.

Decyzja:

- **Zawsze critical** — wymaga koordynacji z PO i zespołem
- Przeczytaj oba wymagania — które jest autorytatywne?
- Rozdziel implementację na osobne funkcje, lub ujednolić wymagania

Akcja: Kliknij finding → "Zobacz sprzeczne wymagania" → wybierz dominujące lub utwórz zadanie sync z PO.

Sortowanie i priorytetyzacja

Domyślnie tabela sortuje po:

1. Severity descending (critical → warning → info)
2. Score ascending (najgorsze dopasowanie pierwsze)
3. Requirement ID

Możesz kliknąć nagłówek kolumny aby zmienić sortowanie. Typowe podejście:

1. **Critical + inconsistent** — natychmiast (P1)
2. **Critical + missing** — ten sprint (P1)
3. **Warning + different** — następny sprint (P2)

4. **Info + extra** — techniczny dług (P4)

Eksport

Przycisk **"Eksportuj CSV"** w prawym górnym rogu pobiera wszystkie findings (widoczne po filtrach) jako CSV z kolumnami: `req_id`, `gap_type`, `severity`, `description`, `file_path`, `line_number`, `score`, `match_method`.

Panel podsumowania

Górny panel pokazuje:

- **Liczba findings per typ** (4 chipy z licznikami)
- **Średni score** — ogólna jakość dopasowania
- **Coverage %** — procent wymagań z dobrym dopasowaniem
- **Liczba wymagań** — total / zaimplementowanych / brakujących

Następne kroki

- 03 — Compliance i priorytetyzacja
- 04 — Rekomendacje i skeleton

Compliance i priorytetyzacja

> Przewodnik P2 — compliance scoring, snapshoty, porównanie wersji.

Widok Compliance

Dostęp: Project Detail → **"Compliance"** (route: `/projects/:id/compliance`).

Struktura

1. **Gauge projektu** — okrągły wskaźnik 0–100 z kolorem:
   - 80–100 zielony (dobry)
   - 60–80 żółty (wymaga uwagi)
   - < 60 czerwony (krytyczny)
2. **Tabela modułów** — `ModuleScores` per `file_path`:
   - Sortowanie: po score rosnąco (najgorsze pierwsze) lub po liczbie findings
   - Każdy wiersz: file_path, score, liczba findings, breakdown per typ
   - Klik → drill-down do funkcji w module
3. **Drill-down funkcji** — `FunctionScores` per `code_entity_id`:
   - Jak wyżej + nazwa funkcji

Interpretacja score

Score	Interpretacja	Akcja
90–100	Świetnie — drobne ostrzeżenia	Utrzymać
70–90	Dobrze, ale są luki	Zaplanować w następnym sprincie
50–70	Mediokre — znaczące braki	Sprint debt / hardening
30–50	Krytyczne — dużo luk	Stop-feature, focus na compliance
0–30	Projects in crisis	Eskalacja, audit

Breakdown per typ

Dla każdego poziomu (projekt / moduł / funkcja) pokazany jest breakdown:

```
{missing: 4, different: 2, extra: 1, inconsistent: 0}
```

Skup się najpierw na:

1. `missing` z high priority → największy wpływ na biznes
2. `inconsistent` → ryzyko logiczne

3. `different` → często drobne poprawki
4. `extra` → cleanup, niski priorytet

Eksport CSV

Przycisk **"Eksportuj CSV"** pobiera raport:

- Wiersz nagłówka projektu (nazwa, score, data)
- Wszystkie ModuleScores
- Wszystkie FunctionScores
- Format kompatybilny z Excel / LibreOffice

Snapshoty i porównanie wersji

Kiedy powstają snapshoty

- Ręcznie: kliknij **"Utwórz snapshot"** w widoku Compliance
- Automatycznie: po każdej zakończonej analizie luk (post-hook)

Porównanie v1 ↔ v2

Dostęp: route `/projects/:id/gap-analysis/compare?v1={snap1}&v2={snap2}`.

Widok diff:

- **Fixed** (zielony) — luki rozwiązane między wersjami
- **New** (czerwony) — luki dodane od v1
- **Unchanged** (szary) — bez zmian
- **Worsened** (pomarańczowy) — score się obniżył

Trend

Route `/projects/:id/compliance/trend` — wykres liniowy:

- Oś X: data snapshotu
- Oś Y: compliance score 0–100
- Pozwala ocenić długoterminowy trend poprawy/degresji

Priorytetyzacja napraw

Heurystyka

1. Posortuj findings po:
   - Severity (critical > warning > info)
   - Priorytet wymagania (high > medium > low)
   - Wpływ na compliance penalty (missing+critical = 15 punktów!)
2. Wybierz top 20% findings (zasadą Pareto) do tego sprintu.
3. Po naprawie uruchom ponownie analizę — sprawdź, czy score rośnie.

Mapowanie severity → akcja

Severity	gap_type	Akcja	Kiedy
critical	missing	Implementacja	Ten sprint
critical	inconsistent	Sync z PO + rozdzielenie	Ten tydzień
critical	different	Rewizja implementacji	Ten sprint
warning	missing	Implementacja	Następny sprint
warning	different	Poprawki jakościowe	Backlog
info	extra	Cleanup	Techniczny dług

Przykład planu sprintu

Projekt o compliance 62 (30 findings):

- Sprint 1: 5× missing+critical (P1) → oczekiwany score po naprawie: 62 + 75 = 87
- Sprint 2: 3× inconsistent (P1) → score: 87 + 30 = 100 (teoretycznie)
- Sprint 3: 8× different+warning (P2) → utrzymanie 100
- Sprint 4: 14× extra+info (P4) → cleanup

Następne kroki

- 04 — Rekomendacje i skeleton
- 02 — Interpretacja wyników

Rekomendacje i skeleton generator

> Przewodnik P2 — praca z rekomendacjami i generowanie skeleton kodu.

Widok Rekomendacji

Dostęp: Project Detail → **"Rekomendacje"** (route: `/projects/:id/recommendations`).

Struktura tabeli

Kolumna	Opis
Priority badge	P1 (czerwony), P2 (pomarańczowy), P3 (żółty), P4 (niebieski)
Gap	ID luky powiązanej + typ (chip)
Sugerowana akcja	Tekst rekomendacji
Confidence	0–100% + ostrzeżenie jeśli < 50%
Effort	S / M / L
Source	pochodzenie rekomendacji (badge)
Status	open / accepted / applied / dismissed

Generowanie rekomendacji

Jeśli lista jest pusta lub chcesz odświeżyć:

1. Kliknij **"Generuj rekomendacje"** w prawym górnym rogu.
2. Potwierdź — backend uruchamia `RecommendationEngine` i persystuje wyniki.
3. Tabela odświeża się automatycznie.

Akcje na rekomendacji

Kliknij wiersz → panel boczny pokazuje szczegóły + akcje:

- **"Oznacz jako zaakceptowana"** — zmienia status na `accepted` (zespół zatwierdził)
- **"Oznacz jako zastosowana"** — zmienia na `applied` (implementacja ukończona)
- **"Odrzuć"** — status `dismissed` + pole `reason` (wymagane)
- **"Generuj skeleton"** — tworzy plik skeleton (jeśli finding type=missing)
- **"Sync do Jira/ADO"** — tworzy issue w zewnętrznym trackerze

Skeleton generator

Kiedy dostępny

Skeleton jest dostępny **tylko** dla findings typu `missing` (gdy wymagane jest stworzenie nowej funkcji/klasy).

Generowanie

1. W tabeli findings lub w panelu rekomendacji kliknij **"Generuj skeleton"**.
2. Modal pokazuje:
   - Język (auto-detected z codebase'u projektu)
   - Nazwę funkcji (auto-pochodząca z nazwy wymagania)
   - Podgląd kodu z syntax highlighting
3. Akcje:
   - **"Kopiuj do schowka"** — skopiuj kod
   - **"Pobierz plik"** — pobierz `.py` / `.ts` / `.java` / `.cs`
   - **"Otwórz w edytorze"** — wskaż miejsce docelowe

Przykład

Dla wymagania `REQ-ANL-004: "System must export an analytics report to PDF"`:

```
"""TODO: System must export an analytics report to PDF

Requirement ID: REQ-ANL-004
Priority: low
Generated from gap analysis — do not edit this header.
"""


def export_pdf_report(data):
    """System must export an analytics report to PDF."""
    raise NotImplementedError(
        "Skeleton for REQ-ANL-004 — implement per spec."
    )
```

Walidacja

Skeleton jest walidowany przez tree-sitter przed zwróceniem. Gdyby szablon wygenerował niepoprawną składnię, backend zwróci HTTP 422 z informacją.

Tryby pracy z rekomendacjami

Workflow 1: Developer samodzielnie

1. Uruchom analizę po implementacji feature'a.
2. Przejrzyj P1 / P2 rekomendacje.

3. Zaakceptuj / zastosuj / odrzuć.
4. Generuj skeleton dla brakujących funkcji.
5. Commit + powtórz.

Workflow 2: Zespół z PO

1. Po każdym sprincie uruchom analizę luk.
2. Wyeksportuj rekomendacje do CSV.
3. Na planningu omów P1 — wymagają decyzji biznesowej.
4. Zaplanuj P2 / P3 do sprintu.
5. Oznacz w UI statusy.

Workflow 3: Automatyzacja z Jira

1. Skonfiguruj integrację Jira (patrz 05 — Integracje).
2. Po analizie kliknij **"Sync do Jira"** — wszystkie P1 tworzą issues.
3. Zespół pracuje w Jira — zmiany statusów w TestGEN po ponownym sync.

Konfidencja rekomendacji

- **≥ 0.8** — wysoka ufność (czerwony/zielony check)
- **0.5–0.8** — średnia (żółty)
- **< 0.5** — niska, ostrzeżenie (czerwony alert)

Rekomendacje z niską ufnością wymagają weryfikacji człowieka przed wprowadzeniem zmian.

Następne kroki

- 05 — Integracje z trackerami
- 03 — Compliance

Integracje z trackerami (Jira, Azure DevOps)

> Przewodnik P2 — konfiguracja i sync znalezisk do zewnętrznych trackerów.

Dostęp

Ustawienia → **Integracje** (route: `/projects/:id/integrations`).

Konfiguracja Jira

Wymagania

- Konto Jira Cloud z uprawnieniami "Create Issues"
- API token wygenerowany na https://id.atlassian.com/manage-profile/security/api-tokens
- Email używany do logowania do Jira
- Project Key docelowy (np. ENG, DEV, PROJ)

Kroki konfiguracji

1. Otwórz **Integrations** → zakładka **Jira**.
2. Wypełnij formularz:
   - **Base URL**: np. `https://twoja-org.atlassian.net`
   - **Email**: twój email do Jira
   - **API Token**: wklej token
   - **Project Key**: np. `ENG`
   - **Issue Type**: `Task` (domyślnie), `Story`, `Bug`
3. Kliknij **Test connection** — backend wywoła `/rest/api/3/serverinfo`.
4. Po sukcesie kliknij **Zapisz konfigurację**.

Walidacja

Backend sprawdza:

- URL jest HTTPS
- API token nie jest pusty
- Poświadczenia szyfrowane Fernet (at-rest)
- Test połączenia zwraca 200 z Jira

Konfiguracja Azure DevOps

Wymagania

- Organizacja Azure DevOps z projekt
- PAT (Personal Access Token) z scope `Work Items: Read & Write`
- Nazwa organizacji i projektu

Kroki konfiguracji

1. Otwórz **Integrations** → zakładka **Azure DevOps**.
2. Wypełnij formularz:
   - **Organization URL**: `https://dev.azure.com/twoja-org`
   - **Project Name**: np. `MyProject`
   - **PAT**: wklej Personal Access Token
   - **Work Item Type**: `Task` (domyślnie), `User Story`, `Bug`
3. Kliknij **Test connection** — backend wywoła `/_apis/projects/{project}`.
4. Po sukcesie kliknij **Zapisz konfigurację**.

Sync znalezisk

Pojedyncze finding

1. W tabeli gap-findings kliknij finding.
2. W panelu bocznym kliknij **"Sync do {provider}"**.
3. Backend tworzy issue / work item i zapisuje `external_id`.

Bulk sync

1. Zaznacz wiele findings w tabeli (checkbox).
2. Kliknij **"Sync zaznaczonych"** w toolbarze.
3. Wybierz providera (Jira / ADO).
4. Backend uruchamia asynchroniczny task sync.
5. Progress widoczny w toolbarze.

Sync wszystkie P1

1. W tabeli recommendations kliknij **"Sync P1 do Jira"**.
2. Wszystkie rekomendacje o priority P1 zostaną zgrane.
3. Warto domyślnie filtrować po critical, aby uniknąć spamu w trackerze.

Mapowanie pól

Jira

Pole TestGEN	Pole Jira
Severity: critical	Priority: Highest
Severity: warning	Priority: High
Severity: info	Priority: Medium
Gap type	Labels: testgen, gap-analysis, {gap_type}
Requirement ID	Description (formatted text)
Code reference	Description (formatted text)
Match score	Custom field (jeśli skonfigurowane)

Azure DevOps

Pole TestGEN	Pole ADO
Severity: critical	Priority: 1
Severity: warning	Priority: 2
Severity: info	Priority: 3
Gap type	Tags: testgen; gap-analysis; {gap_type}
Requirement ID	Description (HTML)
Code reference	Description (HTML)

Status sync

Każdy sync tworzy wpis w `integration_sync_logs`:

Status	Znaczenie
`pending`	W kolejce
`running`	W trakcie
`success`	Wszystko OK
`partial`	Część findingów miała błąd (np. rate limit)
`failed`	Cały sync się nie udał (np. poświadczenia wygasły)

Historia logów dostępna w **Sync Logs** poniżej formularza konfiguracji.

Deduplikacja

Backend nie tworzy duplikatów issues. Pole `external_ids` w logu sync mapuje `gap_finding_id → external_issue_key`. Ponowny sync pomija findings, które już mają external_key, chyba że użytkownik wybierze **"Wymuś ponowne**

utworzenie".

Bezpieczeństwo

- **Szyfrowanie at-rest**: poświadczenia szyfrowane Fernet. Klucz w zmiennej środowiskowej `INTEGRATION_ENCRYPTION_KEY`.
- **RLS**: integracje są widoczne tylko dla członków tenant.
- **Rate limiting**: max 10 syncs / hour / project (zapobieganie pętlom).
- **Audit log**: wszystkie akcje sync są logowane z `user_id`, `timestamp`, `provider`, `synced_count`.

Troubleshooting

"401 Unauthorized"

- Sprawdź ważność API token / PAT.
- Sprawdź czy email ma uprawnienia w docelowym projekcie.

"404 Project not found"

- Sprawdź Project Key / Project Name.
- Sprawdź czy organizacja jest poprawna.

"Rate limit exceeded"

- Poczekaj 5 minut i spróbuj ponownie.
- Zmniejsz liczbę findings na sync (filtruj po P1).

"Field 'priority' not found"

- Nie każdy projekt Jira ma te same priorytety. Skontaktuj się z adminem Jira aby dostosować mapowanie.

Następne kroki

- 04 — Rekomendacje i skeleton
- 06 — FAQ

FAQ — moduł luk funkcjonalnych

Najczęstsze pytania i problemy.

Uruchamianie

Dlaczego moja analiza zwraca 0 findings?

Sprawdź:

1. Czy projekt ma `AnalysisResult(module="requirements")` (parsowanie SRS).
2. Czy projekt ma `AnalysisResult(module="logic_structures")` (skan kodu).
3. Czy dokument i kod są w tym samym projekcie.
4. Czy istnieje jakikolwiek byt kodu w `code_entities` (analiza nie działa na pustym repo).

Jak często uruchamiać analizę?

- **Po każdym merge do main** — szybki feedback.
- **Przed release** — pełna analiza.
- **Sprint planning** — analiza stanu + porównanie z poprzednim snapshotem.

Czy analiza działa offline?

Tak, w pełni — silniki nie zależą od usług zewnętrznych.

Wyniki

Dlaczego finding ma niski score, ale wymaganie jest zaimplementowane?

Niski score combined oznacza, że nazwa funkcji / docstring słabo korelują z tekstem wymagania. Możliwe przyczyny:

- Nazwa funkcji jest zbyt ogólna (`process_data` zamiast `export_pdf_report`)
- Brak docstring lub docstring jest zbyt krótki
- Wymaganie używa innej terminologii niż implementacja

Rozwiązanie: popraw docstring (np. dodaj tekst wymagania), rename funkcji, lub oznacz finding jako `dismissed`.

Dlaczego "extra" findings pokazują pomocnicze funkcje?

Klasyfikator pomija wzorce `test_*`, `__init__`, `conftest`, `setup`, `teardown`, `main`, akcesory `get_*` / `set_*` / `is_*`. Pozostałe funkcje pomocnicze mogą być oflagowane. **Rozwiązanie**:

- Dodaj dekorator `@internal` lub prefix `_` (private)
- Lub dodaj wymaganie w dokumentacji dla tego helpera

Czy mogę zignorować finding na stałe?

Tak — kliknij finding → **"Oznacz jako dismissed"** + podaj powód. Finding pozostanie w bazie z statusem `dismissed` i nie będzie liczone w compliance score.

Compliance

Dlaczego score spadło po dodaniu nowej funkcjonalności?

Dodanie nowego kodu może spowodować:

- Wykrycie nowych luk `extra` (kod bez wymagań)
- Wykrycie `different` dla częściowo pokrywających się wymagań

Rozwiązanie: jeśli kod jest poprawny, dodaj odpowiednie wymagania do dokumentacji.

Czy mogę ręcznie ustawić compliance score?

Nie — score jest wyliczany z findings. Aby go poprawić, napraw findings (zaimplementuj missing, rozwiąż inconsistent, itp.).

Skeleton

Czy skeleton jest gotowy do commit?

Nie — skeleton to **stub** z `NotImplementedError`. Musisz zaimplementować ciało funkcji zgodnie z wymaganiem.

Dlaczego skeleton ma złą nazwę funkcji?

Skeleton używa `_normalize_identifier()` do konwersji tekstu wymagania na nazwę funkcji. Jeśli wymaganie jest długie / złożone, nazwa może być nieoptymalna. **Rozwiązanie**: popraw nazwę w generowanym pliku + dodaj docstring.

Integracje

Czy mogę użyć GitLab / GitHub Issues?

Obecnie wspieramy Jira i Azure DevOps. GitLab i GitHub Issues są na roadmapie.

Czy integracja wymaga admina?

Nie, wymaga tylko uprawnień `write:projects` w TestGEN i uprawnień do tworzenia issues w trackerze docelowym.

Czy mogę mieć osobne integracje per projekt?

Tak — każda integracja jest per `project_id`. Możesz mieć projekt A z Jira i projekt B z Azure DevOps.

Wydajność

Analiza jest wolna (> 60 s)

Sprawdź:

1. Rozmiar projektu (> 500 bytów kodu = dłuższa analiza)
2. Obciążenie kolejki zadań (Celery) w logach

Memory usage wysoki

Comparator ma złożoność O(R×E). Dla 1000 wymagań × 1000 bytów = 1M par. W takich przypadkach:

- Podziel projekt na moduły
- Lub zwiększ RAM serwera backend

Bezpieczeństwo

Gdzie są przechowywane API tokens?

Poświadczenia Jira / ADO są szyfrowane Fernet i persystowane w `integration_configs.encrypted_credentials`. Klucz szyfrowania w zmiennej środowiskowej `INTEGRATION_ENCRYPTION_KEY`.

Kto widzi moje findingi?

Tylko użytkownicy w tym samym tenant (RLS PostgreSQL). Administrator tenant zarządza członkami.

Następne kroki

- Glosariusz
- 01 — Architektura — przegląd

Indeks terminów podręcznika

> Alfabetyczna nawigacja po pojęciach z Rozdziałów 1–17 — każde hasło wskazuje plik i rozdział, w którym temat jest omówiony. Pełne definicje: Glosariusz.

Termin	Gdzie
Analiza bezpieczeństwa	02, rozdz. 7
Analiza dokumentacji	02, rozdz. 5
Analiza kodu źródłowego	02, rozdz. 4
Analiza luk funkcjonalnych (gap analysis)	02, rozdz. 5.6
Analiza UML	02, rozdz. 6
API platformy	03, rozdz. 11
Archiwizacja projektu	02, rozdz. 3.5
Azure DevOps (integracja)	03, rozdz. 10.3
Błędy kodu	02, rozdz. 4.6
BLIK / płatności powtarzalne	04, rozdz. 13.4; billing
CLI (`testgen`)	03, rozdz. 11.7
Compliance (zgodność kodu z dokumentacją)	02, rozdz. 5.6
CVSS	02, rozdz. 7.3
DAST	02, rozdz. 7.1
Dostępność (WCAG 2.1 AA)	03, rozdz. 12
Eksport do TMS (TestRail, Xray, Zephyr)	02, rozdz. 8.6 (plikiem); 03, rozdz. 10.8 (bezpośrednio)
Faktury VAT	04, rozdz. 13.5
Framework testowy (wybór)	02, rozdz. 4.3 i 8.1
Gherkin/BDD	02, rozdz. 6.4
GitHub i GitLab (integracja)	03, rozdz. 10.5
Glosariusz dokumentacji	02, rozdz. 5.5
Import wymagań z Jiry	03, rozdz. 10.10
Jira (integracja)	03, rozdz. 10.2
Karta projektu	02, rozdz. 3.3
Klucze API	03, rozdz. 11.2; 04, rozdz. 14.5
Kod testowy (generowanie)	02, rozdz. 8.5
Kody zapasowe (MFA)	04, rozdz. 14.2
Konfiguracja CI/CD (generator)	03, rozdz. 10.7

Termin	Gdzie
Limit żądań (rate limit)	03, rozdz. 11.3
Limity planów	02, rozdz. 4.1; 04, rozdz. 13.6
Macierz śledzenia (UML/GUI)	02, rozdz. 6.6
Martwy kod	02, rozdz. 4.6
MFA (weryfikacja dwuetapowa)	04, rozdz. 14.2
Niespójności w dokumentacji	02, rozdz. 5.4
OCR (skany PDF, zrzuty GUI)	02, rozdz. 5.1 i 6.3
Patch (poprawka bezpieczeństwa)	02, rozdz. 7.5
Plany (Freemium/Basic/Premium)	01, rozdz. 1.5; 04, rozdz. 13.3
PoC (Proof of Concept)	02, rozdz. 7.5
Podatności (znaleziska bezpieczeństwa)	02, rozdz. 7.2–7.3
Podłączenie repozytorium Git	02, rozdz. 4.1
Powiadomienia (limity, dzwonek)	04, rozdz. 13.6
Preprocessing (języki, frameworki)	02, rozdz. 4.5
Priorytety przypadków testowych	02, rozdz. 8.1
Przypadek testowy	02, rozdz. 8
Regresja kodu (zestaw regresyjny)	02, rozdz. 4.7
Regresja wizualna (GUI)	02, rozdz. 6.5
Rekomendacje (luki, naprawy)	02, rozdz. 5.6 i 7.5
RODO i prywatność	04, rozdz. 14.4
Rola (Użytkownik, Właściciel)	02, rozdz. 3.4; 04, rozdz. 13.2
SAST	02, rozdz. 7.1
SBOM (CycloneDX, SPDX)	02, rozdz. 7.4
Skan zależności (CVE)	02, rozdz. 7.4
Status check (pull request)	03, rozdz. 10.6
Statusy przypadku (Szkic, Gotowy…)	02, rozdz. 8.3
Synchronizacja dwukierunkowa z TMS	03, rozdz. 10.9
Szkielet kodu (skeleton)	02, rozdz. 5.6
Taint analysis	02, rozdz. 7.1
TestRail, Xray, Zephyr (integracja)	03, rozdz. 10.8
Troubleshooting	04, rozdz. 16
Uprawnienia i zespoły	02, rozdz. 3.4
Użytkownicy firmy (zapraszanie)	04, rozdz. 13.2
Webhooki wychodzące	03, rozdz. 11.8
Wsparcie techniczne	04, rozdz. 17

Termin	Gdzie
Wymagania (funkcjonalne, niefunkcjonalne)	02, rozdz. 5.3
Zrzut stanu (snapshot zgodności)	02, rozdz. 5.6
Zrzuty ekranu GUI (analiza)	02, rozdz. 6.3

Glosariusz TestGEN

> Wspólny słownik pojęć dokumentacji końcowej — scalony z glosariuszy modułowych (bezpieczeństwo, luki funkcjonalne, interfejs webowy, billing) i uzupełniony o terminy architektury. Terminy techniczne pozostają w angielskim oryginale, definicje po polsku.

A — analiza, integracje

Termin	Definicja
Active scan	Faza ZAP wstrzykująca payloady ataków do odkrytych punktów wejścia.
ADR	Architecture Decision Record — zapis decyzji architektonicznej z kontekstem i konsekwencjami (`architecture/adr/`).
ADO (Azure DevOps)	Platforma Microsoft do zarządzania cyklem życia oprogramowania; moduł integracji synchronizuje znaleziska jako Work Items.
AOT	Ahead-of-Time compilation — produkcyjna kompilacja szablonów Angulara; łapie błędy typów niewidoczne w testach.
Alembic	Narzędzie migracji schematu bazy danych dla SQLAlchemy (`backend/alembic/versions/`).
Analiza regresji	Porównanie wyników analizy między uruchomieniami — wykrywa, co się zmieniło w kodzie/dokumentacji.
Angular	Framework frontendowy (v21, komponenty standalone, Material, signals) — warstwa UI platformy.
AnalysisResult	Rekord wyniku uruchomienia modułu analizy (`analysis_results`); np. `module="requirements"` , `module="logic_structures"` .
ARIA	Atrybuty dostępności (role, etykiety, stany) dla technologii wspomagających.
Argon2id	Algorytm haszowania haseł (wybór ADR-003).
Audit log	Dziennik operacji wrażliwych (kto, co, kiedy, tenant) — retencja 1 rok.
axe-core	Silnik automatycznego audytu dostępności; w CI próg: 0 naruszeń critical/serious (WCAG 2.1 A/AA).
Azure DevOps	patrz ADO.

B — billing

Termin	Definicja
Beat (Celery beat)	Harmonogram zadań okresowych (odnowienia 03:30, snapshoty 00:00, progi co godzinę).
BLIK	Metoda płatności mobilnej w Polsce; w TestGEN z Płatnościami Powtarzalnymi (alias).

Termin	Definicja
Checkout	Przepływ zakupu: utworzenie transakcji u tpay i przekierowanie na Panel Transakcyjny.
CodeEntity	Ustrukturyzowana reprezentacja bytu kodu (funkcja/klasa): `entity_id`, `name`, `type`, `file_path`, `language`, `docstring`.
Compliance score	Ocena 0–100, jak dobrze kod spełnia dokumentację; wyliczana przez `ComplianceEngine` z `gap_findings`.
Compliance snapshot	Punkt w czasie z compliance score — podstawa porównań wersji.
Confidence	Zaufanie do znaleziska/rekomendacji (0.0–1.0); zależy od narzędzia i potwierdzeń; < 0.5 = weryfikacja człowieka.
Context (luki)	Fragment wymagania + kodu dołączany do znaleziska — pomaga zrozumieć źródło luki.
CSRF	Cross-Site Request Forgery — wymuszenie niezamierzonej operacji (CWE-352).
CVE	Common Vulnerabilities and Exposures — publiczny identyfikator znanej podatności.
CVSS	Common Vulnerability Scoring System — ocena powagi 0.0–10.0 (wersja 3.1).
CVE pipeline	Miesięczna aktualizacja bazy CVE z NVD/OSV/GitHub Advisory + auto-reguły SAST.
CWE	Common Weakness Enumeration — klasyfikacja typów słabości (np. CWE-89 = SQLi).

D—F

Termin	Definicja
DAST	Dynamic Application Security Testing — analiza uruchomionej aplikacji (OWASP ZAP).
Deep link	Link `testgen://ścieżka` otwierający konkretny ekran w aplikacji natywnej.
Diff viewer	Widok porównania dwóch wersji obok siebie (kod↔dokumentacja, porównania analiz).
Dunning	Procedura ponowień po nieudanym obciążeniu odnowieniowym; kończy się downgrade do Freemium.
Drag & drop	Przeciąganie plików na obszar uploadu (wspólny `file-upload.component`).
Effort	Szacowany koszt naprawy: S/M/L (luki) lub low/medium/high (bezpieczeństwo).
Event_uid	Deterministyczny identyfikator notyfikacji tpay — podstawa idempotencji webhooków.
Extra	Typ luki: kod nie ma odpowiadającego wymagania w dokumentacji; severity info.

Termin	Definicja
False positive (FP)	Fałszywy alarm — znalezisko, które nie jest realną podatnością.
FastAPI	Framework backendowy (Python) — REST API `/api/v1` z automatycznym OpenAPI.
Feature gating (402)	Blokada modułu analizy poza planem — widoczna-zablokowana w UI, HTTP 402 w API (FR71).
Finding	Pojedynczy rekord luki w `gap_findings` (lub zagrożenia w `security_findings`).
Faktura VAT	Automatyczna faktura po płatności: numer `FV/rok/miesiąc/seq`, VAT 23%, snapshoty stron, PDF w MinIO + email.
Fernet	Szyfrowanie poświadczeń integracji (Jira/ADO) w `integration_configs`.
FindingStatus	Status znaleziska: OPEN / ACKNOWLEDGED / FIXED / FALSE_POSITIVE.
Flutter	Framework aplikacji natywnych — powłoka WebView (mobile/desktop) dla frontendu webowego.
FR38–FR42	Wymagania funkcjonalne Epic 5: dopasowanie wymagań, 4 typy luk, compliance, detekcja błędów, rekomendacje.

G—I

Termin	Definicja
GapClassifier	Serwis klasyfikujący wynik `ComparatorService` w 4 typy luk + severity.
Golden data	Korpus referencyjny (wejście + oczekiwany wynik) do testów dokładności analiz (ADR-008).
Grace (7 dni)	Okres łaski po dunningu — limit płatnego planu jeszcze obowiązuje.
Guard	Ochrona tras Angulara: `auth.guard` (zalogowanie), `role.guard` (rola).
Hardcoded secret	Sekret zakodowany na stałe w kodzie (CWE-798).
HttpOnly cookie	Ciasteczko sesyjne niedostępne dla JavaScript — nośnik uwierzytelnienia.
IDOR	Insecure Direct Object Reference — dostęp do obiektu po ID bez kontroli właściciela (CWE-639).
Idempotencja	Gwarancja, że powtórzona operacja nie wywoła podwójnych skutków (webhooki, faktury, seed).
Inconsistent	Typ luki: byt kodu mapowany do sprzecznych wymagań; severity critical.
Interceptor	Przetwarzanie żądań HTTP w Angularze: auth (token, 401), snake-case (konwersja pól JSON).
IntegrationProvider	Enum dostawcy integracji: `jira` | `azure_devops`.
IntegrationSyncLog	Wpis historii synchronizacji z zewnętrznym trackerem.

J—M

Termin	Definicja
Jaro-Winkler	Algorytm podobieństwa napisów (favoruje wspólny prefiks) — jeden z 3 algorytmów dopasowania luk.
Jinja2	Silnik szablonów (ADR-006): generowanie kodu testów, skeletonów, raportów, PDF faktur.
Jira	Tracker Atlassian; integracja synchronizuje luki jako issues (skeleton w załączniku).
JWS	JSON Web Signature — podpis notyfikacji tpay w nagłówku `X-JWS-Signature` (detached payload).
JWT RS256	Tokeny uwierzytelniające podpisywane asymetrycznie (klucz prywatny/publiczny RSA; ADR-003).
Kolejki Celery	`default`, `analysis`, `export`, `security` — routing zadań per moduł.
Lazy loading	Ładowanie kodu ekranu dopiero przy wejściu na trasę (`loadComponent` / `loadChildren`).
Levenshtein	Odległość edycyjna napisów — algorytm dopasowania wymagań do bytów kodu.
Liczniki zużycia	`analyses`, `test_cases`, `storage_bytes`, `files_per_analysis` — egzekwowane per plan.
Live region (aria-live)	Region ogłaszający zmiany czytnikom ekranu bez zmiany fokusu.
Luka (gap)	Rozbieżność między wymaganie a implementacją: missing / different / extra / inconsistent.

N—P

Termin	Definicja
Missing	Typ luki: wymaganie istnieje, kod nie zawiera odpowiadającego bytu.
MinIO	Object storage (S3-compatible): pliki projektów, wizerunki, faktury (bucket `testgen-invoices`).
Monorepo	Jedno repozytorium: `backend/` + `frontend/` + `mobile/` (ADR-001).
Multi-tenancy	Izolacja danych firm (tenantów) — wiersze z `tenant_id` + polityki RLS (ADR-002).
MFA / TOTP	Drugi składnik uwierzytelnienia — kody jednorazowe na podstawie czasu (opcjonalny).
NVD / OSV	Bazy podatności: NIST National Vulnerability Database / Open Source Vulnerabilities.
Okres rozliczeniowy	Miesięczne okno subskrypcji; w jego granicach liczone są limity.

Termin	Definicja
OpenAPI	Specyfikacja REST API generowana z FastAPI (`/openapi.json` , Swagger UI `/docs`).
OWASP Top 10	Lista najczęstszych ryzyk aplikacji webowych (2021) — kategorie znalezisk bezpieczeństwa.
Owner / User / Admin	Role: Owner zarządca tenanta, User użytkownik, Admin platforma.
Panel Transakcyjny	Strona tpay, gdzie płatnik wybiera metodę (BLIK/karta/przelew) i autoryzuje płatność.
PaymentToken	Zapisana metoda płatności: token karty lub alias BLIK (szyfrowany per-tenant).
Plan (Freemium/Basic/Premium)	Poziom subskrypcji decydujący o limitach i dostępnych modułach analizy.
Playwright	Framework testów E2E frontendu (`frontend/e2e/`).
PoC	Proof of Concept — bezpieczny dowód istnienia podatności (filtr `poc_safety`).
Proration	Rozliczenie proporcjonalne przy upgrade: dopłata = różnica cen × pozostałe dni okresu.
Progi powiadomień	80 / 90 / 100% limitu — powiadomienie in-app + email, dedup per okres.
Prognoza limitu	Ekstrapolacja dziennego zużycia (≥7 dni danych) — ostrzeżenie przed wyczerpaniem limitu.
Priority	Kolejność naprawy (P1 najwyższy dla luk; 1 najwyższy dla bezpieczeństwa) — wyliczana przez risk engine.
Prometheus	Metryki (m.in. `webhook_processed_total`) + alerty operacyjne.
Raport dwupoziomowy	Executive summary (zarząd) + szczegóły techniczne (QA) z jednego zbioru wyników (FR44).

R—S

Termin	Definicja
RabbitMQ	Broker komunikatów dla Celery (ADR-004).
Reconciliation	Jednorazowe dopytanie tpay o status transakcji, gdy lokalny status zbyt długo wisi na „oczekująca".
Redis	Cache + backend wyników Celery + postęp zadań.
Rekomendacja	Actionable sugestia naprawy luki/podatności: priority, confidence, akcja, effort.
Replay webhooka	Ponowne przetwarzanie zapisanej notyfikacji tpay przez admina (rozjazd stanu po awarii).
Requirement	Wymaganie wyciągnięte z dokumentacji (ID np. REQ-ANL-004) — podstawa porównania z kodem.

Termin	Definicja
RLS	Row Level Security — izolacja wierszy per tenant w PostgreSQL; admin ma politykę bypass.
Sandbox tpay	Środowisko testowe tpay (kwoty sterujące scenariuszami) — runbook w troubleshooting.
Sanitizer	Funkcja neutralizująca skażenie dla określonego typu sinka (`html.escape` chroni HTML, nie SQL).
SAST	Static Application Security Testing — analiza statyczna kodu (reguły wzorców).
SBOM	Software Bill of Materials — inwentarz komponentów (CycloneDX, SPDX).
SCA	Software Composition Analysis — analiza zależności pod kątem znanych podatności.
Severity	Powaga: CRITICAL / HIGH / MEDIUM / LOW / INFO.
Signals	Reaktywny stan Angulara (`signal()`, `computed()`) — podstawa stanu UI.
Sink	Niebezpieczna operacja na danych (np. `cursor.execute`, `eval`).
Skeleton code	Stub implementacji dla luki `missing` (Jinja2 + tree-sitter; `NotImplementedError`).
Skip-link	Link „Przejdź do treści" — pierwszy fokus strony (WCAG 2.4.1).
Source	Punkt wejścia niezaufanych danych (np. `request.GET`, `argv`).
Source (znaleziska)	Silnik, który wykrył znalezisko/rekomendację.
Spider	Faza ZAP odkrywająca powierzchnię ataku (URL-e, formularze).
SQLi	SQL Injection — wstrzyknięcie do zapytania SQL (CWE-89).
Standalone component	Komponent Angular bez NgModule — sam deklaruje zależności.
Snapshot danych fakturowych	Kopia danych sprzedawcy/nabywcy w fakturze w chwili wystawienia — niezmienna.
Taint analysis	Śledzenie przepływu niezaufanych danych od source do sink.
Tenant	Firma (organizacja) — jednostka rozliczeniowa i izolacji danych.

T—Ż

Termin	Definicja
TF-IDF cosine	Podobieństwo wektorów term-frequency — trzeci algorytm dopasowania luk.
TMS	Test Management System (TestRail, Xray, Zephyr) — cel eksportu test case'ów (CSV/XML).
Tokenizacja karty	Zapis karty u tpay bez obciążenia — platforma dostaje token zamiast numeru.

Termin	Definicja
tpay	Jedyny provider płatności (decyzja CR 2026-08-28): BLIK, karta, przelew; webhooki JWS.
Traceability	Powiązanie wymaganie ↔ test ↔ kod — macierz pokrycia (E3).
Tree-sitter	Parser AST wielu języków — analiza kodu i walidacja skeletonów (ADR-005).
Troubleshooting	Sekcja runbooków i rozwiązywania problemów (`troubleshooting/`).
Unified diff	Format patcha (`git apply`) z kontekstem i numerami linii.
Vitest / ng test	Runner testów komponentowych frontendu (builder `@angular/build:unit-test`).
WCAG 2.1 AA	Web Content Accessibility Guidelines — standard dostępności całego UI.
Webhook	Notyfikacja HTTP od tpay o zdarzeniu płatności — podpisana JWS; aktywuje plan.
WebView	Osadzona przeglądarka w aplikacji natywnej — sposób działania powłoki Flutter.
Widoczne-zablokowane	Zasada FR71: moduł poza planem widoczny z kłódką (nie ukryty), backend odpowiada 402.
Wysoki kontrast	Motyw czarno-biały z kontrastem tekstu ≥ 7:1 (AAA) — klasa `high-contrast` .
XSS	Cross-Site Scripting — wstrzyknięcie skryptu do wyjścia HTML (CWE-79).
Zaplanowany downgrade	Zmiana na niższy plan skuteczna na koniec bieżącego okresu (lokalna, bez zwrotów).
Zasoby (usage)	Liczniki zużycia per tenant — dashboard, historia, prognoza.
md5sum	Dodatkowa suma kontrolna notyfikacji transakcyjnych tpay.